



SYNTHETIC MOONQUAKE GENERATOR

USER MANUAL

Version 1.0

OSCAR ARCILA
DANIEL GÓMEZ
ALEJANDRO CRUZ

June 18, 2024

Contents

1	Introduction	5
2	Main Window	5
2.1	File Menu	5
2.2	? (Help) Menu	5
3	Synthetic Accelerogram Tab	7
3.1	White Noise Panel	7
3.2	Band-Pass Filter Panel	8
3.3	Temporal Envelope Panel	10
3.4	PGA Panel	12
3.5	Synthetic Accelerogram Plot Panel	12
3.6	Synthetic Accelerogram Export Results	13
4	Response Spectrum Tab	14
4.1	Damping Ratio Panel	14
4.2	Response Spectrum Periods Panel	15
4.3	Gravitational Acceleration Panel	15
4.4	<i>SpecMQ</i> Function for the calculus of seismic response spectra	15
4.5	Response Spectrum Plot Panel	16
4.6	Response Spectrum Export Results	17
5	Code Review	19
5.1	Code Repository	19
5.2	Source Code Analysis	22
5.2.1	Entry Point of the <i>Synthetic Moonquake Generator</i> application	23
5.2.2	Main Figure of the <i>Synthetic Moonquake Generator</i> Application	29
5.2.3	White Noise Panel	46
5.2.4	Band-Pass Filter Panel	53
5.2.5	Temporal Envelope Panel	71
5.2.6	PGA Panel	96
5.2.7	Synthetic Accelerogram plot Panel	97
5.2.8	Export Synthetic Accelerogram Figures Button	99
5.2.9	Export Synthetic Accelerogram Button	101
5.2.10	Damping Ratio Panel	102
5.2.11	Response Spectrum Periods Panel	103
5.2.12	Gravitational Acceleration Magnitude Panel	105
5.2.13	<i>SpecMQ</i> Function for the calculus or the seismic response spectra	107
5.2.14	Response Spectrum plot Panel	108
5.2.15	Export Response Spectrum Figures Button	109
5.2.16	Export Response Spectrum Button	110
6	Compilation	112
7	Requirements and Installation	116
7.1	Hardware Requirements	116
7.2	Software Requirements	116
7.3	Installation Process	116
7.4	File Association	118
8	License Agreement	119
8.1	Attribution	120
8.2	Attribution-ShareAlike 4.0 International	120

List of Figures

1	Main window of the <i>Synthetic Moonquake Generator</i> software	6
2	<i>Menu bar</i> and tabs <i>Synthetic Accelerogram</i> and <i>Response Spectrum</i>	6
3	<i>File menu</i> and its options	6
4	? (<i>Help</i>) <i>Menu</i> and its options	7

5	Window opened by the option <i>About</i>	7
6	<i>Synthetic Accelerogram</i> tab	8
7	<i>White Noise</i> Panel	8
8	Tab <i>Observed Values</i> of the panel <i>Band-Pass Filter</i>	9
9	Tab <i>User Values</i> of the panel <i>Band-Pass Filter</i>	10
10	Figure describing the components and equations of the temporal envelope of the <i>Temporal Envelope</i> panel. Adapted from Clough and Penzien, 2003.	10
11	<i>Observed Values</i> tab of the <i>Temporal Envelope</i> panel	11
12	<i>User Values</i> tab of the <i>Temporal Envelope</i> panel	12
13	<i>Peak Ground Acceleration</i> panel of the <i>Synthetic Accelerogram</i> tab	12
14	<i>Plot Panel</i> of the <i>Synthetic Accelerogram</i> Tab	13
15	Controls embedded in each figure of the <i>Plot Panel</i> of the <i>Synthetic Moonquake Generator</i> application	13
16	<i>Export Figures</i> button of the <i>Synthetic Accelerogram</i> tab	14
17	<i>Export Accelerogram</i> button of the <i>Synthetic Accelerogram</i> tab	14
18	<i>Damping Ratio</i> panel of the <i>Response Spectrum</i> tab	14
19	<i>Response Spectrum Periods</i> panel of the <i>Response Spectrum</i> tab	15
20	<i>Gravitational Acceleration</i> panel of the <i>Response Spectrum</i> tab	15
21	<i>Plot</i> panel of the <i>Response Spectrum</i> tab	17
22	<i>Export Figures</i> button of the <i>Response Spectrum</i> tab	17
23	<i>Export Spectra</i> button of the <i>Response Spectrum</i> tab	18
24	Code repository of <i>Synthetic Moonquake Generator</i> application	20
25	Contents of the Source Code folder of the repository - (Part 1 of 2)	21
26	Contents of the Source Code folder of the repository - (Part 2 of 2)	22
27	Executable icon: <i>SMG.exe</i>	116
28	Option <i>Run as administrator</i> of the right click menu	116
29	First screen of the installer application	116
30	Install options	117
31	Required software	117
32	License Agreement for Matlab Runtime	117
33	Confirmation	118
34	Install complete	118
35	Right-click menu of the file <i>*.smg</i>	118
36	<i>Choose an app on your PC</i> option	119
37	<i>SMG.exe</i> file on the install directory	119
38	Final step of the file association	119

List of Code Listings

1	Algorithm used by the <i>Synthetic Moonquake Generator</i> application for the band-limited white noise signal and its PSD calculation	8
2	Algorithm used by the <i>Synthetic Moonquake Generator</i> application for the Pass-Band filter calculation and application in the <i>Observed Values</i> tab of the <i>Band-Pass Filter</i> panel	9
3	Algorithm used by the <i>Synthetic Moonquake Generator</i> application for the calculus of the Pass-Band filter in the <i>User Values</i> tab of the panel <i>Band-Pass Filter</i>	9
4	Algorithm used by the application <i>Synthetic Moonquake Generator</i> for the calculus of the temporal envelope on the <i>Observed Values</i> tab of the <i>Temporal Envelope</i> panel	11
5	Algorithm used by the application <i>Synthetic Moonquake Generator</i> for the calculus of the temporal envelope on the <i>User Values</i> tab of the <i>Temporal Envelope</i> panel	12
6	Initial part of a Synthetic Moonquake Accelerogram Record exported by the <i>Synthetic Moonquake Generator</i> application.	14
7	Algorithm used by the <i>Synthetic Moonquake Generator</i> application to calculate the periods of the <i>Response Spectrum Periods</i> panel	15
8	<i>SpecMQ.m</i> : Function for the calculus of the seismic response spectra of the tab <i>Response Spectrum</i>	16
9	Initial part of a <i>Synthetic Moonquake Response Spectrum</i> file exported by the <i>Synthetic Moonquake Generator</i> application	18
10	<i>SMG.m</i> : Entry point of the <i>Synthetic Moonquake Generator</i> application	27
11	<i>Main.m</i> : Main window of the <i>Synthetic Moonquake Generator</i> application	32
12	<i>OpenFileCallBack.m</i> : Callback for the <i>Open...</i> option of the <i>File</i> menu	40

13	<i>SaveFileCallBack.m</i> : Callback for the <i>Save As...</i> option of the <i>File</i> menu	42
14	<i>UserManualCallBack.m</i> : Callback for the <i>User Manual</i> option of the ? (<i>Help</i>) menu	42
15	<i>FileAssociaionCallBack.m</i> : Callback for the <i>File Associaion</i> option of the ? (<i>Help</i>) menu	43
16	<i>LicenseAgreementCallBack.m</i> : Callback for the <i>License Agreement</i> option of the ? (<i>Help</i>) menu	43
17	<i>AboutCallBack.m</i> : Callback for the <i>About</i> option of the ? (<i>Help</i>) menu	44
18	<i>MtgCallBack.m</i> : Callback for the tab change of the main window	45
19	<i>GPpanel.m</i> : <i>White Noise</i> panel of the <i>Synthetic Accelerogram</i> tab	47
20	<i>GPCallBack.m</i> : Callback for the <i>White Noise</i> panel of the <i>Synthetic Accelerogram</i> tab	51
21	<i>FPpanel.m</i> : <i>Band-Pass Filter</i> panel of the <i>Synthetic Accelerogram</i> tab	57
22	<i>FPTgCallBack.m</i> : Callback for the change of tab in the <i>Band-pass Filter</i> panel	62
23	<i>FPCallBack1.m</i> : Callback for the slider controls of the <i>Observed Values</i> tab in the <i>Band-Pass Filter</i> panel	65
24	<i>FPCallBack2.m</i> : Callback for the editable text box controls of the <i>Observed Values</i> tab in the <i>Band-Pass Filter</i> panel	67
25	<i>FPCallBack3.m</i> : Callback for the editable text box controls of the <i>User Values</i> tab in the <i>Band-Pass Filter</i> panel	70
26	<i>TPpanel.m</i> : <i>Temporal Envelope</i> panel of the <i>Synthetic Accelerogram</i> tab	77
27	<i>imgzoompan.m</i> : Function that implements panning and zooming of the <i>Temporal Envelope</i> panel image	84
28	<i>TPtgCallBack.m</i> : Callback for the change of tab in the <i>Temporal Envelope</i> panel	84
29	<i>RadioButtonCallBack.m</i> : Callback of the radio button group of the <i>Temporal Envelope</i> panel	87
30	<i>TPCallBack1.m</i> : Callback for the slider controls of the <i>Observed Values</i> tab in the <i>Temporal Envelope</i> panel	89
31	<i>TPCallBack2.m</i> : Callback for the editable text box controls of the <i>Observed Values</i> tab in the <i>Temporal Envelope</i> panel	93
32	<i>TPCallBack3.m</i> : Callback for the editable text box controls of the <i>User Values</i> tab in the <i>Temporal Envelope</i> panel	95
33	<i>PGApanel.m</i> : <i>Peak Ground Acceleration</i> panel of the <i>Synthetic Accelerogram</i> tab	97
34	<i>PGACallBack.m</i> : Callback for the editable text box control of the <i>Peak Ground Acceleration</i> panel	97
35	<i>PlotAcc.m</i> <i>Plot</i> panel of the <i>Synthetic Accelerogram</i> tab	99
36	<i>ExportAccFig.m</i> : <i>Export Figures</i> button of the <i>Synthetic Accelerogram</i> tab	99
37	<i>ExportAccFigCallBack.m</i> : Callback for the <i>Export Figures</i> button of the <i>Synthetic Accelerogram</i> tab	100
38	<i>ExportAcc.m</i> : <i>Export Accelerogram</i> button of the <i>Synthetic Accelerogram</i> tab	101
39	<i>ExportAccCallBack.m</i> : Callback for the <i>Export Accelerogram</i> button of the <i>Synthetic Accelerogram</i> tab	102
40	<i>ZZpanel.m</i> : <i>Damping Ratio</i> panel of the <i>Response Spectrum</i> tab	102
41	<i>ZZCallBack.m</i> : Callback for the editable text box control of the <i>Damping Ratio</i> panel	103
42	<i>TNpanel.m</i> : <i>Response Spectrum Periods</i> panel of the <i>Response Spectrum</i> tab	104
43	<i>TNCallBack.m</i> : Callback for the editable text box controls of the <i>Response Spectrum Periods</i> panel	105
44	<i>GGpanel.m</i> : <i>Gravitational Acceleration Magnitude</i> panel of the <i>Response Spectrum</i> tab	106
45	<i>GGCallBack.m</i> : Callback for the editable text box controls of the <i>Gravitational Acceleration Magnitude</i> panel	106
46	<i>SpecMQ.m</i> : Function for the calculus or the seismic response spectra	108
47	<i>PlotSpec.m</i> : <i>Plot</i> panel of the <i>Response Spectrum</i> tab	108
48	<i>ExportSpecFig.m</i> : <i>Export Figures</i> button of the <i>Response Spectrum</i> tab	109
49	<i>ExportSpecFigCallBack.m</i> : Callback for the <i>Export Figures</i> button of the <i>Response Spectrum</i> tab	110
50	<i>ExportSpec.m</i> : <i>Export Spectra</i> button of the <i>Response Spectrum</i> tab	110
51	<i>ExportSpecCallBack.m</i> : Callback for the <i>Export Spectra</i> button of the <i>Response Spectrum</i> tab	111

List of Hierarchy Diagrams

1	Main window	33
2	File Menu	34
3	Help menu	35
4	Tab group	36
5	White Noise Panel	48
6	Band-Pass Filter Panel	57
7	Observed Values tab of the Band-Pass Filter Panel	58
8	User Values tab of the Band-Pass Filter Panel	58
9	Temporal Envelope Panel	77
10	Observed Values tab of the Temporal Envelope Panel	78
11	User Values tab of the Temporal Envelope Panel	79

List of Flowcharts

1	Entry point of the <i>Synthetic Moonquake Generator</i> application	27
2	<i>Synthetic Moonquake Generator</i> Start Without Arguments	28
3	<i>Synthetic Moonquake Generator</i> Start With Arguments	29
4	Main figure of the <i>Synthetic Moonquake Generator</i> aplication	37
5	File/Open menu CallBack	41
6	File/Save menu CallBack	42
7	Help/User Manual menu CallBack	43
8	Help/File Association menu CallBack	43
9	Help/License Agreement menu CallBack	43
10	Help/About menu CallBack	45
11	Main figure Tab Change CallBack	46
12	White Noise Panel	48
13	White Noise Panel CallBack	52
14	Band-Pass Filter Panel	59
15	Observed Values tab of the Band-Pass Filter Panel	60
16	User Values tab of the Band-Pass Filter Panel	61
17	Callback for the change of tab in the <i>Band-pass Filter</i> panel	63
18	Callback for the slider controls of the <i>Observed Values</i> tab in the <i>Band-Pass Filter</i> panel	65
19	Callback for the editable text box controls of the <i>Observed Values</i> tab in the <i>Band-Pass Filter</i> panel	68
20	Callback for the editable text box controls of the <i>User Values</i> tab in the <i>Band-Pass Filter</i> panel	71
21	Temporal Envelope Panel	79
22	Observed Values tab of the Temporal Envelope Panel	80
23	User Values tab of the Temporal Envelope Panel	81
24	Callback for the change of tab in the Temporal Envelope Panel	85
25	Callback for the radio button group in the Temporal Envelope Panel	88
26	Callback for the sliders controls	90
27	Callback for the text edit controls of the Observed Values tab	93
28	Callback for the text edit controls of the User Values tab	96

1 Introduction

Synthetic Moonquake Generator is a sophisticated software application designed to generate synthetic lunar quakes (moonquakes), specifically adapted to reproduce the characteristics of surface-type lunar seism recorded by long-period, horizontally oriented sensors of the Apollo network. The software employs a modified Kanai-Tajimi methodology to ensure accuracy and authenticity in the simulation of these seismic events. This makes it possible for anyone to calculate the accelerogram of a synthetic moonquake and its response spectra.

The application was developed in the “MATLAB” language, version “2022A”. It is written using a “figure”, and its compilation to an executable was done with the help of the “MATLAB Compiler”. The application consists of, “43” source code files (‘.m’), three image files, and three ‘.pdf’ files. Additionally, there are auxiliary files for compilation, including six image files, one text file, and three ‘.pdf’ files.

Regarding the hardware used for development, a personal laptop with an “Intel Core i5 10th generation” processor, “20 GB DDR4 RAM @ 3200MHz”, and a “500GB NVME M.2 PCIe3 solid-state drive” was employed. Other specifications are typical for a general consumer-grade machine.

The application consists of a main menu and two main tabs from which you can access the various controls to configure the parameters for the calculations and obtain the results. To properly use the application, any order of action can be followed. However, the logical order of applying the Kanai–Tajimi algorithm and generating the response spectra is fundamental for the software operation.

First, it is suggested that the user interacts with the *Synthetic Accelerogram* tab, where he/she can define the characteristics of the base white noise, design a filter in frequency and time window, apply them, and scale the record to a desired PGA level. Each time a change is made the application will automatically adjust and recalculate the results and display them in the graph region.

Additionally, there is a tab *Response Spectrum*, where the seismic response spectra can be calculated. As in the previous tab, the operations can be performed in any order. However, a suggested logical order involves defining the desired damping ratio, range and number of periods, and the magnitude of gravitational acceleration. The response spectra will be calculated automatically after each change and displayed in the graph region.

The software can export the numerical and graphical results of both the accelerogram and the response spectra and save the environment for later use. Once the installation process is complete, you can associate the file extension so that double-clicking on a project file opens the application and loads the values associated with the project. In the following, we will describe the graphical interface and the internal processes of calculating synthetic lunar accelerograms and seismic response spectra with this application.

2 Main Window

The main window contains the application logo, the *menu bar*, and the tabs *Synthetic Accelerogram* and *Response Spectrum* (Fig. 1). Initially, the contents of the *Synthetic Accelerogram* tab are displayed, however, the *Response Spectrum* tab can be chosen freely since a synthetic sample accelerogram and its response spectra are already calculated at the start of the application.

A detail of the menu bar and the tabs *Synthetic Accelerogram* and *Response Spectrum* is shown in Fig. 2. where the menus *File* and *? (Help)* can be seen.

2.1 File Menu

The *File* menu contains a total of three items: *Open...* which allows opening a saved working environment with a custom extension *.smg, *Save As...* allows to save the working environment with all values, graphics, and settings under a file with custom extension *.smg and *Close* (this command as usual closes the application). Each of the above commands has a keyboard shortcut for its invocation. Fig. 3 shows a detail of the *File* menu.

2.2 ? (Help) Menu

The *? (Help)* menu contains a total of four items:

- *User Manual* Displays the user manual for the application.

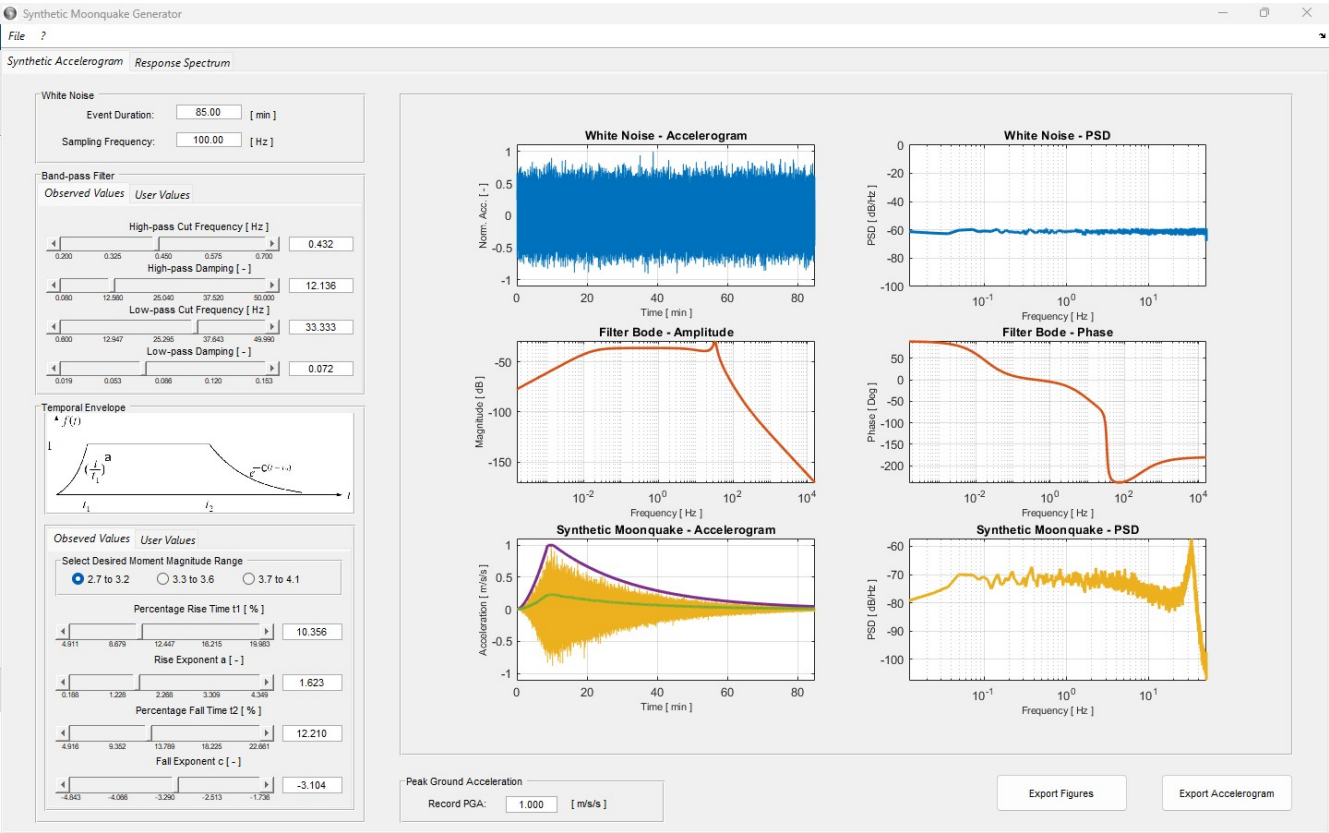


Figure 1: Main window of the *Synthetic Moonquake Generator* software

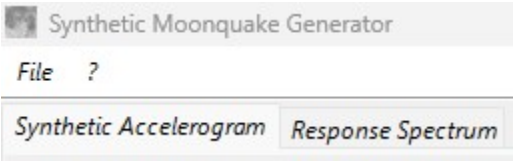


Figure 2: *Menu bar* and tabs *Synthetic Accelerogram* and *Response Spectrum*

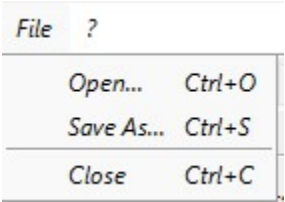


Figure 3: *File menu* and its options



Figure 4: ? (*Help*) Menu and its options

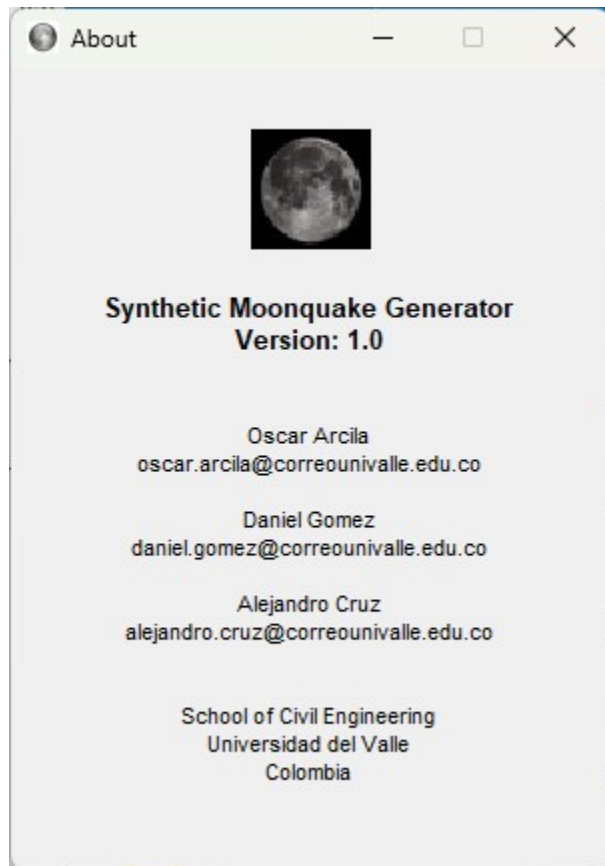


Figure 5: Window opened by the option *About*

- *File Association* Provides instructions for associating the file format *.smg* with the application.
- *License Agreement* Presents a document covering all relevant details related to the license agreement.
- *About* Opens a window with information about the software, including version details, developers, and affiliation with the School of Civil Engineering and Geomatics at Universidad del Valle.

Figure 4 shows a detailed view of the expanded ? (*Help*) menu, while Figure 5 displays the window opened by the *About* option.

3 Synthetic Accelerogram Tab

The application starts within the *Synthetic Accelerogram* tab shown in Fig. 6, where it can be seen that consists of the panels *White Noise* where we can control the characteristics of the white noise to be used, *Band-pass Filter* which allows us to design and apply a band-pass filter in frequency, *Temporal Envelope* where we can design the temporal envelope of the accelerograms, *PGA* that allows to scale the records, a panel where several graphs related to the synthetic accelerogram generation process are displayed and two buttons that allow exporting the results both numerically and graphically.

3.1 White Noise Panel

The interface is designed so that the user can follow any order of calculation, however there is a suggested order. First, the user can define the duration and sampling rate characteristics of the band-limited white noise to be employed using the editable text boxes in the *White Noise* panel

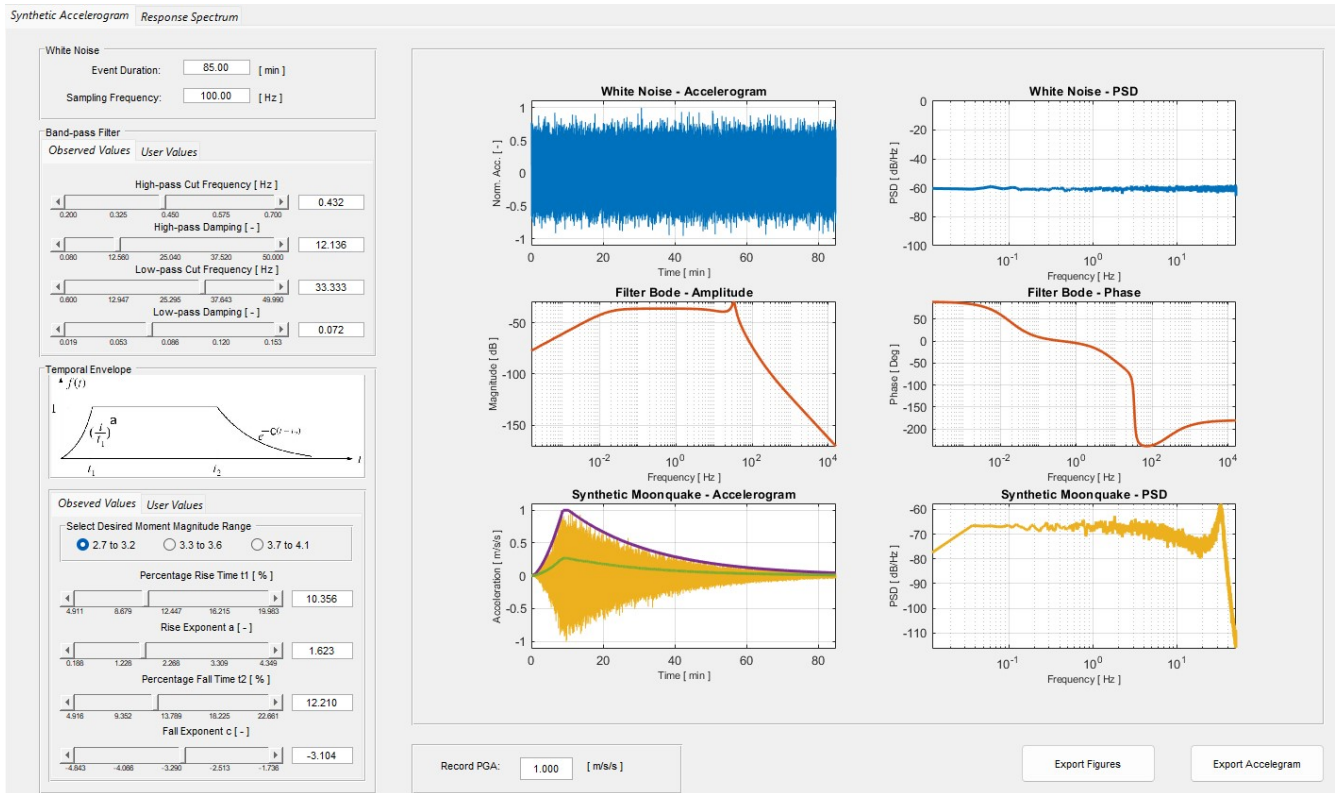


Figure 6: *Synthetic Accelerogram* tab



Figure 7: *White Noise* Panel

(Fig. 7). The white noise signal generation algorithm is based on the instructions in the Code 1 where the band-limited white noise generation process is described, along with the instructions necessary for the Power Spectra Density (PSD) calculation. In this Code, $GP.fs$ and $GP.T$ make reference to the sampling frequency of the record measured in Hz and the length of the record measured in min . While $GP.NGenAcc$, $GP.psd$ and $GP.ff$ correspond to the normalized band-limited white noise signal measured in $m/s/s$ and to the PSD amplitude and phase components of the signal. The signal length is also limited to an even number of elements due to the synthetic moonquake generation process requirements.

```

1 GP.tt = 1/GP.fs*(0:GP.T*60*GP.fs).';
2 if (mod(length(GP.tt),2)==1)
3     GP.tt(end) = [];
4 end
5 GP.RndAcc = randn(1,length(GP.tt));
6 GP.NGenAcc = GP.RndAcc/max(abs(GP.RndAcc));
7 GP.nfft = 2^(nextpow2(length(GP.tt))-6);
8 GP.noverlap = GP.nfft/2;
9 GP.window = GP.nfft;
10 [GP.psd,GP.ff] = pwelch(GP.NGenAcc,GP.nfft,GP.noverlap,GP.window,GP.fs);

```

Code 1: Algorithm used by the *Synthetic Moonquake Generator* application for the band-limited white noise signal and its PSD calculation

3.2 Band-Pass Filter Panel

The next logical step consists of the design and application of the filter in frequency, this can be done in two ways, the first of these is through the tab *Observed Values*, of the panel *Band-Pass Filter*, which allows to generate a band-pass filter composed by a low-pass filter and a high-pass filter both of order two, which correspond closely with the characteristics observed in the frequency domain of the real moonquakes. Both filters are defined in terms of their cutoff frequency and damping ratio. The values of these four parameters can be modified using the sliders or with

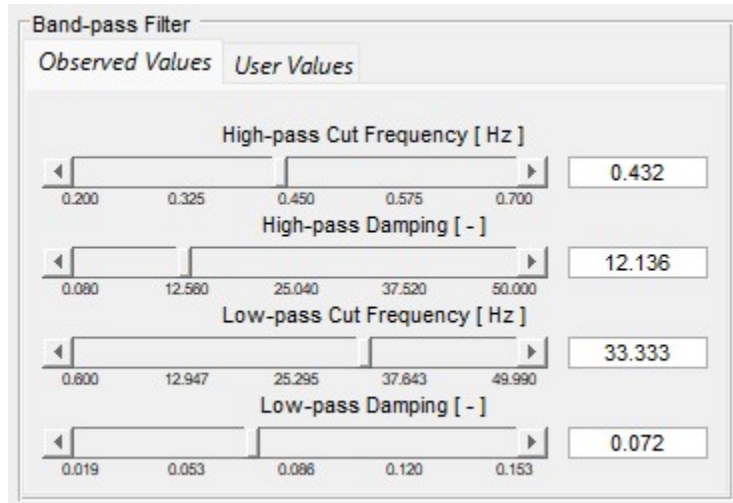


Figure 8: Tab *Observed Values* of the panel *Band-Pass Filter*

the editable text boxes in front of each slider control (Fig. 8). The Band-Pass filter generation algorithm of the tab *Observed Values* is based on the instructions in the Code 2 where the filter construction process is described along with the instructions necessary for the Bode diagram calculation. In this Code, $FP.fhone$, $FP.zhone$, $FP.fggone$, and $FP.zgone$ refer to the cut frequency of High-Pass filter measured in Hz , the damping ratio of the High-Pass filter, the cut frequency of the Low-Pass filter measured in Hz and the damping ratio of the Low-Pass filter.

```

1  FP.whone = FP.fhone*2*pi;
2  FP.wgone = FP.fggone*2*pi;
3  FP.numeqone = conv([1 0],[2*FP.zgone*FP.wgone FP.wgone^2]);
4  FP.deneqone = conv([1 2*FP.zhone*FP.whone FP.whone^2],[1 2*FP.zgone*FP.wgone FP.wgone^2]);
5  FP.Hone = tf(FP.numeqone,FP.deneqone);
6  FP.H = FP.Hone;
7  FP.GenAccone = lsim(FP.Hone,GP.RndAcc,GP.tt);
8  FP.NGenAcc = FP.GenAccone/max(abs(FP.GenAccone));
9  [FP.mag,FP.ph,FP.w] = bode(FP.H);

```

Code 2: Algorithm used by the *Synthetic Moonquake Generator* application for the Pass-Band filter calculation and application in the *Observed Values* tab of the *Band-Pass Filter* panel

The second method to design and apply the filter is to use the *User Values* tab of the *Band-Pass Filter* panel, where it is possible to define a Band-Pass filter composed of a Low-Pass filter and a High-Pass filter with a variable order from one to four. Both filters are defined, as Butterworth filters of given cutoff frequency and order (independently). The values of these four parameters can be modified by using the editable text boxes arranged as shown in Fig. 9. The Band-Pass filter generation algorithm of the *User Values* tab is based on the instructions shown in the Code 3 where the filter construction process is described together with the necessary instructions for the Bode diagram calculation. In this Code, $FP.fhtwo$, $FP.fhotwo$, $FP.fggtwo$, and $FP.fggotwo$ refer to the cutoff frequency of the High-Pass filter measured in Hz , the order of the High-Pass filter, the cutoff frequency of the Low-Pass filter measured in Hz and the order of the Low-Pass filter.

```

1  [FP.fhbtwo,FP.fhatwo] = butter(FP.fhotwo,FP.fhtwo/(GP.fs/2),'high');
2  [FP.fggbtwo,FP.fggatwo] = butter(FP.fggotwo,FP.fggtwo/(GP.fs/2),'low');
3  FP.btwo = conv(FP.fhbtwo,FP.fggbtwo);
4  FP.atwo = conv(FP.fhatwo,FP.fggatwo);
5  FP.Htwo = tf(FP.btwo,FP.atwo);
6  FP.H = FP.Htwo;
7  FP.GenAcctwo = filtfilt(FP.btwo,FP.atwo,GP.RndAcc);
8  FP.GenAcctwo = FP.GenAcctwo(:);
9  FP.NGenAcc = FP.GenAcctwo/max(abs(FP.GenAcctwo));
10 [FP.mag,FP.ph,FP.w] = bode(FP.H);

```

Code 3: Algorithm used by the *Synthetic Moonquake Generator* application for the calculus of the Pass-Band filter in the *User Values* tab of the panel *Band-Pass Filter*

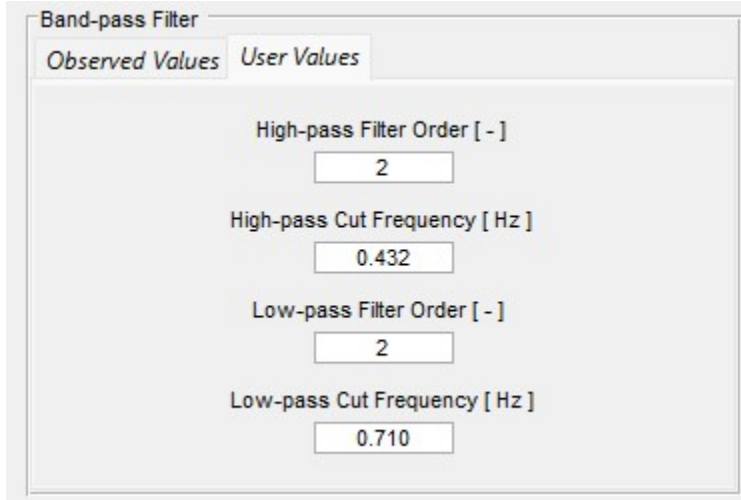


Figure 9: Tab *User Values* of the panel *Band-Pass Filter*

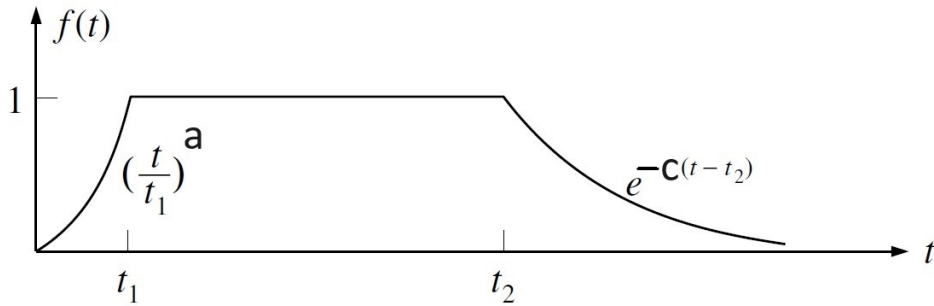


Figure 10: Figure describing the components and equations of the temporal envelope of the *Temporal Envelope* panel. Adapted from Clough and Penzien, 2003.

3.3 Temporal Envelope Panel

The natural next step in the generation of synthetic accelerograms is the application of a suitable time window. This can be done with the help of the *Temporal Envelope* panel and the controls arranged there. First, we have a picture that describes schematically the notation of the elements of the temporal envelope windows used by the *Synthetic Moonquake Generator* software. These consist of an initial region of increasing amplitude, a plateau, and a final region of exponential decrease.

- t_1 [%] relative time of the end of growth, expressed as a percentage fraction of the time where the record's growth region ends to the record's total duration.
- a [-] exponent of the growth region curve.
- t_2 [%] The relative decrement onset time represents the portion of an event where the decrement region begins, expressed as a percentage fraction of the total duration of the event.
- c [-] exponent of the decreasing region.

The process of designing and applying the temporal envelope can be done in two ways, the first of these is by using the *Observed Values* tab of the *Temporal Envelope* panel which appears by default and is shown in Fig. 11. The *Select Desired Moment Magnitude Range* panel has three radial controls that allow choosing a magnitude range corresponding to one of the three proposed temporal envelope models based on the characteristics observed for surface-type moonquakes. Once one of these buttons is chosen, the new window is automatically applied and the necessary results and plots are recalculated. Fig. shows the appearance of the *Observed Values* tab of the *Temporal Envelope* panel.

Selecting a magnitude range applies the corresponding predefined time envelope for that range but this can be further customized by using sliders or their corresponding editable text boxes that allow modifying the values of the four parameters describing the time envelope within a range of suggested values according to the results of the characterization of the studied moonquakes. It should be noted, that the suggested ranges of values and the default values are different for each moment magnitude range available in the *Select Desired Moment Magnitude Range* panel.

The Code 4 shows the algorithm used for the calculus of, the time envelope. In this *TP.t1one*, *TP.aone*, *TP.t2one*, and *TP.cone* correspond to the relative time of the end of the growth, the

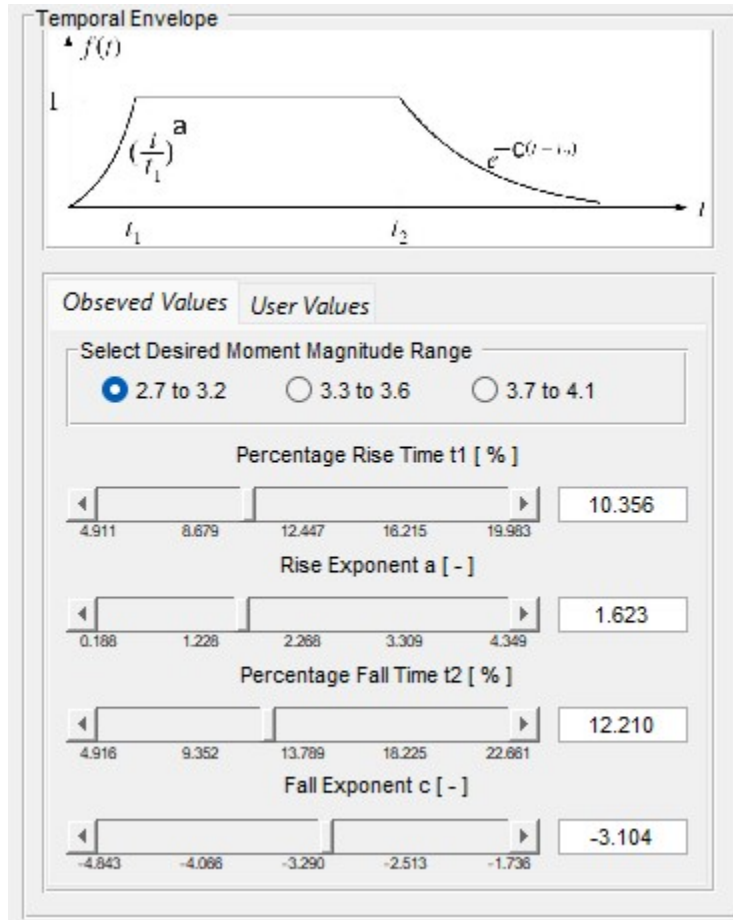


Figure 11: *Observed Values* tab of the *Temporal Envelope* panel

exponent of the growth region curve, the relative time of the beginning of the decrease and the exponent of the decrease region. While $TP.winN$ corresponds to the time envelope.

```

1 TP.N01 = floor(TP.t1one*length(GP.tt)/100);
2 TP.n01 = TP.N01;
3 TP.win01 = ((1:TP.n01)/TP.n01).^TP.aone;
4 TP.win01 = TP.win01(:);
5 TP.N02 = ceil(TP.t2one*length(GP.tt)/100);
6 TP.N03 = length(GP.tt);
7 TP.n03 = TP.N03 - TP.N02 + 1;
8 TP.win03 = exp(TP.cone*(1:TP.n03)/TP.n03);
9 TP.win03 = TP.win03(:);
10 TP.n02 = length(GP.tt) - length(TP.win01) - length(TP.win03);
11 TP.win02 = ones(TP.n02,1);
12 TP.winNone = [TP.win01; TP.win02; TP.win03];
13 TP.winN = TP.winNone;

```

Code 4: Algorithm used by the application *Synthetic Moonquake Generator* for the calculus of the temporal envelope on the *Observed Values* tab of the *Temporal Envelope* panel

The second method for designing and applying the temporal envelope is to use the *User Values* tab of the *Temporal Envelope* panel. This panel has four editable text boxes that allow modification of the values of the parameters describing the temporal envelope. Modifying the value of any of these fields automatically recalculates the results and updates the corresponding plots. Fig. 12 shows the appearance of the "User Values" tab of the "Temporal Envelope" panel.

The Code 5 shows the algorithm used to, calculate the temporal envelope. In this $TP.t1two$, $TP.atwo$, $TP.t2two$, and $TP.ctwo$ correspond to the relative time of the end of the growth, the exponent of the growth region curve, the relative time of the beginning of the decrease and the exponent of the decrease region. While $TP.winN$ corresponds to the time envelope.

```

1 TP.N01 = floor(TP.t1two*length(GP.tt)/100);
2 TP.n01 = TP.N01;
3 TP.win01 = ((1:TP.n01)/TP.n01).^TP.atwo;
4 TP.win01 = TP.win01(:);
5 TP.N02 = ceil(TP.t2two*length(GP.tt)/100);

```

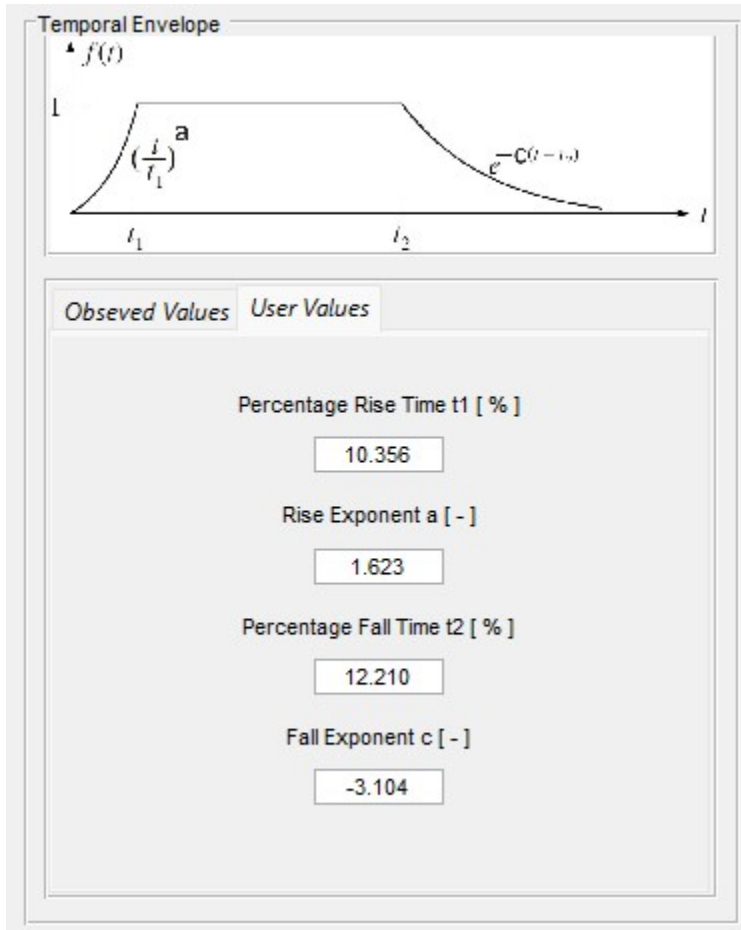


Figure 12: *User Values* tab of the *Temporal Envelope* panel

Figure 13: *Peak Ground Acceleration* panel of the *Synthetic Accelerogram* tab

```

6  TP.N03 = length(GP.tt);
7  TP.n03 = TP.N03 - TP.N02 + 1;
8  TP.win03 = exp(TP.ctwo*(1:TP.n03)/TP.n03);
9  TP.win03 = TP.win03(:);
10 TP.n02 = length(GP.tt) - length(TP.win01) - length(TP.win03);
11 TP.win02 = ones(TP.n02,1);
12 TP.winNtwo = [TP.win01; TP.win02; TP.win03];
13 TP.winN = TP.winNtwo;

```

Code 5: Algorithm used by the application *Synthetic Moonquake Generator* for the calculus of the temporal envelope on the *User Values* tab of the *Temporal Envelope* panel

3.4 PGA Panel

The modification of the record PGA can be done thanks to the *Peak Ground Acceleration* panel, which has an editable text box that, when modified, automatically recalculates the results and figures related to the synthetic accelerogram. The *Peak Ground Acceleration* pane is shown in Fig. 13.

3.5 Synthetic Accelerogram Plot Panel

The last panel in the *Synthetic Accelerogram* tab is the figure panel that displays the results of the synthetic moonquake generation process. It consists of six figures arranged from the top left to the right and then downwards. These figures include:

1. The generated white noise accelerogram.
2. The power spectral density of the white noise.

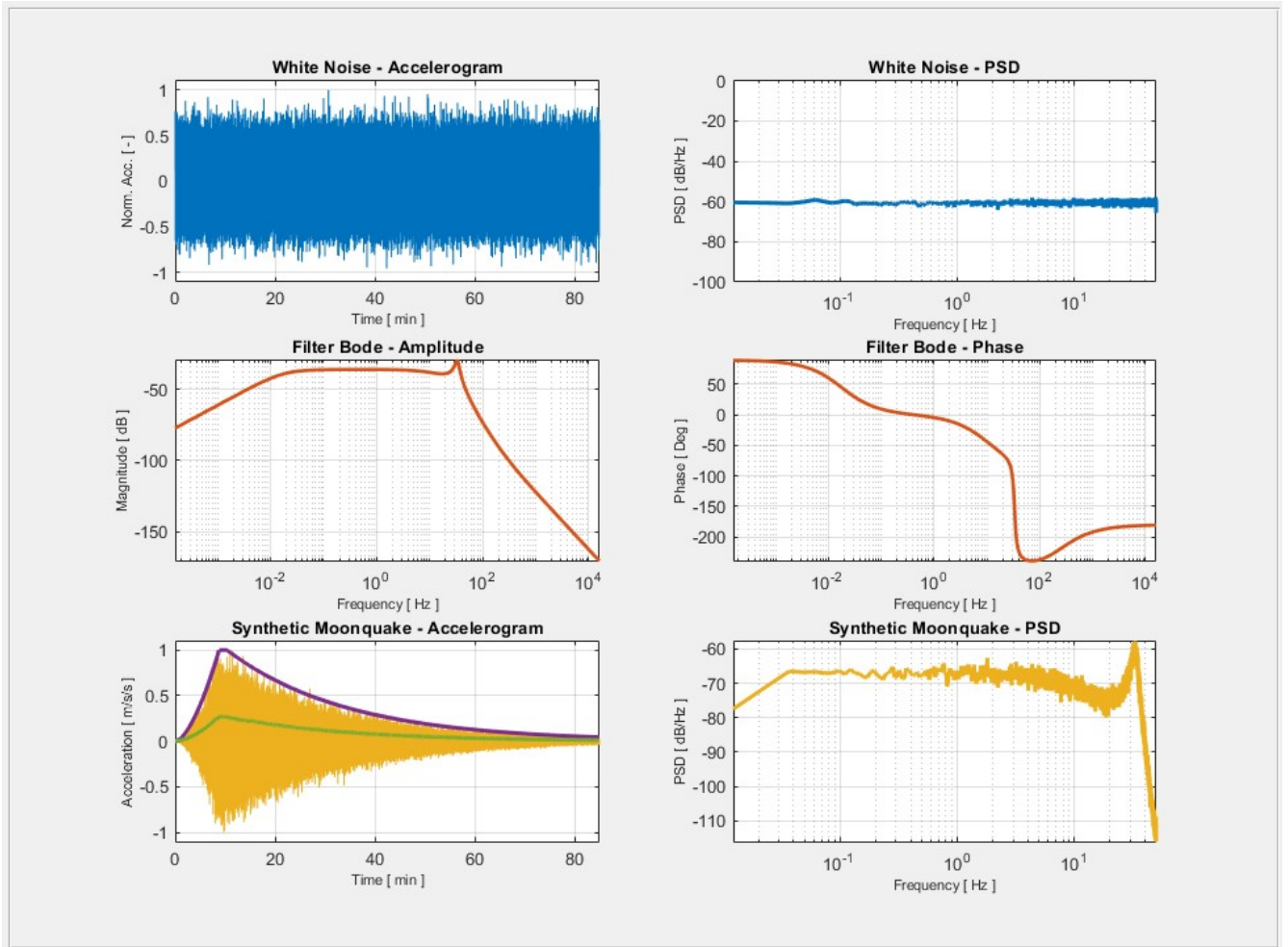


Figure 14: *Plot Panel* of the *Synthetic Accelerogram* Tab

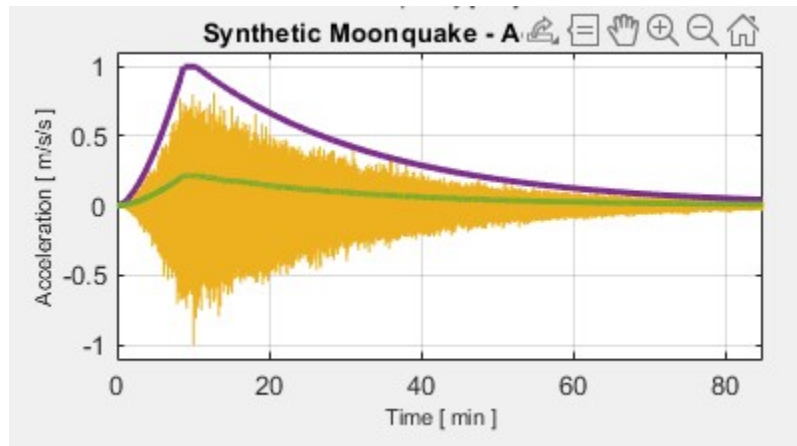


Figure 15: Controls embedded in each figure of the *Plot Panel* of the *Synthetic Moonquake Generator* application

3. The amplitude component of the Bode diagram of the filter.
4. The phase component of the Bode diagram of the filter.
5. The synthetic moonquake accelerogram (in yellow) along with its temporal envelope (in violet), and root mean square envelope (in green).
6. The power spectral density of the synthetic moonquake.

Additionally, each figure of the application is equipped with embedded controls that allow actions such as zooming, panning, cursor-based value selection, restoring the original view, and exporting to various image formats. The figure panel in the *Synthetic Accelerogram* tab is shown in Fig. 14, while the details of the controls within one of these figures are depicted in Fig. 15.

3.6 Synthetic Accelerogram Export Results

Images from the figures panel can be exported as MATLAB *.fig* files for further analysis and processing by clicking on the *Export Figures* button on the *Synthetic Accelerogram* tab (Fig. 16). Pressing this button opens the dialog box *Export figures as 'MATLAB .fig' files* which allows you to choose the location to save the figures.



Figure 16: *Export Figures* button of the *Synthetic Accelerogram* tab

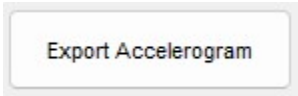


Figure 17: *Export Accelerogram* button of the *Synthetic Accelerogram* tab

On the other hand, it is possible to export the generated synthetic moonquake as a two-column text file (Time and Acceleration) using the functionality provided by the *Export Accelerogram* button of the *Synthetic Accelerogram* tab (Fig. 17). Pressing this button opens a window with the dialog box *Export Accelerogram as '.txt' file* that allows assigning a specific name and location to the text file.

The exported synthetic accelerogram file has a descriptive header indicating the file creation date, record type (Synthetic Moonquake Accelerogram Record), file name, record duration, record sampling rate, first column name and unit of measurement (*Time (s)*), second column name and unit of measurement (*Synthetic Moonquake Acc. (m/s/s)*), set of instructions for loading the file into MATLAB, and two-column record data. A portion of the beginning of a *Synthetic Moonquake Accelerogram Record* file is shown in the Code 6.

```
1 % Date: 28/February/2024-21:40:02
2 % Synthetic Moonquake Accelerogram Record
3 % File: MQ001.txt
4 % Duration: 85.000 min
5 % Sampling Frequency: 100.000 Hz
6 % PGA: 1.000 m/s/s
7 % First Column: Time (s)
8 % Second Column: Synthetic Moonquake Acc. (m/s/s)
9 % Matlab Loading Method:
10 % fileID = fopen('MQ001.txt');
11 % Data = cell2mat(textscan(fileID,'%f %f','headerlines',14));
12 % fclose(fileID);
13
14
15 0.000000      4.457583e-27
16 0.010000      7.682189e-09
17 0.020000      4.556491e-08
```

Code 6: Initial part of a Synthetic Moonquake Accelerogram Record exported by the *Synthetic Moonquake Generator* application.

4 Response Spectrum Tab

4.1 Damping Ratio Panel

The value of the damping ratio can be modified with the help of the editable text box in the *Damping Ratio* panel. Once this value has been modified, the application calculates the response spectra and updates the graphs in the figures panel of the *Response Spectrum* tab.



Figure 18: *Damping Ratio* panel of the *Response Spectrum* tab

Figure 19: *Response Spectrum Periods* panel of the *Response Spectrum* tab

Figure 20: *Gravitational Acceleration* panel of the *Response Spectrum* tab

4.2 Response Spectrum Periods Panel

The process of defining periods for the calculus of response spectra can be carried out with the help of the Response Spectrum Periods panel. This panel has three editable text boxes that allow modifying the value of the minimum, maximum, and number of periods. The algorithm for the specific calculus of the vector of periods is shown in Code 7 where the variables $TN.Tmin$, $TN.Tmax$ and $TN.Tnum$ corresponds to the minimum, maximum, and number of periods.

```
1 TN.Tn=logspace(log10(TN.Tmin),log10(TN.Tmax),TN.Tnum);
```

Code 7: Algorithm used by the *Synthetic Moonquake Generator* application to calculate the periods of the *Response Spectrum Periods* panel

4.3 Gravitational Acceleration Panel

To modify the value of the magnitude of the gravitational acceleration it is necessary to use the editable text box of the *Gravitational Acceleration* panel. Once this value has been modified, the application will automatically calculate the response spectra and update the graphs in the figures panel of the *Response Spectrum* tab.

4.4 SpecMQ Function for the calculus of seismic response spectra

The algorithm that computes the response spectra for the *Synthetic Moonquake Generator* application is based on the excitation interpolation method for speed of computation (Chopra, 2012). The code implementation is shown in Cod. 8 and it should be noted that the variables $SP.ResD$, $SP.ResV$, $SP.ResA$, $SP.ResSV$ and $SP.ResSA$ corresponds to the spectra in Displacement, Velocity, Acceleration, Pseudo-Velocity, and Pseudo-Acceleration respectively.

```
1 function SpecMQ()
2
3 global M GP FP TP ZZ TN GG SP
4
5     set(M.fg, 'pointer', 'watch')
6     drawnow;
7
8     SP.ResD=zeros;
9     SP.ResV=zeros;
10    SP.ResA=zeros;
11    SP.ResSV=zeros;
12    SP.ResSA=zeros;
13    for ii=1:length(TN.Tn)
14        SP.tn=TN.Tn(ii);
15        SP.wn=2*pi/SP.tn;
16        SP.c=2*ZZ.Z/100*SP.wn;
17        SP.k=SP.wn^2;
18        [SP.u,SP.v,SP.a]=interpolation(1,SP.c,SP.k,-TP.mqSN*TP.PGA/GG.gg,0,0,1/GP.fs,length(
        ↪ TP.mqSN));
```

```

19     SP.D=max(abs(SP.u));
20     SP.V=max(abs(SP.v));
21     SP.A=max(abs(SP.a));
22     SP.SV=SP.wn*SP.D;
23     SP.SA=SP.wn*SP.wn*SP.D;
24     SP.ResD(ii)=SP.D;
25     SP.ResV(ii)=SP.V;
26     SP.ResA(ii)=SP.A;
27     SP.ResSV(ii)=SP.SV;
28     SP.ResSA(ii)=SP.SA;
29 end
30
31 set(M.fg, 'pointer', 'arrow');
32
33 function [u,v,a]=interpolation(m,c,k,pt,uo,vo,dt,N)
34     u=zeros;
35     v=zeros;
36     a=zeros;
37     wn=sqrt(k/m);
38     zita= c/(2*m*wn);
39     wd=wn*sqrt(1- zita^2);
40     p=pt;
41     u(1)=uo;
42     v(1)=vo;
43     a(1)=(p(1)/m)-(2*zita*wn*v(1))-(wn^2)*u(1);
44     A=exp(-zita*wn*dt)*(((zita*sin(wd*dt))/sqrt(1-(zita)^2))+cos(wd*dt));
45     B=exp(-zita*wn*dt)*(sin(wd*dt)/wd);
46     C=(1/k)* (((2*zita)/ (wn*dt)) + (exp(-zita*wn*dt)*((((1- 2*(zita^2))/(wn*dt)) -
47     ↪ (zita/sqrt(1 - (zita^2))))*sin(wd*dt)) -((1 +((2*zita)/(wn*dt)))*cos(wd*dt)))));
48     D=(1/k)* ( (1-(2*zita/(wn*dt))) +
49     ↪ (exp(-zita*wn*dt)*((-1+2*(zita^2))/(wn*dt))*sin(wd*dt)
50     ↪ +(2*zita/(wn*dt))*cos(wd*dt))));
51     Apr=-exp(-zita*wn*dt)*(wn*sin(wd*dt)/sqrt(1 - zita^2));
52     Bpr=exp(-zita*wn*dt)*(-zita*sin(wd*dt)/sqrt(1 - zita^2) + cos(wd*dt));
53     Cpr=(1/k)* ((-1/ dt) + exp(-zita*wn*dt)*(((wn/sqrt(1 - zita^2)) + zita/(dt*sqrt(1 -
54     ↪ zita^2))))*sin(wd*dt) +(1/dt)*cos(wd*dt)));
55     Dpr=(1/(k*dt))*(1-exp(-zita*wn*dt)*((zita/sqrt(1 - zita^2))*sin(wd*dt) +cos(wd*dt)));
56     for i=1:N-1
57         u(i+1)= A*u(i)+B*v(i)+C*p(i)+D*p(i+1);
58         v(i+1)= Apr*u(i)+Bpr*v(i)+Cpr*p(i)+Dpr*p(i+1);
59         a(i+1)=(p(i+1)/m)-2*zita*wn*v(i+1)-(wn^2)*u(i+1);
60     end
61 end
62 end

```

Code 8: *SpecMQ.m*: Function for the calculus of the seismic response spectra of the tab *Response Spectrum*

4.5 Response Spectrum Plot Panel

The final section of the *Response Spectrum* tab features the figures panel, displaying the results of the synthetic moonquake response spectra calculations. This panel consists of five individual plots, arranged from top left to bottom right:

- Displacement Response Spectrum
- Velocity Response Spectrum
- Acceleration Response Spectrum
- Pseudo-Velocity Response Spectrum
- Pseudo-Acceleration Response Spectrum

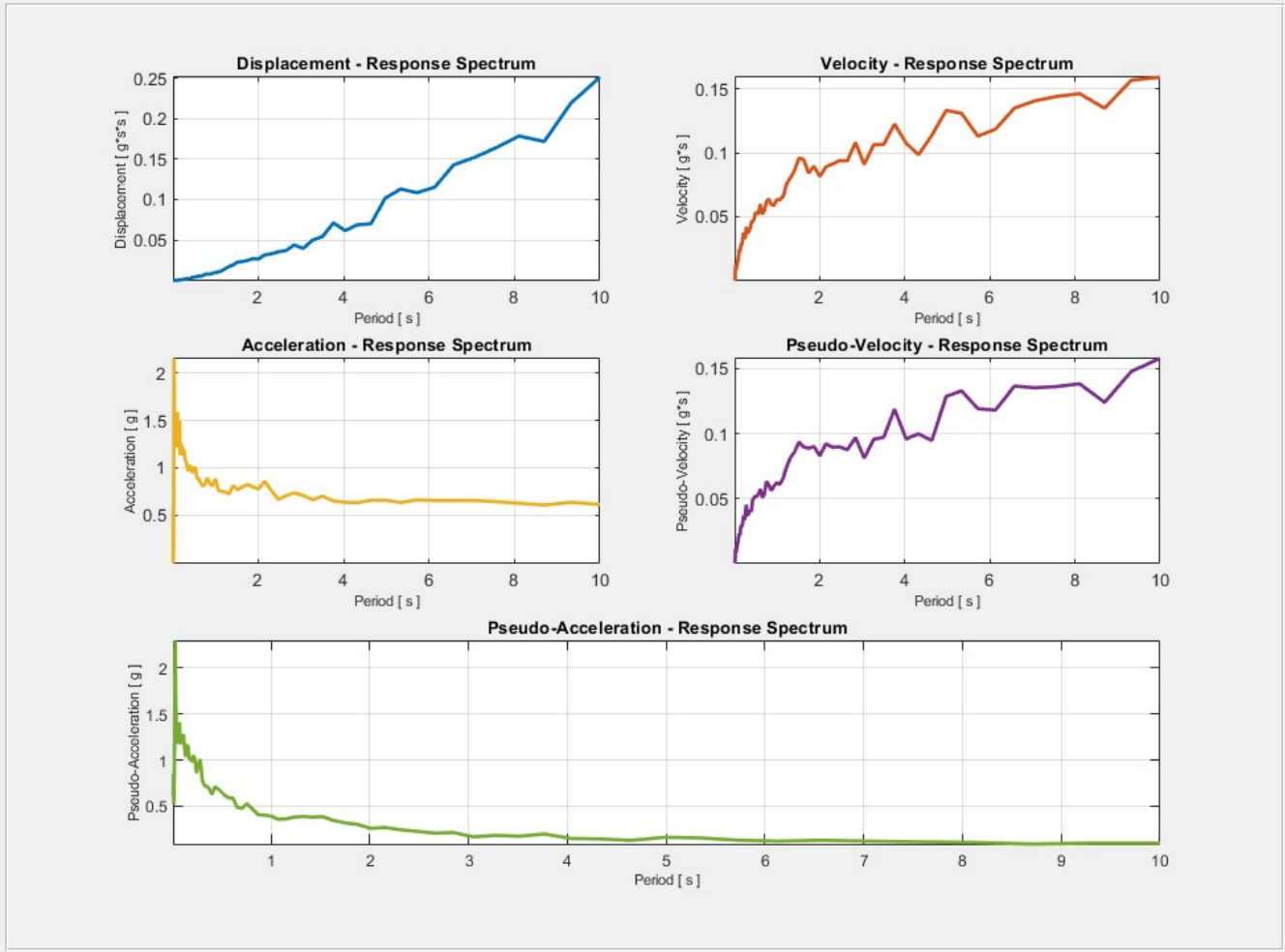


Figure 21: Plot panel of the *Response Spectrum* tab

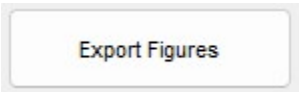


Figure 22: *Export Figures* button of the *Response Spectrum* tab

Each of these response spectra corresponds to the synthetic moonquake generated. Additionally, within each figure, there are embedded controls that allow users to perform the following actions:

- Zoom In/Out: Adjust the view scale.
- Pan: Navigate within the plot.
- Cursor Selection: Interactively select values.
- Cursor Selection: Interactively select values.
- Export: Save the figure in various image formats.

The figures panel of the *Response Spectrum* tab is depicted in Fig. 21, while a detailed view of the controls within one of the figures is shown in Fig. 15.

4.6 Response Spectrum Export Results

The figure panel plots can be exported as MATLAB *.fig* files for further analysis and processing by clicking on the *Export Figures* button on the *Response Spectrum* tab (Fig. 22). Pressing this button opens the dialog box *Export figures as 'MATLAB .fig' files* which allows you to choose the location to save the five figures.

It is possible to export the calculated response spectra as a six-column text file (Period, Displacement, Velocity, Acceleration, Pseudo-Velocity, and Pseudo-Acceleration) using the functionality provided by the *Export Spectra* button of the *Response Spectrum* tab (Fig. 23). Pressing this button opens a window with the dialog box *Export Response Spectra as '.txt' file* that allows assigning a specific name and location to the text file.

The exported response spectra file includes a descriptive header that provides essential information. This header consists of the following details:



Figure 23: *Export Spectra* button of the *Response Spectrum* tab

- File Creation Date
- Record Type: Synthetic Moonquake Response Spectrum.
- File Name
- Damping Ratio
- Minimum Period
- Maximum Period
- Number of Periods
- Magnitude of the Gravitational Acceleration
- Column Names and Units of Measurement:
 - *Period (s)*
 - *Displacement (g*s*s)*
 - *Velocity (g*s)*
 - *Acceleration (g)*
 - *Pseudo-Velocity (g*s)*
 - *Pseudo-Acceleration (g)*
- Instructions for Loading the File data into MATLAB
- Six-Column Log Data

```
1 % Date: 28/February/2024-21:40:37
2 % Synthetic Moonquake Response Spectrum's
3 % File: Spect001.txt
4 % Structural Damping Ratio: 5.000 %
5 % Minimum Period: 0.010 s
6 % Maximum Period: 10.000 s
7 % Number of Periods: 100 -
8 % Gravitational Acceleration: 1.625 m/s/s
9 % First Column: Period (s)
10 % Second Column: Displacement (g*s*s)
11 % Third Column: Velocity (g*s)
12 % Fourth Column: Acceleration (g)
13 % Fifth Column: Pseudo-Velocity (g*s)
14 % Sixth Column: Pseudo-Acceleration (g)
15 % Matlab Loading Method:
16 % fileID = fopen('Spect001.txt');
17 % Data = cell2mat(textscan(fileID,'%f %f','headerlines',20));
18 % fclose(fileID);
19
20
21 0.010000      1.553475e-06      4.525456e-05      6.555663e-04      9.760771e-04      6.132873e-01
22 0.010723      1.868718e-06      7.281014e-05      4.145625e-02      1.095016e-03      6.416487e-01
23 0.011498      2.269645e-06      1.407785e-04      1.066917e-01      1.240314e-03      6.778062e-01
```

Code 9: Initial part of a *Synthetic Moonquake Response Spectrum* file exported by the *Synthetic Moonquake Generator* application

5 Code Review

5.1 Code Repository

The code repository has a directory structure as shown in Fig. 24. Here, we observe the root directory *SMG*, the Source Code folder *Code*, its subdirectories, the Auxiliary Documents Folder *Docs*, the Auxiliary Images Folder *Images*, and the Application Entry Point *SMG.m*.

The structure of the source code directory is divided into 15 subdirectories, which correspond to the logical units of the application design. For example, the folder *01Main* contains the code responsible for creating the main window and defining the necessary callbacks and functions for its proper functioning. In total, there are 43 source code files required for implementation. Additionally, the documents and images in the auxiliary folders are necessary for the correct execution of the application. Fig. 25 and 26 illustrate the Source Code directory structure and the contained files.

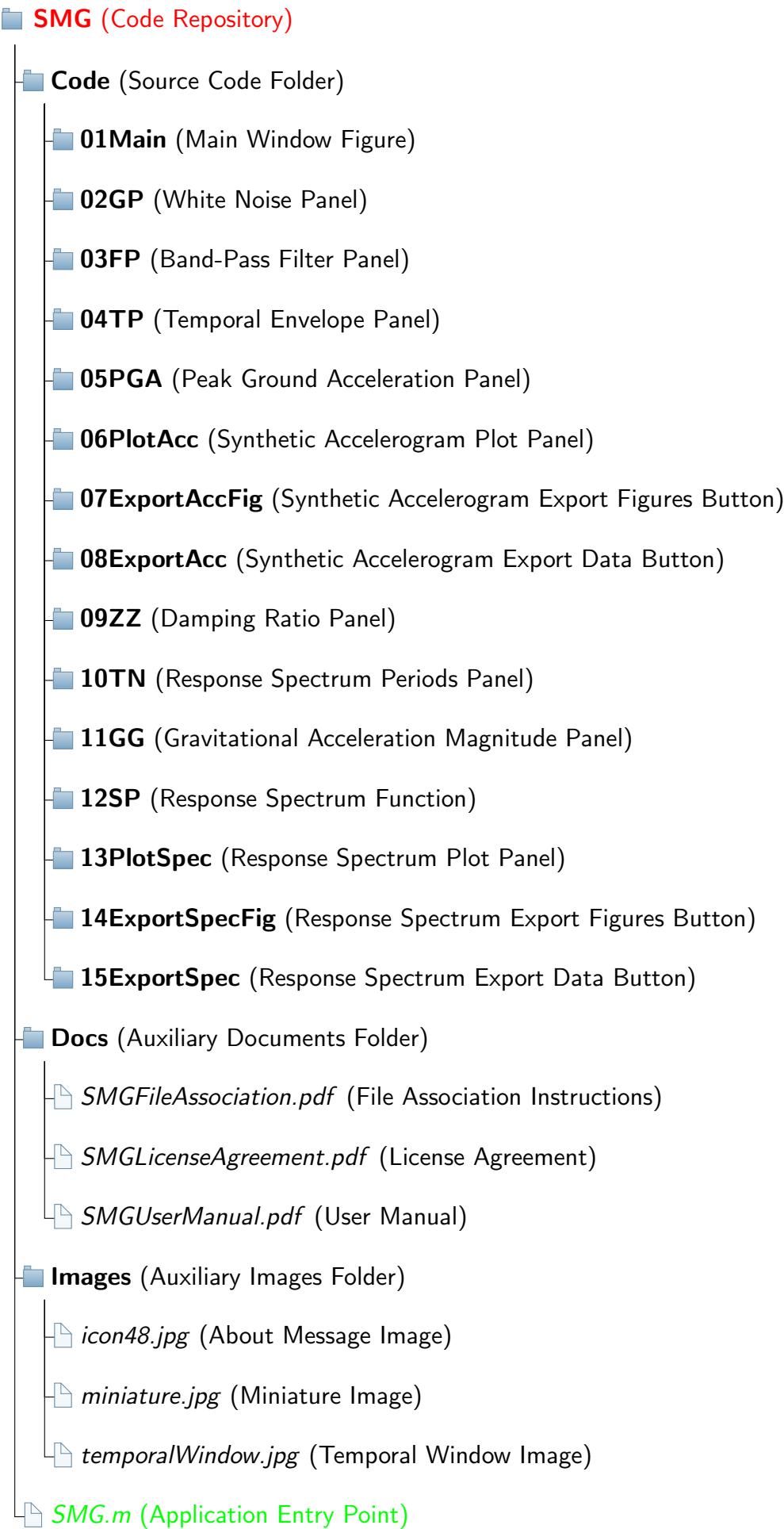


Figure 24: Code repository of *Synthetic Moonquake Generator* application

Code (Source Code Folder) - (Part 1 of 2)

01Main (Main Window Figure)

- AboutCallBack.m (About Menu CallBack)
- FileAssociaionCallBack.m (File Association Menu CallBack)
- LicenseAgreementCallBack.m (License Agreement Menu CallBack)
- Main.m (Figure Constructor)
- MtgCallBack.m (Tab Change CallBack)
- OpenFileCallBack.m (Open Menu CallBack)
- SaveFileCallBack.m (Save As Menu CallBack)
- UserManualCallBack.m (User Manual Menu CallBack)

02GP (White Noise Panel)

- GPCallBack.m (Edit Boxes CallBack)
- GPpanel.m (Panel Constructor)

03FP (Band-Pass Filter Panel)

- FPCallBack1.m (Sliders CallBack of the Observed Values Tab)
- FPCallBack2.m (Edit Boxes CallBack of the Observed Values Tab)
- FPCallBack3.m (Edit Boxes CallBack of the User Values Tab)
- FPpanel.m (Panel Constructor)
- FPtgCallBack.m (Tab Change CallBack)

04TP (Temporal Envelope Panel)

- imgzoompan.m (Zoom and Pan Function) {Third Party Code}
- RadioButtonCallBack.m (Radio Button CallBack)
- TPCallBack1.m (Sliders CallBack of the Observed Values Tab)
- TPCallBack2.m (Edit Boxes CallBack of the Observed Values Tab)
- TPCallBack3.m (Edit Boxes CallBack of the User Values Tab)
- TPpanel.m (Panel Constructor)
- TPtgCallBack.m (Tab Change CallBack)

05PGA (Peak Ground Acceleration Panel)

- PGACallBack.m (Edit Box CallBack)
- PGApanel.m (Panel Constructor)

Figure 25: Contents of the Source Code folder of the repository - (Part 1 of 2)

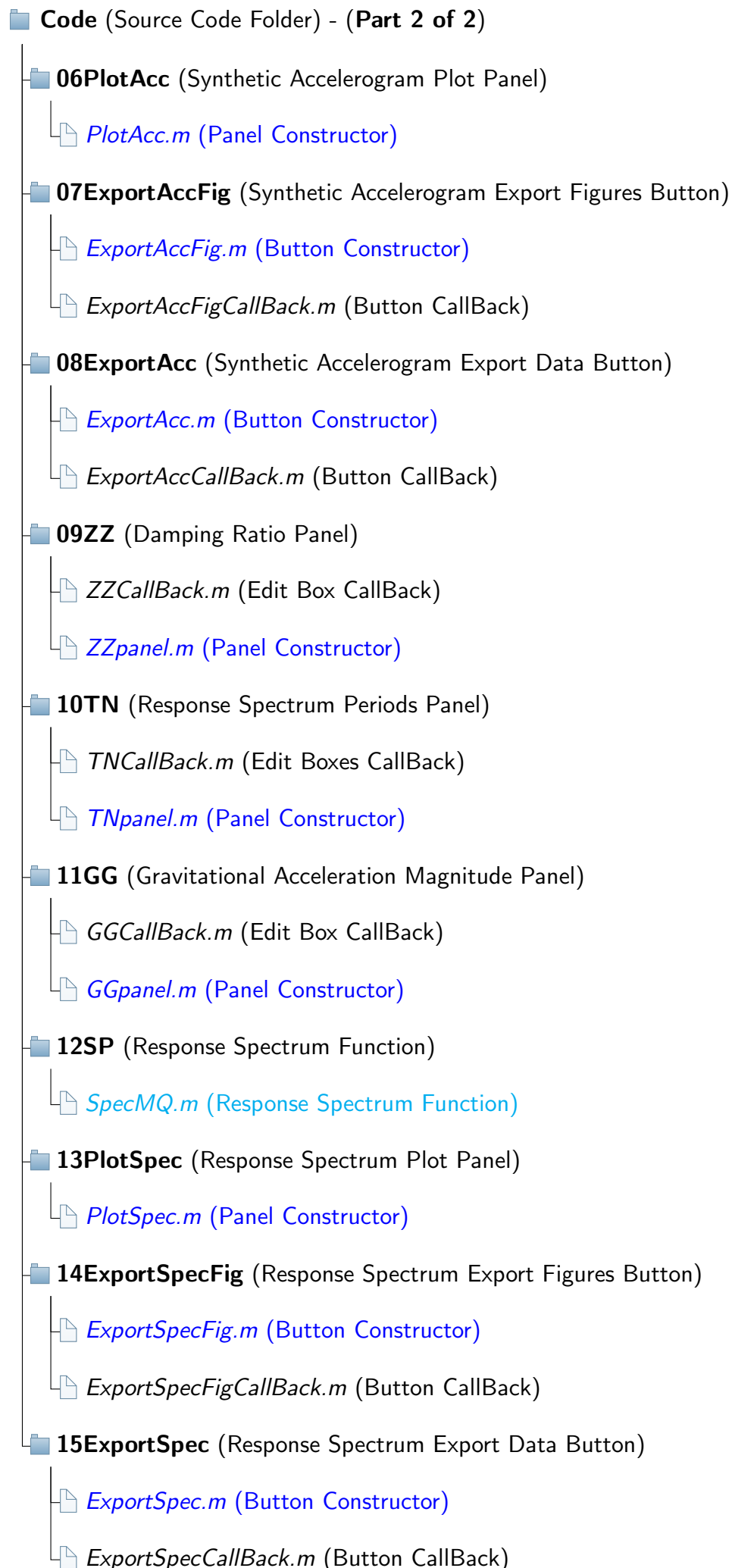


Figure 26: Contents of the Source Code folder of the repository - (Part 2 of 2)

5.2 Source Code Analysis

The code is organized following a pattern where a constructor file defines the elements of the graphical interface, and a set of callbacks and other functions implement the functionality corresponding to the elements defined by the constructor. In this sense, the source code analysis

should encompass the source code itself, the hierarchical structure of the graphical elements, and the corresponding flowcharts.

5.2.1 Entry Point of the *Synthetic Moonquake Generator* application

```

1 function SMG(varargin)
2
3     if isdeployed
4         warning off;
5     else
6         close all;
7         clearvars -except varargin
8         clc;
9         warning('on','verbose')
10        warning off MATLAB:ui:javaframe:PropertyToBeRemoved
11        warning off MATLAB:Figure:FigureSavedToMATFile
12        warning off Control:analysis:LsimUndersampled
13    end
14
15    global M GP FP TP ZZ TN GG SP ...
16        miniaturePath temporalWindowPath aboutIconPath ...
17        userManualPath licenseAgreementPath fileAssociationPath
18
19    % MATLAB Compiler don't support addpath
20    addpath(genpath(pwd));
21
22    if isdeployed
23        [~, result] = system('path');
24        installPath = char(regexp(result,'Path=(.*)','tokens','once'));
25        miniaturePath = fullfile(installPath,'miniature.jpg');
26        temporalWindowPath = fullfile(installPath,'temporalWindow.jpg');
27        aboutIconPath = fullfile(installPath,'icon48.jpg');
28        userManualPath = fullfile(installPath,'SMGUserManual.pdf');
29        fileAssociationPath = fullfile(installPath,'SMGFileAssociation.pdf');
30        licenseAgreementPath = fullfile(installPath,'SMGLicenseAgreement.pdf');
31    else
32        miniaturePath = fullfile(pwd,'Images','miniature.jpg');
33        temporalWindowPath = fullfile(pwd,'Images','temporalWindow.jpg');
34        aboutIconPath = fullfile(pwd,'Images','icon48.jpg');
35        userManualPath = fullfile(pwd,'Docs','SMGUserManual.pdf');
36        fileAssociationPath = fullfile(pwd,'Docs','SMGFileAssociation.pdf');
37        licenseAgreementPath = fullfile(pwd,'Docs','SMGLicenseAgreement.pdf');
38    end
39
40    if isempty(varargin)
41
42        Main
43        GPpanel
44        FPpanel
45        TPpanel
46        PGAPanel
47        PlotAcc
48        ExportAccFig
49        ExportAcc
50        ZZpanel
51        TNpanel
52        GGpanel
53        SpecMQ
54        PlotSpec
55        ExportSpecFig
56        ExportSpec

```

```

57
58     guidata(M.fg,M);
59 elseif ~isempty(varargin)
60     if (length(varargin) == 1)
61         fileName = varargin{1};
62         if exist(fileName, 'file') == 2
63             msg1 = msgbox(['Processing the file: ', fileName]);
64             jframe = get(msg1,'javaframe');
65             jframe.setFigureIcon(javax.swing.ImageIcon(miniaturePath));
66
67             load(fileName, '-mat');
68
69             set(GP.Tedit, 'string', num2str(GP.T, '%.2f'));
70             set(GP.fsedit, 'string', num2str(GP.fs, '%.2f'));
71
72             set(FP.zhSlider1, 'value', FP.zhone);
73             set(FP.zhedit1, 'string', num2str(FP.zhone, '%.3f'));
74             set(FP.fggSlider1, 'value', FP.fggone);
75             set(FP.fggedit1, 'string', num2str(FP.fggone, '%.3f'));
76             set(FP.fhSlider1, 'value', FP.fhone);
77             set(FP.fhedit1, 'string', num2str(FP.fhone, '%.3f'));
78             set(FP.zgSlider1, 'value', FP.zgone);
79             set(FP.zgedit1, 'string', num2str(FP.zgone, '%.3f'));
80             set(FP.fhoedit2, 'string', num2str(FP.fhotwo, '%.0f'));
81             set(FP.fggoedit2, 'string', num2str(FP.fggotwo, '%.0f'));
82             set(FP.fggedit2, 'string', num2str(FP.fggtwo, '%.3f'));
83             set(FP.fhedit2, 'string', num2str(FP.fhtwo, '%.3f'));
84
85             set(TP.t1Slider, 'min', TP.t1min, 'max', TP.t1max, 'value', TP.t1one);
86             set(TP.aSlider, 'min', TP.amin, 'max', TP.amax, 'value', TP.aone);
87             set(TP.t2Slider, 'min', TP.t2min, 'max', TP.t2max, 'value', TP.t2one);
88             set(TP.cSlider, 'min', TP.cmin, 'max', TP.cmax, 'value', TP.cone);
89             set(TP.t1edit, 'string', num2str(TP.t1one, '%.3f'));
90             set(TP.aedit, 'string', num2str(TP.aone, '%.3f'));
91             set(TP.t2edit, 'string', num2str(TP.t2one, '%.3f'));
92             set(TP.cedit, 'string', num2str(TP.cone, '%.3f'));
93
94             TPslid = [0.025 0.70-0.05 0.725 0.045];
95             TP1bl = [0 TPslid(2)-0.03 0.15 0.03];
96             Tnticks = 5;
97             Tlblmargin = 0.05;
98             Tlblvals = linspace(TP.t1min, TP.t1max, Tnticks);
99             for k = 1:Tnticks
100                 Txpos = (Tlblmargin + (k-1)*(TPslid(3) - 2*Tlblmargin)/(Tnticks-1)) -
101                     ↪ TP1bl(3)/2;
102                 TP1bl(1) = 0.025 + Txpos;
103                 uicontrol('Parent', TP.tb1, ...
104                     'Units', 'normalized', ...
105                     'FontSize', 6, ...
106                     'HorizontalAlignment', 'center', ...
107                     'Style', 'text', ...
108                     'Position', TP1bl, ...
109                     'String', sprintf('%s', Tlblvals(k)), ...
110                     'Tag', 't1Slider1ticklabel');
111             end
112
113             TPslid = [0.025 0.50-0.05 0.725 0.045];
114             TP1bl = [0 TPslid(2)-0.03 0.15 0.03];
115             Tnticks = 5;
116             Tlblmargin = 0.05;
117             Tlblvals = linspace(TP.amin, TP.amax, Tnticks);

```

```

117     for k = 1:Tnticks
118         Txpos = (Tlblmargin + (k-1)*(TPslid(3) - 2*Tlblmargin)/(Tnticks-1)) -
            ↪ TPlbl(3)/2;
119         TPlbl(1) = 0.025 + Txpos;
120         uicontrol('Parent',TP.tb1,...
121                 'Units','normalized',...
122                 'FontSize',6,...
123                 'HorizontalAlignment','center',...
124                 'Style','text',...
125                 'Position',TPlbl,...
126                 'String',sprintf('%.3f',Tlblvals(k)),...
127                 'Tag','aslider1ticklabel');
128     end
129
130     TPslid = [0.025 0.30-0.05 0.725 0.045];
131     TPlbl = [0 TPslid(2)-0.03 0.15 0.03];
132     Tnticks = 5;
133     Tlblmargin = 0.05;
134     Tlblvals = linspace(TP.t2min,TP.t2max,Tnticks);
135     for k = 1:Tnticks
136         Txpos = (Tlblmargin + (k-1)*(TPslid(3) - 2*Tlblmargin)/(Tnticks-1)) -
            ↪ TPlbl(3)/2;
137         TPlbl(1) = 0.025 + Txpos;
138         uicontrol('Parent',TP.tb1,...
139                 'Units','normalized',...
140                 'FontSize',6,...
141                 'HorizontalAlignment','center',...
142                 'Style','text',...
143                 'Position',TPlbl,...
144                 'String',sprintf('%.3f',Tlblvals(k)),...
145                 'Tag','t2slider1ticklabel');
146     end
147
148     TPslid = [0.025 0.10-0.05 0.725 0.045];
149     TPlbl = [0 TPslid(2)-0.03 0.15 0.03];
150     Tnticks = 5;
151     Tlblmargin = 0.05;
152     Tlblvals = linspace(TP.cmin,TP.cmax,Tnticks);
153     for k = 1:Tnticks
154         Txpos = (Tlblmargin + (k-1)*(TPslid(3) - 2*Tlblmargin)/(Tnticks-1)) -
            ↪ TPlbl(3)/2;
155         TPlbl(1) = 0.025 + Txpos;
156         uicontrol('Parent',TP.tb1,...
157                 'Units','normalized',...
158                 'FontSize',6,...
159                 'HorizontalAlignment','center',...
160                 'Style','text',...
161                 'Position',TPlbl,...
162                 'String',sprintf('%.3f',Tlblvals(k)),...
163                 'Tag','cslider1ticklabel');
164     end
165
166     set(TP.t1edit2,'string',num2str(TP.t1two,'%.3f'));
167     set(TP.aedit2,'string',num2str(TP.atwo,'%.3f'));
168     set(TP.t2edit2,'string',num2str(TP.t2two,'%.3f'));
169     set(TP.cedit2,'string',num2str(TP.ctwo,'%.3f'));
170
171     set(TP.PGAedit, 'String',num2str(TP.PGA,'%.3f'));
172
173     set(M.LN1,'XData',GP.tt/60,'YData',GP.NGenAcc);
174     set(M.LN2,'XData',GP.ff,'YData',db(GP.psd));

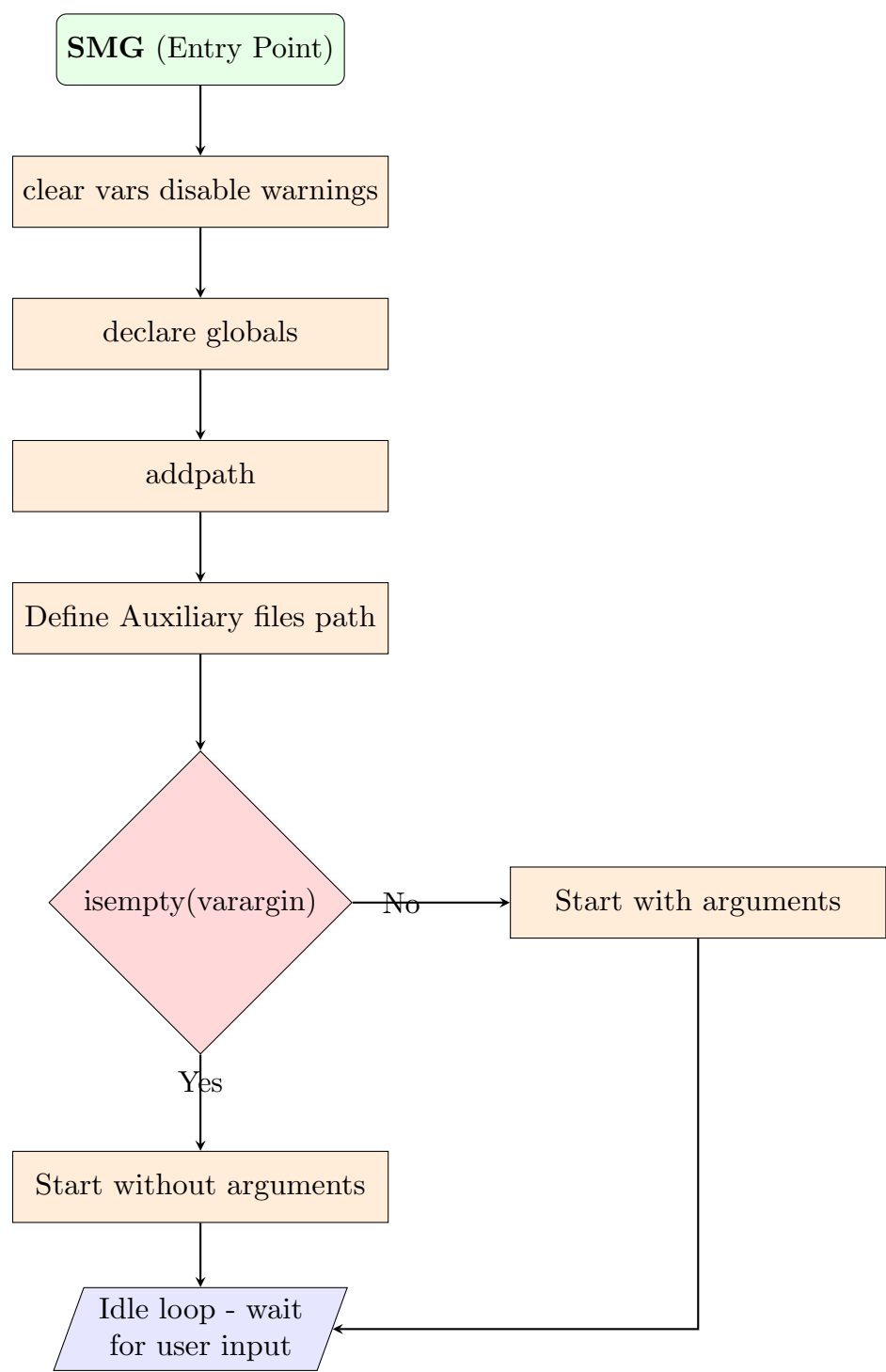
```

```

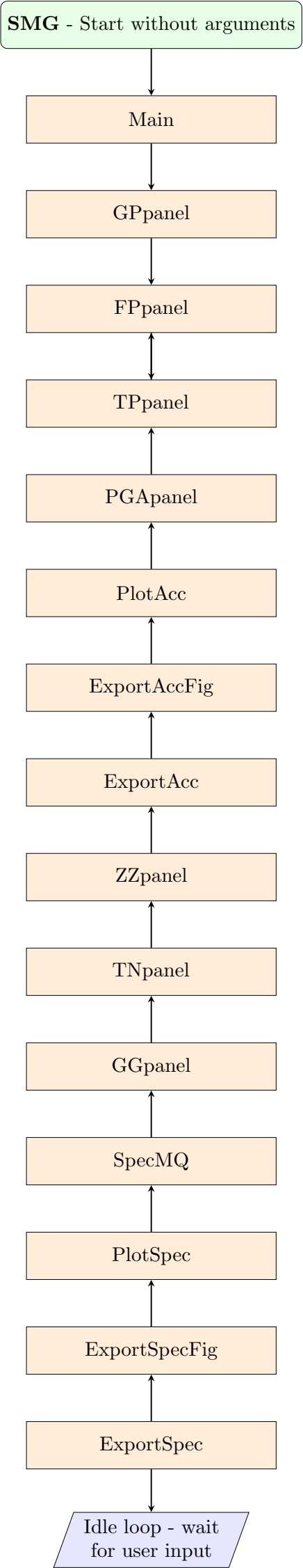
175     set(M.LN3,'XData',FP.w/2/pi,'YData',db(squeeze(FP.mag)));
176     set(M.LN4,'XData',FP.w/2/pi,'YData',squeeze(FP.ph));
177     set(M.LN5,'XData',GP.tt/60,'YData',TP.mqSN*TP.PGA);
178     set(M.LN5.Parent,'YLim',[-1.1 1.1]*TP.PGA);
179     set(M.LN6,'XData',GP.tt/60,'YData',TP.winN*TP.PGA);
180     set(M.LN7,'XData',GP.tt/60,'YData',envelope(TP.mqSN*TP.PGA,floor(length(
    ↵ TP.mqSN)/100),'rms'));
181     set(M.LN8,'XData',TP.ff,'YData',db(TP.psd));
182
183     set(ZZ.Zedit,'string',num2str(ZZ.Z,'%.3f'));
184     set(TN.Tminedit,'string',num2str(TN.Tmin,'%.3f'));
185     set(TN.Tmaxedit,'string',num2str(TN.Tmax,'%.3f'));
186     set(TN.Tnumedit,'string',num2str(TN.Tnum,'%.0f'));
187     set(GG.ggedit,'string',num2str(GG.gg,'%.3f'));
188
189     SpecMQ
190
191     set(M.LN9,'XData',TN.Tn,'YData',SP.ResD);
192     set(M.LN10,'XData',TN.Tn,'YData',SP.ResV);
193     set(M.LN11,'XData',TN.Tn,'YData',SP.ResA);
194     set(M.LN12,'XData',TN.Tn,'YData',SP.ResSV);
195     set(M.LN13,'XData',TN.Tn,'YData',SP.ResSA);
196
197     if FP.Seltab == 1
198         set(FP.tg,'SelectedTab',FP.tb1)
199     elseif FP.Seltab == 2
200         set(FP.tg,'SelectedTab',FP.tb2)
201     end
202
203     if TP.Seltab == 1
204         set(TP.tg,'SelectedTab',TP.tb1)
205     elseif FP.Seltab == 2
206         set(TP.tg,'SelectedTab',TP.tb2)
207     end
208
209     if TP.Checked == 1
210         set(TP.radioButton1,'Value',1);
211     elseif TP.Checked == 2
212         set(TP.radioButton2,'Value',1);
213     elseif TP.Checked == 3
214         set(TP.radioButton3,'Value',1);
215     end
216
217     if M.Seltab == 1
218         set(M.tg,'SelectedTab',M.tb1)
219     elseif M.Seltab == 2
220         set(M.tg,'SelectedTab',M.tb2)
221     end
222
223     guidata(M.fg,M);
224
225     msg2 = msgbox(['Project loaded: ', fileName]);
226     jframe = get(msg2,'javaframe');
227     jframe.setFigureIcon(javax.swing.ImageIcon(miniaturePath));
228 else
229     msg3 = msgbox(['The file does not exist: ', fileName]);
230     jframe = get(msg3,'javaframe');
231     jframe.setFigureIcon(javax.swing.ImageIcon(miniaturePath));
232 end
233 end
234 end

```

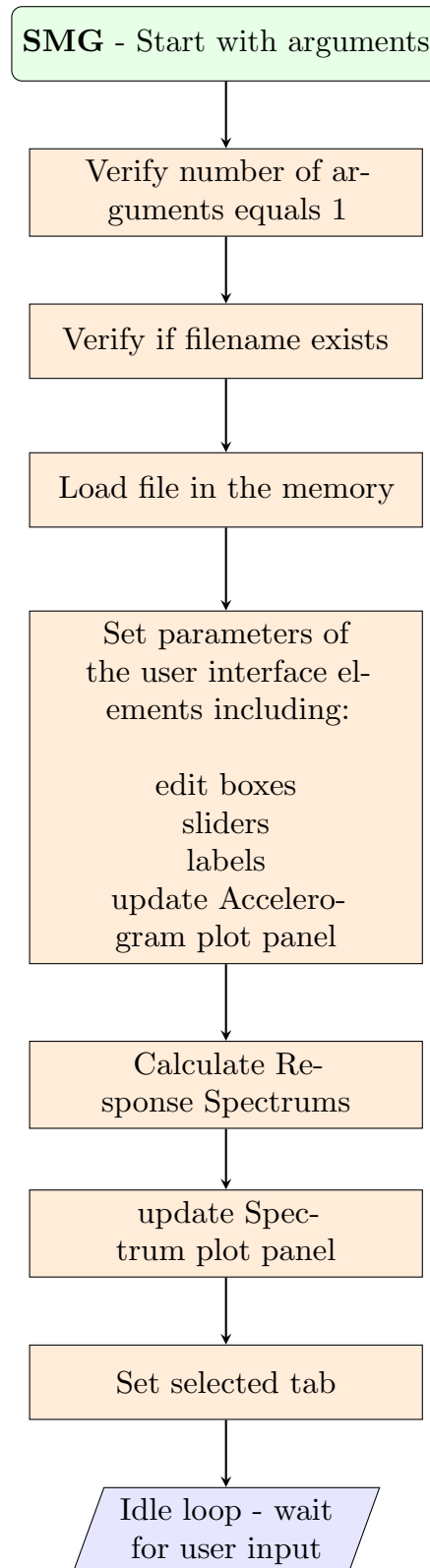
Code 10: *SMG.m*: Entry point of the *Synthetic Moonquake Generator* application



Flowchart 1: Entry point of the *Synthetic Moonquake Generator* application



Flowchart 2: *Synthetic Moonquake Generator* Start Without Arguments



Flowchart 3: *Synthetic Moonquake Generator Start With Arguments*

5.2.2 Main Figure of the *Synthetic Moonquake Generator Application*

```

1 M.fig = figure('Name','Synthetic Moonquake Generator',...
2             'NumberTitle','off',...
3             'MenuBar','none',...
4             'Resize','on',...
5             'Visible','off');
6
7 monitors = get(0,'MonitorPositions');
8 numMonitors = size(monitors,1);
9
10 largestArea = 0;
11 largestMonitor = 1;
12 for i = 1:numMonitors
13     monitorArea = prod(monitors(i,3:4));
14     if monitorArea > largestArea
15         largestArea = monitorArea;

```

```

16         largestMonitor = i;
17     end
18 end
19
20 largestMonitorPosition = monitors(largestMonitor,:);
21
22 width = 0.90 * largestMonitorPosition(3);
23 height = 0.90 * largestMonitorPosition(4);
24 xPos = largestMonitorPosition(1) + (largestMonitorPosition(3) - width)/2;
25 yPos = largestMonitorPosition(2) + (largestMonitorPosition(4) - height)/2;
26
27 set(M.fg,'Position',[xPos,yPos,width,height],'Visible','on');
28
29 jframe = get(gcf,'javaframe');
30 jframe.setFigureIcon(javax.swing.ImageIcon(miniaturePath));
31
32 M.fl = uimenu('Parent',M.fg,'Label','File');
33
34 uimenu(M.fl,...
35     'Label','Open... ',...
36     'Callback',@OpenFileCallBack,...
37     'Accelerator','O');
38
39 uimenu(M.fl,...
40     'Label','Save As... ',...
41     'Callback',@SaveFileCallBack,...
42     'Accelerator','S');
43
44 uimenu(M.fl,...
45     'Label','Close',...
46     'Callback','close all;',...
47     'Separator','on',...
48     'Accelerator','C');
49
50 M.hl = uimenu('Parent',M.fg,'Label','?');
51
52 uimenu(M.hl,...
53     'Label','User Manual',...
54     'Callback',@UserManualCallBack);
55
56 uimenu(M.hl,...
57     'Label','File Associaion',...
58     'Callback',@FileAssociaionCallBack);
59
60 uimenu(M.hl,...
61     'Label','License Agreement',...
62     'Callback',@LicenseAgreementCallBack);
63
64 uimenu(M.hl, ...
65     'Label','About',...
66     'Separator','on',...
67     'Callback',@AboutCallBack);
68
69 M.tg = uitabgroup(M.fg,'SelectionChangedFcn',@MtgCallBack);
70
71 M.tb1 = uitab(M.tg,'Title','Synthetic Accelerogram');
72
73 M.tb2 = uitab(M.tg,'Title','Response Spectrum');
74
75 M.Seltab = 1;
76 M.change = 0;

```

```

77
78 GP.T = 85;
79 GP.fs = 100;
80 GP.tt = 1/GP.fs*(0:GP.T*60*GP.fs).';
81 if (mod(length(GP.tt),2)==1)
82     GP.tt(end) = [];
83 end
84 GP.RndAcc = randn(1,length(GP.tt));
85 GP.NGenAcc = GP.RndAcc/max(abs(GP.RndAcc));
86 GP.nfft = 2^(nextpow2(length(GP.tt))-6);
87 GP.noverlap = GP.nfft/2;
88 GP.window = GP.nfft;
89 [GP.psd,GP.ff] = pwelch(GP.NGenAcc,GP.nfft,GP.noverlap,GP.window,GP.fs);
90
91 FP.Seltab =1;
92 FP.fho = 2;
93 FP.fh = 0.4318;
94 FP.zh = 12.1362;
95 FP.fggo = 2;
96 FP.fgg = 0.7100;
97 FP.zg = 0.0723;
98 FP.fhone = 0.4318;
99 FP.zhone = 12.1362;
100 FP.fggone = GP.fs/3;
101 FP.fggmaxone = GP.fs/2 - 0.01;
102 FP.zgone = 0.0723;
103 FP.fhotwo = 2;
104 FP.fhtwo = 0.4318;
105 FP.fggotwo = 2;
106 FP.fggtwo = 0.7100;
107 FP.whone = FP.fhone*2*pi;
108 FP.wgone = FP.fggone*2*pi;
109 FP.numeqone = conv([1 0],[2*FP.zgone*FP.wgone FP.wgone^2]);
110 FP.deneqone = conv([1 2*FP.zhone*FP.whone FP.whone^2],[1 2*FP.zgone*FP.wgone FP.wgone^2]);
111 FP.Hone = tf(FP.numeqone,FP.deneqone);
112 FP.GenAccone = lsim(FP.Hone,GP.RndAcc,GP.tt);
113 FP.NGenAcc = FP.GenAccone/max(abs(FP.GenAccone));
114 FP.H = FP.Hone;
115 [FP.mag,FP.ph,FP.w] = bode(FP.H);
116
117 TP.Seltab = 1;
118 TP.Checked = 1;
119 TP.t1 = 10.3561;
120 TP.a = 1.6230;
121 TP.t2 = 12.2103;
122 TP.c = -3.1044;
123 TP.t1one = 10.3561;
124 TP.aone = 1.6230;
125 TP.t2one = 12.2103;
126 TP.cone = -3.1044;
127 TP.t1two = 10.3561;
128 TP.atwo = 1.6230;
129 TP.t2two = 12.2103;
130 TP.ctwo = -3.1044;
131 TP.t1min = 4.9112;
132 TP.t1max = 19.9832;
133 TP.amin = 0.1879;
134 TP.amax = 4.3490;
135 TP.t2min = 4.9164;
136 TP.t2max = 22.6608;
137 TP.cmin = -4.8428;

```

```

138 TP.cmax = -1.7363;
139 TP.PGA = 1;
140 TP.N01 = floor(TP.t1*length(GP.tt)/100);
141 TP.n01 = TP.N01;
142 TP.win01 = ((1:TP.n01)/TP.n01).^TP.a;
143 TP.win01 = TP.win01(:);
144 TP.N02 = ceil(TP.t2*length(GP.tt)/100);
145 TP.N03 = length(GP.tt);
146 TP.n03 = TP.N03 - TP.N02 + 1;
147 TP.win03 = exp(TP.c*(1:TP.n03)/TP.n03);
148 TP.win03 = TP.win03(:);
149 TP.n02 = length(GP.tt) - length(TP.win01) - length(TP.win03);
150 TP.win02 = ones(TP.n02,1);
151 TP.winN = [TP.win01; TP.win02; TP.win03];
152 TP.winNone = TP.winN;
153 TP.winNtwo = TP.winN;
154 TP.mqS = FP.NGenAcc.*TP.winN;
155 TP.mqSN = TP.mqS/max(abs(TP.mqS));
156 [TP.psd,TP.ff] = pwelch(TP.mqSN*TP.PGA,GP.nfft,GP.noverlap,GP.window,GP.fs);
157
158 ZZ.Z = 5;
159
160 TN.Tmin = 0.01;
161 TN.Tmax = 10;
162 TN.Tnum = 100;
163 TN.Tn=logspace(log10(TN.Tmin),log10(TN.Tmax),TN.Tnum);
164
165 GG.gg = 1.625;
166
167 guidata(M.fg,M);

```

Code 11: *Main.m*: Main window of the *Synthetic Moonquake Generator* application

M.fg
Class: figure { Main.m } Parent: <None> Name: Synthetic Moonquake Generator NumberTitle: off MenuBar: none Resize: on Visible: on
Child: M.fl (uimenu) { Main.m } Child: M.hl (uimenu) { Main.m } Child: M.tg (uitabgroup) { Main.m }
<i>Description: Creates the application main window</i>

M.fl
Class: uimenu { Main.m } Parent: M.fg { Main.m } Label: File
Child: Open... (uimenu) { Main.m } Child: Save As... (uimenu) { Main.m } Child: Close (uimenu) { Main.m }
<i>Description: Creates the file menu</i>

M.hl
Class: uimenu { Main.m } Parent: M.fg { Main.m } Label: ?
Child: User Manual (uimenu) { Main.m } Child: File Associaion (uimenu) { Main.m } Child: License Agreement (uimenu) { Main.m } Child: About (uimenu) { Main.m }
<i>Description: Creates help menu</i>

M.tg
Class: uitabgroup { Main.m } Parent: M.fg { Main.m } SelectionChangedFcn: MtgCallBack
Child: M.tb1 (uitab) { Main.m } Child: M.tb2 (uitab) { Main.m }
<i>Description: Creates tab group</i>

Hierarchy Diagram 1: Main window

M.fl
Class: uimenu { Main.m } Parent: M.fg { Main.m } Label: File
Child: Open... (uimenu) { Main.m } Child: Save As... (uimenu) { Main.m } Child: Close (uimenu) { Main.m }
<i>Description: Creates the file menu</i>

Open...
Class: uimenu { Main.m } Parent: M.fl { Main.m } Label: Open... Accelerator: O Callback: OpenFileCallBack
<i>Description: Open ‘Open ...’ Dialog</i>

Save As...
Class: uimenu { Main.m } Parent: M.fl { Main.m } Label: Save As... Accelerator: S Callback: SaveFileCallBack Separator: on
<i>Description: Open ‘Save As’ Dialog</i>

Close
Class: uimenu { Main.m } Parent: M.fl { Main.m } Label: Close Accelerator: C Callback: 'close all;' Separator: on
<i>Description: Closes the application</i>

Hierarchy Diagram 2: File Menu

M.hl
Class: uimenu { Main.m } Parent: M.fg { Main.m } Label: ?
Child: User Manual (uimenu) { Main.m } Child: File Association (uimenu) { Main.m } Child: License Agreement (uimenu) { Main.m } Child: About (uimenu) { Main.m }
<i>Description: Creates help menu</i>

User Manual
Class: uimenu { Main.m } Parent: M.hl { Main.m } Label: User Manual Callback: UserManualCallBack
<i>Description: Opens User Manual</i>

File Association
Class: uimenu { Main.m } Parent: M.hl { Main.m } Label: File Association Callback: FileAssociaionCallBack
<i>Description: Opens File Association instructions</i>

License Agreement
Class: uimenu { Main.m } Parent: M.hl { Main.m } Label: License Agreement Callback: LicenseAgreementCallBack
<i>Description: Opens License Agreement</i>

About
Class: uimenu { Main.m } Parent: M.hl { Main.m } Label: About Callback: AboutCallBack Separator: on
<i>Description: Opens About message window</i>

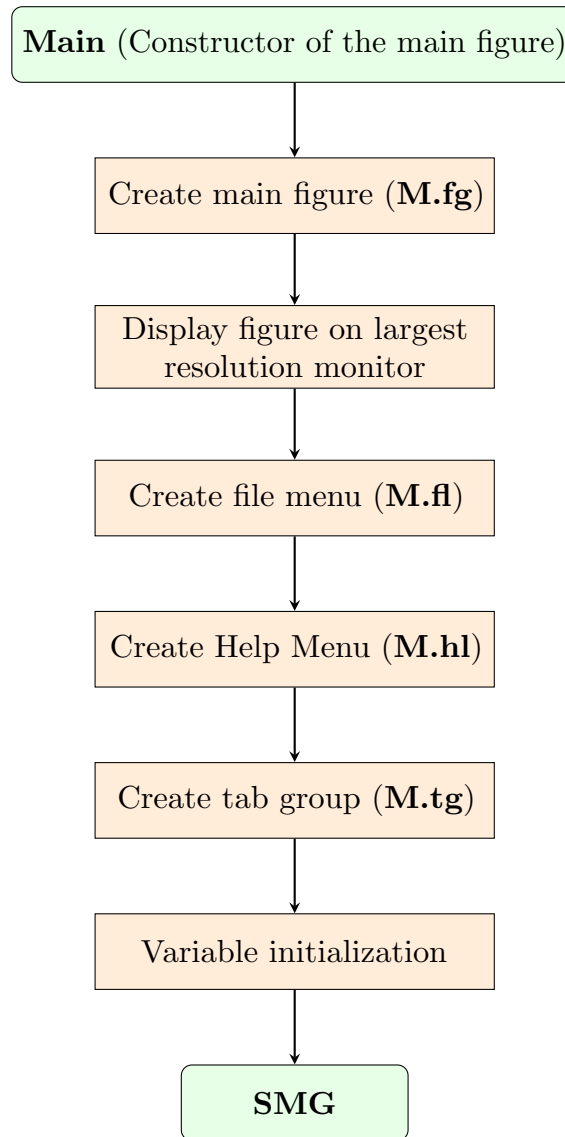
Hierarchy Diagram 3: Help menu

M.tg
Class: uitabgroup { Main.m } Parent: M.fg { Main.m } SelectionChangedFcn: MtgCallBack
Child: M.tb1 (uitab) { Main.m } Child: M.tb2 (uitab) { Main.m }
<i>Description: Creates tabgroup</i>

M.tb1
Class: uitab { Main.m } Parent: M.tg { Main.m } Title: Synthetic Accelerogram
Child: M.pn1 (uipanel) { GPpanel.m } Child: M.pn2 (uipanel) { FPpanel.m } Child: M.pn3 (uipanel) { TPpanel.m } Child: M.pn4 (uipanel) { PGApanel.m } Child: M.pn5 (uipanel) { PlotAcc.m } Child: <None>(uicontrol{Button}) { ExportAccFig.m } Child: <None>(uicontrol{Button}) { ExportAcc.m }
<i>Description: Creates Synthetic Accelerogram tab</i>

M.tb2
Class: uitab { Main.m } Parent: M.tg { Main.m } Title: Response Spectrum
Child: M.pn6 (uipanel) { ZZpanel.m } Child: M.pn8 (uipanel) { TNpanel.m } Child: M.pn9 (uipanel) { GGpanel.m } Child: M.pn10 (uipanel) { PlotSpec.m } Child: <None>(uicontrol{Button}) { ExportSpecFig.m } Child: <None>(uicontrol{Button}) { ExportSpec.m }
<i>Description: Creates Response Spectrum tab</i>

Hierarchy Diagram 4: Tab group



Flowchart 4: Main figure of the *Synthetic Moonquake Generator* application

```

1 function OpenFileCallBack(~,~,~)
2
3     global M GP FP TP ZZ TN GG SP miniaturePath
4
5     [fileName, filePath] = uigetfile('*.smg', 'Select a ''Synthetic Moonquake Generator''
6     ↪ file');
7
8     jframe = get(gcf,'javaframe');
9     jframe.setFigureIcon(javax.swing.ImageIcon(miniaturePath));
10
11     if isequal(fileName, 0) || isequal(filePath, 0)
12         msg1 = msgbox('File selection was canceled.');

```

```

29 set(FP.fggSlider1,'value',FP.fggone);
30 set(FP.fggedit1,'string',num2str(FP.fggone,'%3f'));
31 set(FP.fhSlider1,'value',FP.fhone);
32 set(FP.fhedit1,'string',num2str(FP.fhone,'%3f'));
33 set(FP.zgSlider1,'value',FP.zgone);
34 set(FP.zgedit1,'string',num2str(FP.zgone,'%3f'));
35 set(FP.fhoedit2,'string',num2str(FP.fhotwo,'%0f'));
36 set(FP.fggoedit2,'string',num2str(FP.fggotwo,'%0f'));
37 set(FP.fggedit2,'string',num2str(FP.fggtwo,'%3f'));
38 set(FP.fhedit2,'string',num2str(FP.fhtwo,'%3f'));
39
40 set(TP.t1Slider,'min',TP.t1min,'max',TP.t1max,'value',TP.t1one);
41 set(TP.aSlider,'min',TP.amin,'max',TP.amax,'value',TP.aone);
42 set(TP.t2Slider,'min',TP.t2min,'max',TP.t2max,'value',TP.t2one);
43 set(TP.cSlider,'min',TP.cmin,'max',TP.cmax,'value',TP.cone);
44 set(TP.t1edit,'string',num2str(TP.t1one,'%3f'));
45 set(TP.aedit,'string',num2str(TP.aone,'%3f'));
46 set(TP.t2edit,'string',num2str(TP.t2one,'%3f'));
47 set(TP.cedit,'string',num2str(TP.cone,'%3f'));
48
49 TPslid = [0.025 0.70-0.05 0.725 0.045];
50 TPlbl = [0 TPslid(2)-0.03 0.15 0.03];
51 Tnticks = 5;
52 Tlblmargin = 0.05;
53 Tlblvals = linspace(TP.t1min,TP.t1max,Tnticks);
54 for k = 1:Tnticks
55     Txpos = (Tlblmargin + (k-1)*(TPslid(3) - 2*Tlblmargin)/(Tnticks-1)) - TPlbl(3)/2;
56     TPlbl(1) = 0.025 + Txpos;
57     uicontrol('Parent',TP.tb1,...
58              'Units','normalized',...
59              'FontSize',6,...
60              'HorizontalAlignment','center',...
61              'Style','text',...
62              'Position',TPlbl,...
63              'String',sprintf('%3f',Tlblvals(k)),...
64              'Tag','t1Slider1ticklabel');
65 end
66
67 TPslid = [0.025 0.50-0.05 0.725 0.045];
68 TPlbl = [0 TPslid(2)-0.03 0.15 0.03];
69 Tnticks = 5;
70 Tlblmargin = 0.05;
71 Tlblvals = linspace(TP.amin,TP.amax,Tnticks);
72 for k = 1:Tnticks
73     Txpos = (Tlblmargin + (k-1)*(TPslid(3) - 2*Tlblmargin)/(Tnticks-1)) - TPlbl(3)/2;
74     TPlbl(1) = 0.025 + Txpos;
75     uicontrol('Parent',TP.tb1,...
76              'Units','normalized',...
77              'FontSize',6,...
78              'HorizontalAlignment','center',...
79              'Style','text',...
80              'Position',TPlbl,...
81              'String',sprintf('%3f',Tlblvals(k)),...
82              'Tag','aslider1ticklabel');
83 end
84
85 TPslid = [0.025 0.30-0.05 0.725 0.045];
86 TPlbl = [0 TPslid(2)-0.03 0.15 0.03];
87 Tnticks = 5;
88 Tlblmargin = 0.05;
89 Tlblvals = linspace(TP.t2min,TP.t2max,Tnticks);

```

```

90     for k = 1:Tnticks
91         Txpos = (Tlblmargin + (k-1)*(TPslid(3) - 2*Tlblmargin)/(Tnticks-1)) - TPlbl(3)/2;
92         TPlbl(1) = 0.025 + Txpos;
93         uicontrol('Parent',TP.tb1,...
94                 'Units','normalized',...
95                 'FontSize',6,...
96                 'HorizontalAlignment','center',...
97                 'Style','text',...
98                 'Position',TPlbl,...
99                 'String',sprintf('%.3f',Tlblvals(k)),...
100                'Tag','t2slider1ticklabel');
101     end
102
103     TPslid = [0.025 0.10-0.05 0.725 0.045];
104     TPlbl = [0 TPslid(2)-0.03 0.15 0.03];
105     Tnticks = 5;
106     Tlblmargin = 0.05;
107     Tlblvals = linspace(TP.cmin,TP.cmax,Tnticks);
108     for k = 1:Tnticks
109         Txpos = (Tlblmargin + (k-1)*(TPslid(3) - 2*Tlblmargin)/(Tnticks-1)) - TPlbl(3)/2;
110         TPlbl(1) = 0.025 + Txpos;
111         uicontrol('Parent',TP.tb1,...
112                 'Units','normalized',...
113                 'FontSize',6,...
114                 'HorizontalAlignment','center',...
115                 'Style','text',...
116                 'Position',TPlbl,...
117                 'String',sprintf('%.3f',Tlblvals(k)),...
118                 'Tag','cslider1ticklabel');
119     end
120
121     set(TP.t1edit2,'string',num2str(TP.t1two,'%.3f'));
122     set(TP.aedit2,'string',num2str(TP.atwo,'%.3f'));
123     set(TP.t2edit2,'string',num2str(TP.t2two,'%.3f'));
124     set(TP.cedit2,'string',num2str(TP.ctwo,'%.3f'));
125
126     set(TP.PGAedit, 'String',num2str(TP.PGA,'%.3f'));
127
128     set(M.LN1,'XData',GP.tt/60,'YData',GP.NGenAcc);
129     set(M.LN2,'XData',GP.ff,'YData',db(GP.psd));
130     set(M.LN3,'XData',FP.w/2/pi,'YData',db(squeeze(FP.mag)));
131     set(M.LN4,'XData',FP.w/2/pi,'YData',squeeze(FP.ph));
132     set(M.LN5,'XData',GP.tt/60,'YData',TP.mqSN*TP.PGA);
133     set(M.LN5.Parent,'YLim',[-1.1 1.1]*TP.PGA);
134     set(M.LN6,'XData',GP.tt/60,'YData',TP.winN*TP.PGA);
135     set(M.LN7,'XData',GP.tt/60,'YData',envelope(TP.mqSN*TP.PGA,floor(length(
↵ TP.mqSN)/100),'rms'));
136     set(M.LN8,'XData',TP.ff,'YData',db(TP.psd));
137
138     set(ZZ.Zedit,'string',num2str(ZZ.Z,'%.3f'));
139     set(TN.Tminedit,'string',num2str(TN.Tmin,'%.3f'));
140     set(TN.Tmaxedit,'string',num2str(TN.Tmax,'%.3f'));
141     set(TN.Tnumedit,'string',num2str(TN.Tnum,'%.0f'));
142     set(GG.ggedit,'string',num2str(GG.gg,'%.3f'));
143
144     SpecMQ
145
146     set(M.LN9,'XData',TN.Tn,'YData',SP.ResD);
147     set(M.LN10,'XData',TN.Tn,'YData',SP.ResV);
148     set(M.LN11,'XData',TN.Tn,'YData',SP.ResA);
149     set(M.LN12,'XData',TN.Tn,'YData',SP.ResSV);

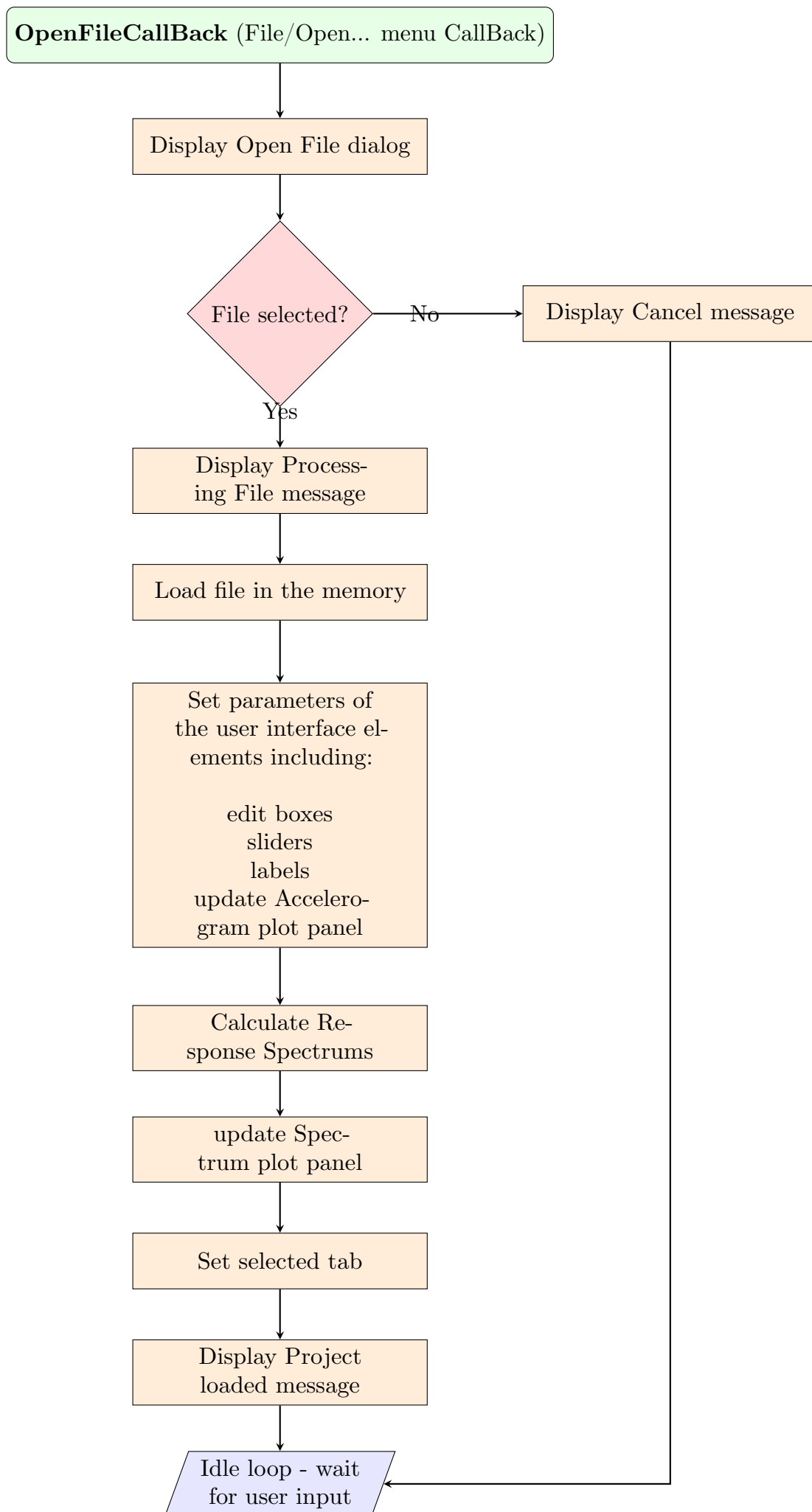
```

```

150     set(M.LN13,'XData',TN.Tn,'YData',SP.ResSA);
151
152     if FP.Seltab == 1
153         set(FP.tg,'SelectedTab',FP.tb1)
154     elseif FP.Seltab == 2
155         set(FP.tg,'SelectedTab',FP.tb2)
156     end
157
158     if TP.Seltab == 1
159         set(TP.tg,'SelectedTab',TP.tb1)
160     elseif FP.Seltab == 2
161         set(TP.tg,'SelectedTab',TP.tb2)
162     end
163
164     if TP.Checked == 1
165         set(TP.radioButton1,'Value',1);
166     elseif TP.Checked == 2
167         set(TP.radioButton2,'Value',1);
168     elseif TP.Checked == 3
169         set(TP.radioButton3,'Value',1);
170     end
171
172     if M.Seltab == 1
173         set(M.tg,'SelectedTab',M.tb1)
174     elseif M.Seltab == 2
175         set(M.tg,'SelectedTab',M.tb2)
176     end
177
178     msg3 = msgbox(['Project loaded: ', selectedFile]);
179     jframe = get(msg3,'javaframe');
180     jframe.setFigureIcon(javax.swing.ImageIcon(miniaturePath));
181
182     guidata(M.fg,M);
183 end
184 end

```

Code 12: *OpenFileCallback.m*: Callback for the *Open...* option of the *File* menu



Flowchart 5: File/Open menu Callback

```

1 function SaveFileCallBack(~,~,~)
2
3     global M GP FP TP ZZ TN GG SP miniaturePath
4
5     [fileName, filePath] = uiputfile('*.smg', 'Save as ''Synthetic Moonquake Generator''
    ↪ file');

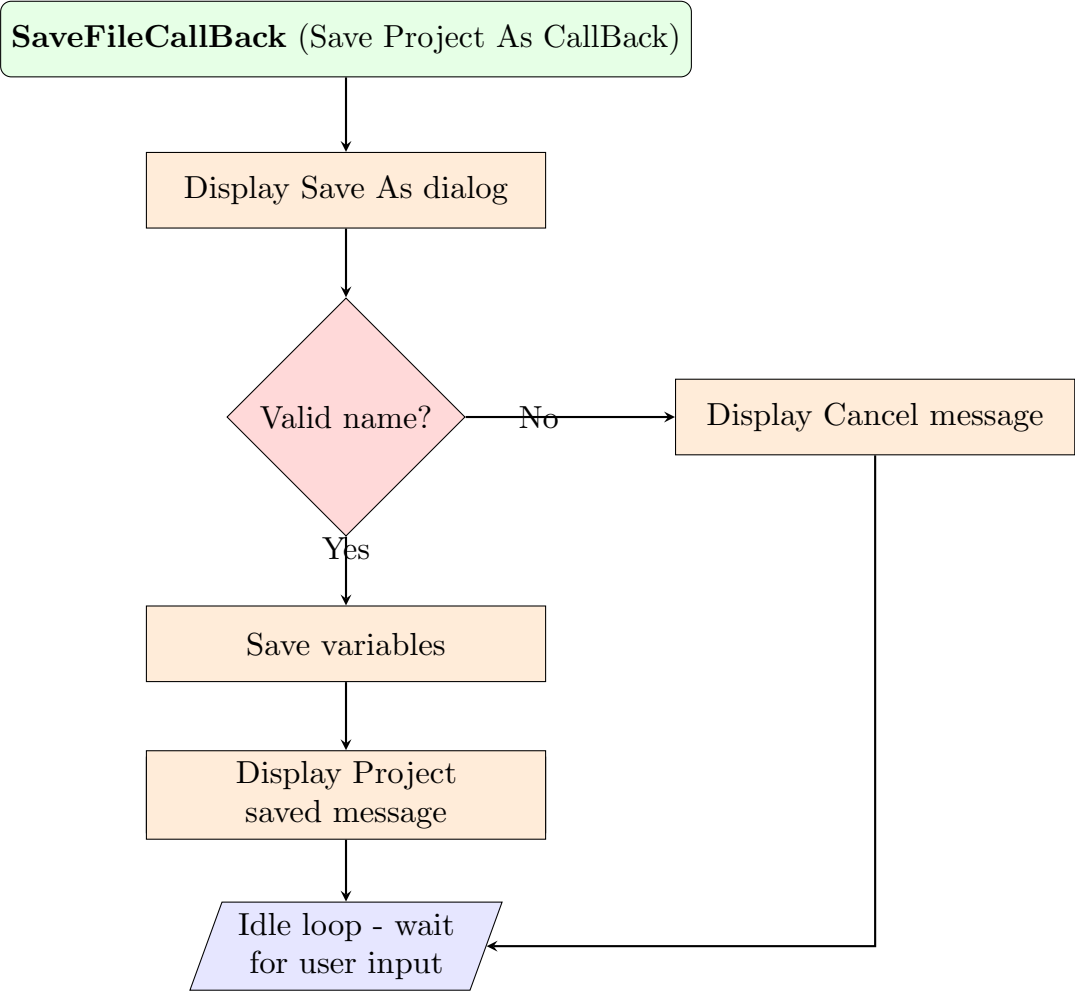
```

```

6
7     jframe = get(gcf,'javaframe');
8     jframe.setFigureIcon(javafx.swing.ImageIcon(miniaturePath));
9
10    if isequal(fileName, 0) || isequal(filePath, 0)
11        msg1 = msgbox('File save was canceled. ');
12        jframe = get(msg1,'javaframe');
13        jframe.setFigureIcon(javafx.swing.ImageIcon(miniaturePath));
14    else
15        saveFilePath = fullfile(filePath, fileName);
16
17        save(saveFilePath, 'M', 'GP', 'FP', 'TP', 'ZZ', 'TN', 'GG', 'SP');
18
19        msg2 = msgbox(['Project saved to: ', saveFilePath]);
20        jframe = get(msg2,'javaframe');
21        jframe.setFigureIcon(javafx.swing.ImageIcon(miniaturePath));
22    end
23 end

```

Code 13: *SaveFileCallBack.m*: Callback for the *Save As...* option of the *File* menu



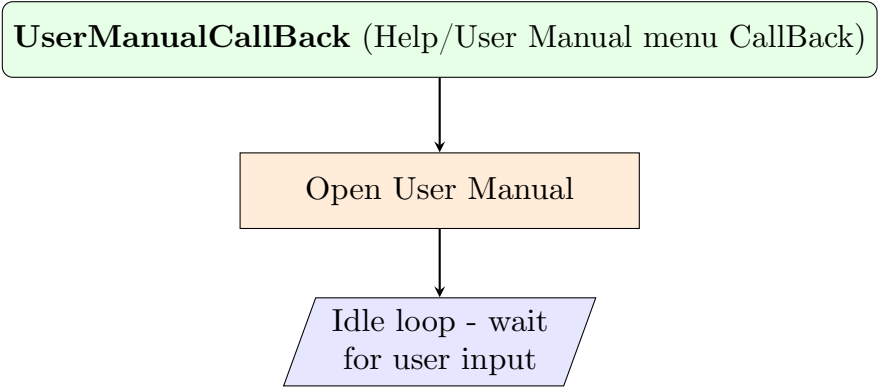
Flowchart 6: File/Save menu CallBack

```

1 function UserManualCallBack(~,~,~)
2
3     global userManualPath
4
5     winopen(userManualPath);
6
7 end

```

Code 14: *UserManualCallBack.m*: Callback for the *User Manual* option of the ? (*Help*) menu

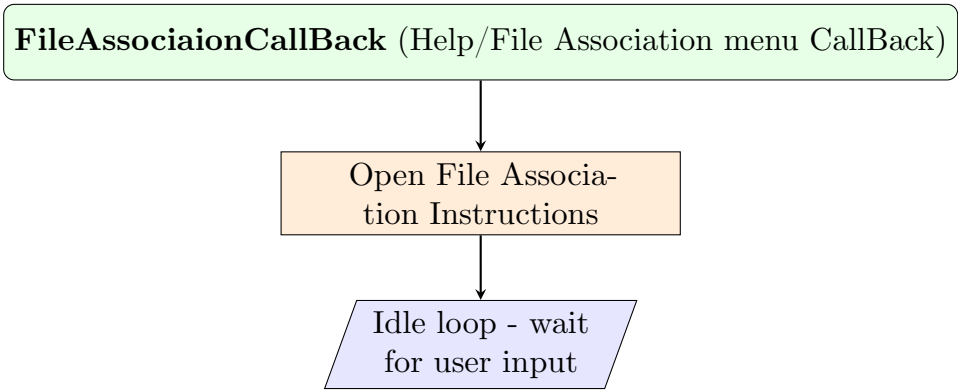


Flowchart 7: Help/User Manual menu Callback

```

1 function FileAssociaionCallBack(~,~,~)
2
3     global fileAssociationPath
4
5     winopen(fileAssociationPath);
6
7 end
  
```

Code 15: *FileAssociaionCallBack.m*: Callback for the *File Associaion* option of the ? (*Help*) menu

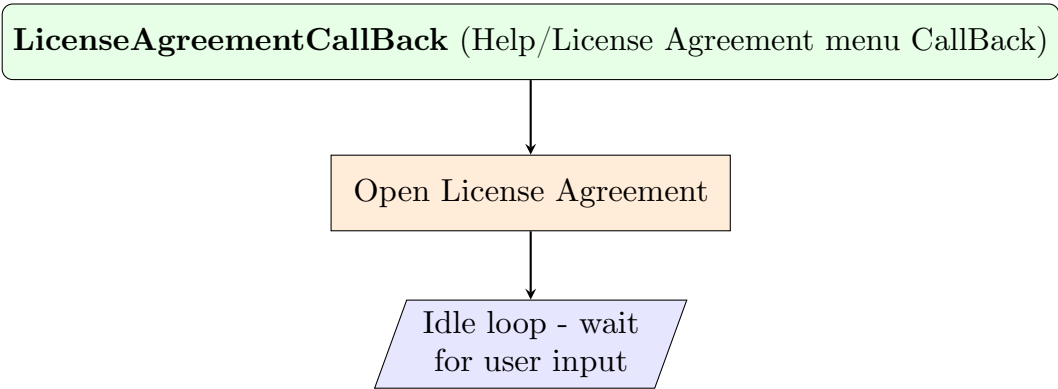


Flowchart 8: Help/File Association menu Callback

```

1 function LicenseAgreementCallBack(~,~,~)
2
3     global licenseAgreementPath
4
5     winopen(licenseAgreementPath);
6
7 end
  
```

Code 16: *LicenseAgreementCallBack.m*: Callback for the *License Agreement* option of the ? (*Help*) menu



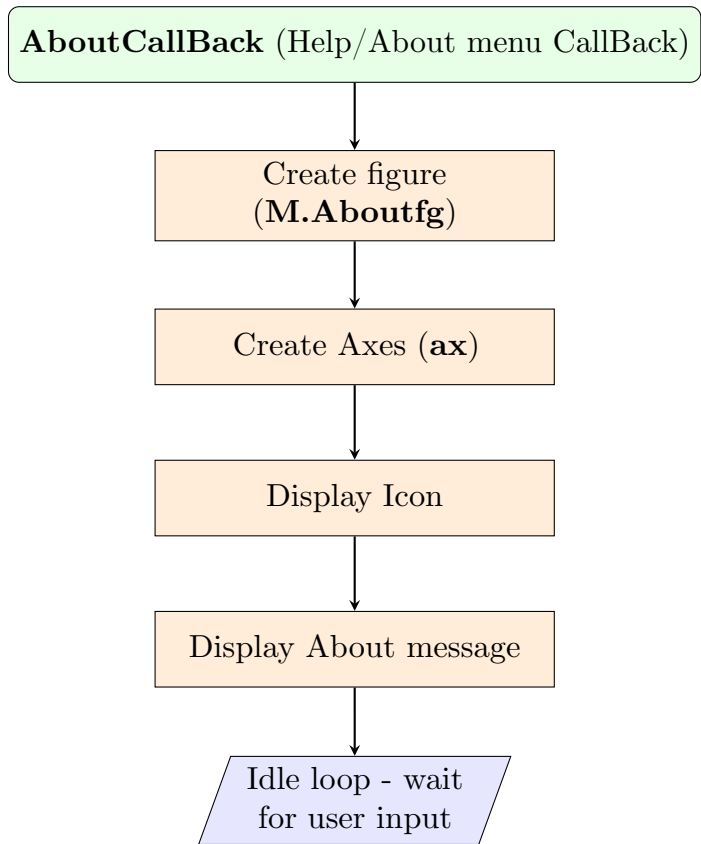
Flowchart 9: Help/License Agreement menu Callback

```

1 function AboutCallBack(~,~,~)
2
3     global miniaturePath aboutIconPath
4
5     M.Aboutfg = figure('Name','About',...
6         'Position',[100, 100, 300, 400],...
7         'NumberTitle','off',...
8         'MenuBar','none',...
9         'Resize','off');
10
11     ax = axes('Parent', M.Aboutfg, 'Position', [0.4, 0.75, 0.2, 0.2]);
12     imshow(imread(aboutIconPath), 'Parent', ax);
13
14     uicontrol('Style', 'text',...
15         'Position',[40, 40, 220, 250],...
16         'String', ...
17         {'Synthetic Moonquake Generator',...
18         'Version: 1.0'},...
19         'HorizontalAlignment','center',...
20         'FontSize',10,...
21         'FontWeight','bold');
22
23
24
25     uicontrol('Style', 'text',...
26         'Position',[40, -25, 220, 250],...
27         'String', ...
28         {'Oscar Arcila',...
29         'oscar.arcila@correounivalle.edu.co',...
30         '',...
31         'Daniel Gomez',...
32         'daniel.gomez@correounivalle.edu.co',...
33         '',...
34         'Alejandro Cruz',...
35         'alejandro.cruz@correounivalle.edu.co',...
36         '',...
37         '',...
38         'School of Civil Engineering',...
39         'Universidad del Valle',...
40         'Colombia'},...
41         'HorizontalAlignment','center',...
42         'FontSize',8);
43
44     jframe = get(M.Aboutfg,'javaframe');
45     jframe.setFigureIcon(javax.swing.ImageIcon(miniaturePath));
46
47     movegui(M.Aboutfg, 'center');
48
49 end

```

Code 17: *AboutCallBack.m*: Callback for the *About* option of the ? (*Help*) menu

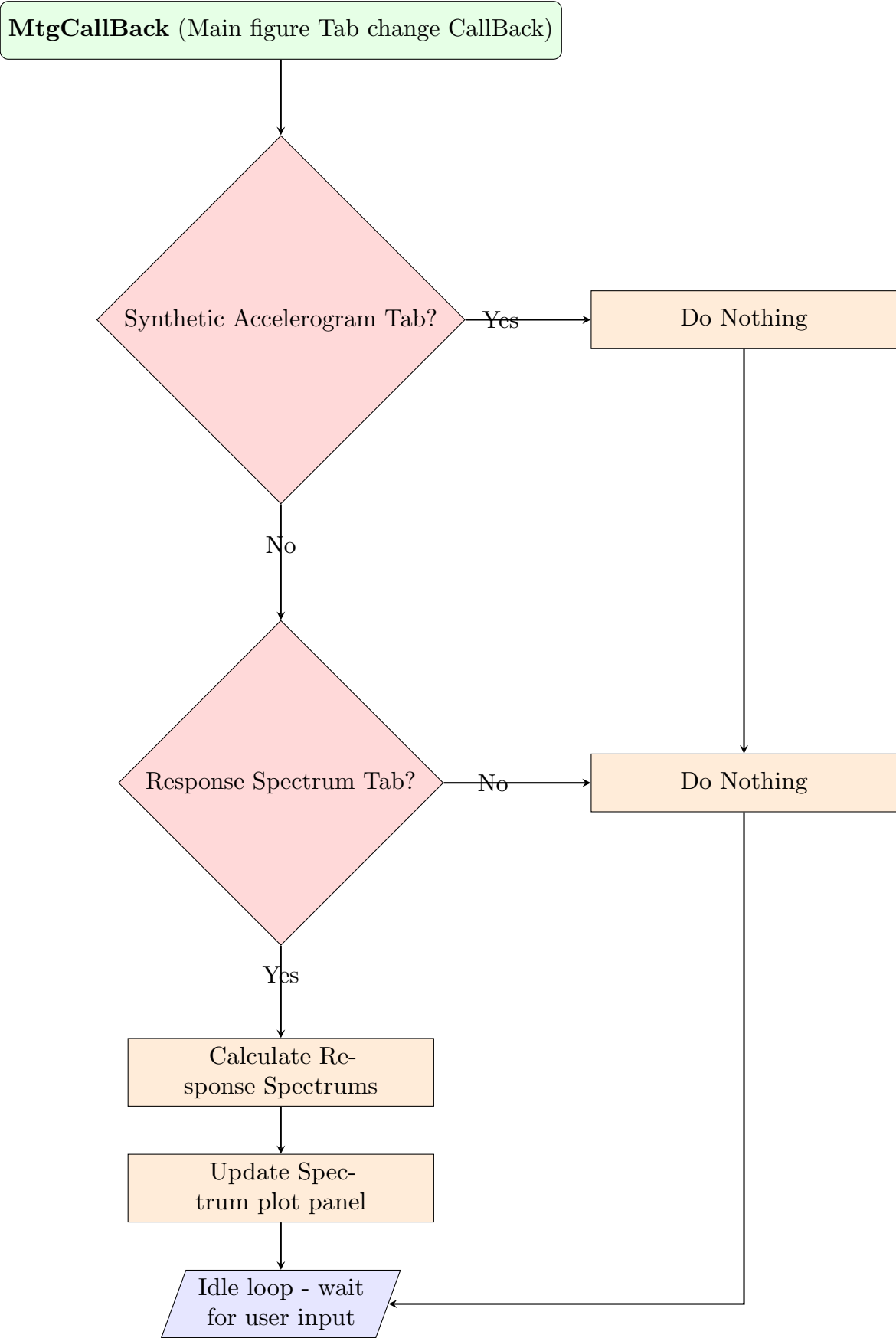


Flowchart 10: Help/About menu Callback

```

1 function MtgCallBack(Source,EventData,~)
2
3     global M GP FP TP ZZ TN GG SP
4
5     t = EventData.NewValue.Title;
6
7     if strcmp(t,'Synthetic Accelerogram')
8         %Do Nothing
9     elseif strcmp(t,'Response Spectrum')
10        if M.change == 1
11
12            SpecMQ
13
14            set(M.LN9,'XData',TN.Tn,'YData',SP.ResD);
15            set(M.LN10,'XData',TN.Tn,'YData',SP.ResV);
16            set(M.LN11,'XData',TN.Tn,'YData',SP.ResA);
17            set(M.LN12,'XData',TN.Tn,'YData',SP.ResSV);
18            set(M.LN13,'XData',TN.Tn,'YData',SP.ResSA);
19
20            M.change = 0;
21        end
22    end
23
24    guidata(Source,M);
25 end
  
```

Code 18: *MtgCallBack.m*: Callback for the tab change of the main window



Flowchart 11: Main figure Tab Change CallBack

5.2.3 White Noise Panel

```
1 M.pn1 = uipanel('Parent',M.tb1,...
2             'Units','normalized',...
3             'Position',[0.025 0.8805 0.25 0.10],...
4             'Title','White Noise');
5
6 GP.Tlabel = uicontrol('Parent',M.pn1,...
7             'Units','normalized',...
8             'FontSize',8,...
9             'HorizontalAlignment','left',...
10            'Style','text',...
11            'Position',[0.15 0.675 0.95 0.20],...
```

```

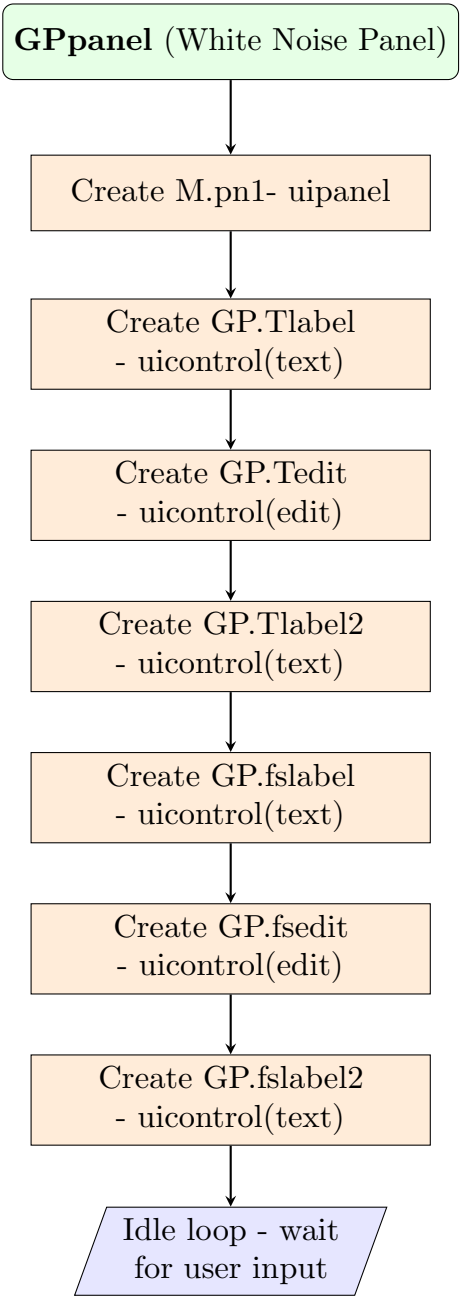
12         'String','Event Duration:');
13
14 GP.Tedit = uicontrol('Parent',M.pn1,...
15                     'style','edit',...
16                     'FontSize',8,...
17                     'units','normalized',...
18                     'position',[0.425 0.675 0.20 0.25],...
19                     'string',num2str(GP.T,'%2f'),...
20                     'Tooltip','Write Desired Event Duration [ min ]',...
21                     'callback',{@GPCallBack,'T'});
22
23 GP.Tlabel2 = uicontrol('Parent',M.pn1,...
24                       'Units','normalized',...
25                       'FontSize',8,...
26                       'HorizontalAlignment','left',...
27                       'Style','text',...
28                       'Position',[0.65 0.675 0.95 0.20],...
29                       'String','[ min ]');
30
31 GP.fslabel = uicontrol('Parent',M.pn1,...
32                       'Units','normalized',...
33                       'FontSize',8,...
34                       'HorizontalAlignment','left',...
35                       'Style','text',...
36                       'Position',[0.075 0.225 0.95 0.20],...
37                       'String','Sampling Frequency:');
38
39 GP.fsedit = uicontrol('Parent',M.pn1,...
40                      'style','edit',...
41                      'FontSize',8,...
42                      'units','normalized',...
43                      'position',[0.425 0.225 0.20 0.25],...
44                      'string',num2str(GP.fs,'%2f'),...
45                      'Tooltip','Write Desired Record Sampling Frequency [ Hz ]',...
46                      'callback',{@GPCallBack,'fs'});
47
48 GP.fslabel2 = uicontrol('Parent',M.pn1,...
49                       'Units','normalized',...
50                       'FontSize',8,...
51                       'HorizontalAlignment','left',...
52                       'Style','text',...
53                       'Position',[0.65 0.225 0.95 0.20],...
54                       'String','[ Hz ]');

```

Code 19: *GPpanel.m*: *White Noise* panel of the *Synthetic Accelerogram* tab

M.pn1
Class: uipanel { GPpanel.m } Parent: M.tb1 { Main.m } Title:White Noise
Child: GP.Tlabel (uicontrol{Text}) { GPpanel.m } Child: GP.Tedit (uicontrol{Edit}) { GPpanel.m } Callback: GPCallBack.T Child: GP.Tlabel2 (uicontrol{Text}) { GPpanel.m } Child: GP.fslabel (uicontrol{Text}) { GPpanel.m } Child: GP.fsedit (uicontrol{Edit}) { GPpanel.m } Callback: GPCallBack.fs Child: GP.fslabel2 (uicontrol{Text}) { GPpanel.m }
<i>Description: Creates White Noise Panel</i>

Hierarchy Diagram 5: White Noise Panel



Flowchart 12: White Noise Panel

```

1 function GPCallbackfn(Source,~,Param)
2
3     global M GP FP TP ZZ TN GG SP
4
5     if isnan(str2double(get(Source,'String')))
6         set(Source,'string',num2str(GP.(Param), '%.2f'));

```

```

7 else
8     M.change = 1;
9
10    GP.(Param) = str2double(get(Source,'String'));
11
12    if strcmp(Param,'T')
13        if GP.T < 25
14            GP.T = 25;
15        end
16    end
17
18    if strcmp(Param,'fs')
19        if GP.fs < 6
20            GP.fs = 6;
21        end
22    end
23
24    set(GP.Tedit,'string',num2str(GP.T,'%.2f'));
25    set(GP.fsedit,'string',num2str(GP.fs,'%.2f'));
26
27    GP.tt = 1/GP.fs*(0:GP.T*60*GP.fs).';
28    if (mod(length(GP.tt),2)==1)
29        GP.tt(end) = [];
30    end
31    GP.RndAcc = randn(1,length(GP.tt));
32    GP.NGenAcc = GP.RndAcc/max(abs(GP.RndAcc));
33    GP.nfft = 2^(nextpow2(length(GP.tt))-6);
34    GP.noverlap = GP.nfft/2;
35    GP.window = GP.nfft;
36    [GP.psd,GP.ff] = pwelch(GP.NGenAcc,GP.nfft,GP.noverlap,GP.window,GP.fs);
37    set(M.LN1,'XData',GP.tt/60,'YData',GP.NGenAcc);
38    set(M.LN2,'XData',GP.ff,'YData',db(GP.psd));
39
40    if FP.fhone > GP.fs/2 - 0.01
41        FP.fhone = GP.fs/2 - 0.01;
42    end
43    FP.fggone = GP.fs/3;
44    FP.fggmaxone = GP.fs/2 - 0.01;
45    set(FP.fhSlider1,'value',FP.fhone);
46    set(FP.fhedit1,'string',num2str(FP.fhone,'%.3f'));
47    set(FP.fggSlider1,'min',0.6,'max',FP.fggmaxone,'value',FP.fggone);
48    set(FP.fggedit1,'string',num2str(FP.fggone,'%.3f'));
49    if FP.fhtwo > GP.fs/2 - 0.01
50        FP.fhtwo = GP.fs/2 - 0.01;
51    end
52    if FP.fggtwo > GP.fs/2 - 0.01
53        FP.fggtwo = GP.fs/2 - 0.01;
54    end
55    set(FP.fhedit2,'string',num2str(FP.fhtwo,'%.3f'));
56    set(FP.fggedit2,'string',num2str(FP.fggtwo,'%.3f'));
57
58    TPslid = [0.025 0.35-0.05 0.725 0.20];
59    TPlbl = [0 TPslid(2)-0.10 0.20 0.1];
60    Tnticks = 5;
61    Tlblmargin = 0.05;
62    Tlblvals = linspace(0.6,FP.fggmaxone,Tnticks);
63    for k = 1:Tnticks
64        Txpos = (Tlblmargin + (k-1)*(TPslid(3) - 2*Tlblmargin)/(Tnticks-1)) - TPlbl(3)/2;
65        TPlbl(1) = 0.025 + Txpos;
66        uicontrol('Parent',FP.tb1,...
67            'Units','normalized',...

```

```

68         'FontSize',6,...
69         'HorizontalAlignment','center',...
70         'Style','text',...
71         'Position',TPlbl,...
72         'String',sprintf('%.3f',Tlblvals(k)));
73     end
74
75     FP.zglabel1 = uicontrol('Parent',FP.tb1,...
76         'Units','normalized',...
77         'FontSize',8,...
78         'HorizontalAlignment','center',...
79         'Style','text',...
80         'Position',[0.025 0.10-0.05 0.95 0.20],...
81         'String','Low-pass Damping [ - ]');
82
83     FP.zgSlider1 = uicontrol('Parent',FP.tb1,...
84         'style','slider',...
85         'units','normalized',...
86         'position',[0.025 0.125-0.05 0.725 0.25*10/30],...
87         'min',0.019,...
88         'max',0.153,...
89         'value',FP.zgone,...
90         'SliderStep',[0.01 0.01],...
91         'Tooltip','Select Desired Low-pass Damping [ - ]',...
92         'callback',{@FPCallBack1,'zg'});
93
94     TPslid = [0.025 0.125-0.05 0.725 0.20];
95     TPlbl = [0 TPslid(2)-0.10 0.20 0.1];
96     Tnticks = 5;
97     Tlblmargin = 0.05;
98     Tlblvals = linspace(0.019,0.153,Tnticks);
99     for k = 1:Tnticks
100         Txpos = (Tlblmargin + (k-1)*(TPslid(3) - 2*Tlblmargin)/(Tnticks-1)) - TPlbl(3)/2;
101         TPlbl(1) = 0.025 + Txpos;
102         uicontrol('Parent',FP.tb1,...
103             'Units','normalized',...
104             'FontSize',6,...
105             'HorizontalAlignment','center',...
106             'Style','text',...
107             'Position',TPlbl,...
108             'String',sprintf('%.3f',Tlblvals(k)));
109     end
110
111     FP.zgedit1 = uicontrol('Parent',FP.tb1,...
112         'style','edit',...
113         'units','normalized',...
114         'position',[0.775 0.125-0.05 0.20 0.25*10/30],...
115         'string',num2str(FP.zgone,'%.3f'),...
116         'Tooltip','Write Desired Low-pass Damping [ - ]',...
117         'callback',{@FPCallBack2,'zg'});
118
119     if FP.Seltab == 1
120         FP.whone = FP.fhone*2*pi;
121         FP.wgone = FP.fggone*2*pi;
122         FP.numeqone = conv([1 0],[2*FP.zgone*FP.wgone FP.wgone^2]);
123         FP.deneqone = conv([1 2*FP.zhone*FP.whone FP.whone^2],[1 2*FP.zgone*FP.wgone
124             ↪ FP.wgone^2]);
125         FP.Hone = tf(FP.numeqone,FP.deneqone);
126         FP.H = FP.Hone;
127         FP.GenAccone = lsim(FP.Hone,GP.RndAcc,GP.tt);
128         FP.NGenAcc = FP.GenAccone/max(abs(FP.GenAccone));

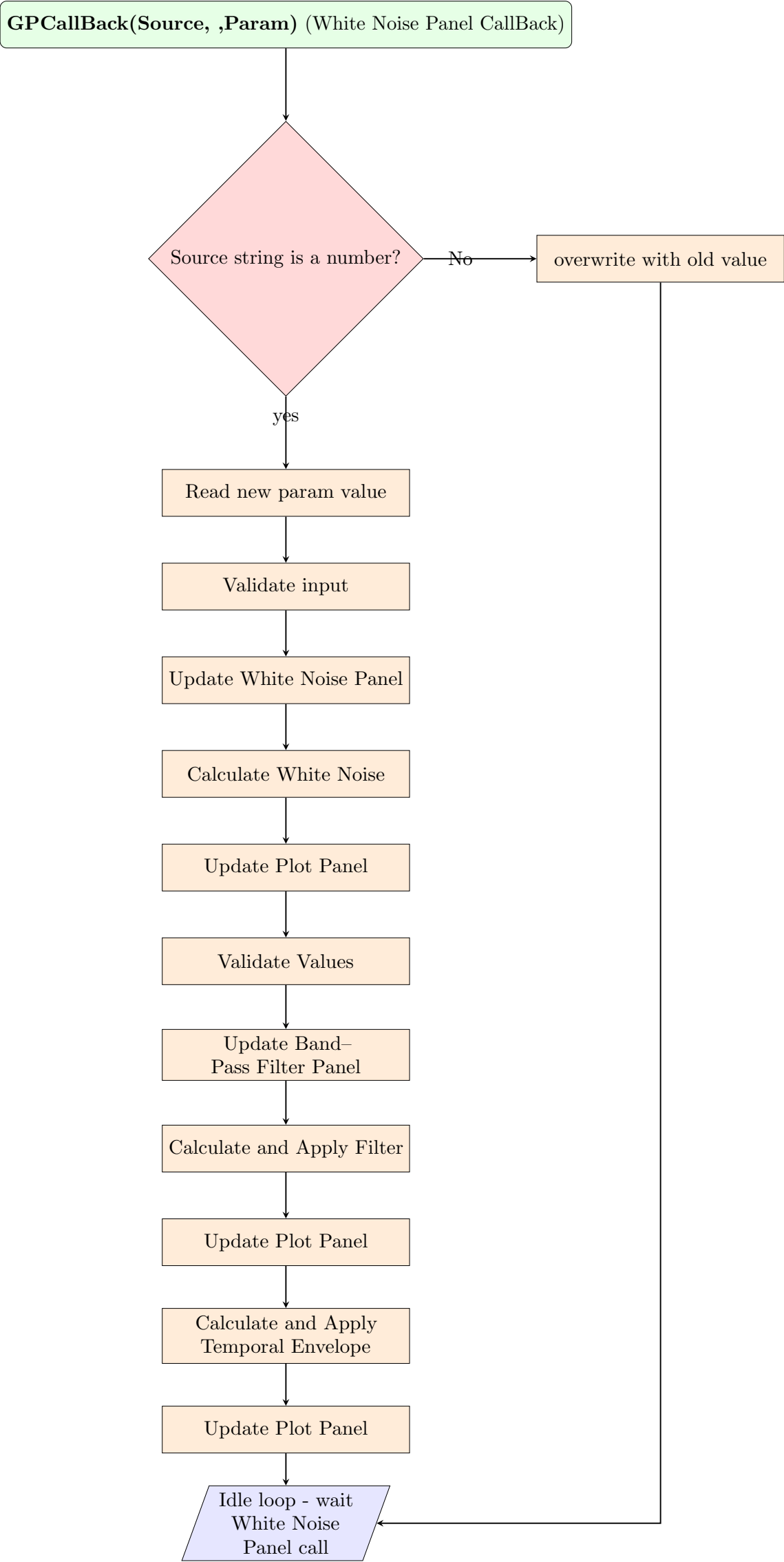
```

```

128 elseif FP.Seltab == 2
129     [FP.fhbtwo,FP.fhatwo] = butter(FP.fhotwo,FP.fhtwo/(GP.fs/2),'high');
130     [FP.fggbtwo,FPtwo.fgga] = butter(FP.fggotwo,FP.fggtwo/(GP.fs/2),'low');
131     FP.btwo = conv(FP.fhbtwo,FP.fggbtwo);
132     FP.atwo = conv(FP.fhatwo,FP.fggatwo);
133     FP.Htwo = tf(FP.btwo,FP.atwo);
134     FP.H = FP.Htwo;
135     FP.GenAcctwo = filtfilt(FP.btwo,FP.atwo,GP.RndAcc);
136     FP.GenAcctwo = FP.GenAcctwo(:);
137     FP.NGenAcc = FP.GenAcctwo/max(abs(FP.GenAcctwo));
138 end
139 [FP.mag,FP.ph,FP.w] = bode(FP.H);
140 set(M.LN3,'XData',FP.w/2/pi,'YData',db(squeeze(FP.mag)));
141 set(M.LN4,'XData',FP.w/2/pi,'YData',squeeze(FP.ph));
142
143 if TP.Seltab == 1
144     TP.N01 = floor(TP.t1one*length(GP.tt)/100);
145     TP.n01 = TP.N01;
146     TP.win01 = ((1:TP.n01)/TP.n01).^TP.aone;
147     TP.win01 = TP.win01(:);
148     TP.N02 = ceil(TP.t2one*length(GP.tt)/100);
149     TP.N03 = length(GP.tt);
150     TP.n03 = TP.N03 - TP.N02 + 1;
151     TP.win03 = exp(TP.cone*(1:TP.n03)/TP.n03);
152     TP.win03 = TP.win03(:);
153     TP.n02 = length(GP.tt) - length(TP.win01) - length(TP.win03);
154     TP.win02 = ones(TP.n02,1);
155     TP.winNone = [TP.win01; TP.win02; TP.win03];
156     TP.winN = TP.winNone;
157 if TP.Seltab == 2
158     TP.N01 = floor(TP.t1two*length(GP.tt)/100);
159     TP.n01 = TP.N01;
160     TP.win01 = ((1:TP.n01)/TP.n01).^TP.atwo;
161     TP.win01 = TP.win01(:);
162     TP.N02 = ceil(TP.t2two*length(GP.tt)/100);
163     TP.N03 = length(GP.tt);
164     TP.n03 = TP.N03 - TP.N02 + 1;
165     TP.win03 = exp(TP.ctwo*(1:TP.n03)/TP.n03);
166     TP.win03 = TP.win03(:);
167     TP.n02 = length(GP.tt) - length(TP.win01) - length(TP.win03);
168     TP.win02 = ones(TP.n02,1);
169     TP.winNtwo = [TP.win01; TP.win02; TP.win03];
170     TP.winN = TP.winNtwo;
171 end
172 TP.mqS = FP.NGenAcc.*TP.winN;
173 TP.mqSN = TP.mqS/max(abs(TP.mqS));
174 [TP.psd,TP.ff] = pwelch(TP.mqSN*TP.PGA,GP.nfft,GP.noverlap,GP.window,GP.fs);
175
176 set(M.LN5,'XData',GP.tt/60,'YData',TP.mqSN*TP.PGA);
177 set(M.LN5.Parent,'YLim',[-1.1 1.1]*TP.PGA);
178 set(M.LN6,'XData',GP.tt/60,'YData',TP.winN*TP.PGA);
179 set(M.LN7,'XData',GP.tt/60,'YData',envelope(TP.mqSN*TP.PGA,floor(length(GP
    ↪ TP.mqSN)/100),'rms'));
180 set(M.LN8,'XData',TP.ff,'YData',db(TP.psd));
181 end
182
183 guidata(Source,M);
184 end

```

Code 20: *GPCallback.m*: Callback for the *White Noise* panel of the *Synthetic Accelerogram* tab



Flowchart 13: White Noise Panel CallBack

5.2.4 Band-Pass Filter Panel

```
1 M.pn2 = uipanel('Parent',M.tb1,...
2             'Units','normalized',...
3             'Position',[0.025 0.575 0.25 0.30], ...
4             'Title','Band-pass Filter');
5
6 FP.tg = uitabgroup(M.pn2,'SelectionChangedFcn',@FPtgCallBack);
7
8 FP.tb1 = uitab(FP.tg,'Title','Observed Values');
9
10 FP.fhlabel1 = uicontrol('Parent',FP.tb1,...
11             'Units','normalized',...
12             'FontSize',8,...
13             'HorizontalAlignment','center',...
14             'Style','text',...
15             'Position',[0.025 0.775-0.05 0.95 0.20],...
16             'String','High-pass Cut Frequency [ Hz ]');
17
18 FP.fhSlider1 = uicontrol('Parent',FP.tb1,...
19             'style','slider',...
20             'units','normalized',...
21             'position',[0.025 0.80-0.05 0.725 0.25*10/30],...
22             'min',0.2,...
23             'max',0.7,...
24             'value',FP.fhone,...
25             'SliderStep',[0.01 0.01],...
26             'Tooltip','Select Desired High-pass Cut Frequency [ Hz ]',...
27             'callback',{@FPCallBack1,'fh'});
28
29 TPslid = [0.025 0.80-0.05 0.725 0.25*10/30];
30 TPlbl = [0 TPslid(2)-0.10 0.20 0.1];
31 Tnticks = 5;
32 Tlblmargin = 0.05;
33 Tlblvals = linspace(0.2,0.7,Tnticks);
34 for k = 1:Tnticks
35     Txpos = (Tlblmargin + (k-1)*(TPslid(3) - 2*Tlblmargin)/(Tnticks-1)) - TPlbl(3)/2;
36     TPlbl(1) = 0.025 + Txpos;
37     uicontrol('Parent',FP.tb1,...
38             'Units','normalized',...
39             'FontSize',6,...
40             'HorizontalAlignment','center',...
41             'Style','text',...
42             'Position',TPlbl,...
43             'String',sprintf('%.3f',Tlblvals(k)));
44 end
45
46 FP.fhedit1 = uicontrol('Parent',FP.tb1,...
47             'style','edit',...
48             'units','normalized',...
49             'position',[0.775 0.80-0.05 0.20 0.25*10/30],...
50             'string',num2str(FP.fhone,'%.3f'),...
51             'Tooltip','Write Desired High-pass Cut Frequency [ Hz ]',...
52             'callback',{@FPCallBack2,'fh'});
53
54 FP.zhlabel1 = uicontrol('Parent',FP.tb1,...
55             'Units','normalized',...
56             'FontSize',8,...
57             'HorizontalAlignment','center',...
58             'Style','text',...
59             'Position',[0.025 0.55-0.05 0.95 0.20],...
```

```

60         'String','High-pass Damping [ - ]');
61
62 FP.zhSlider1 = uicontrol('Parent',FP.tb1,...
63     'style','slider',...
64     'units','normalized',...
65     'position',[0.025 0.575-0.05 0.725 0.25*10/30],...
66     'min',0.08,...
67     'max',50,...
68     'value',FP.zhone,...
69     'SliderStep',[0.01 0.01],...
70     'Tooltip','Select Desired High-pass Damping [ - ]',...
71     'callback',{@FPCallBack1,'zh'});
72
73 TPslid = [0.025 0.575-0.05 0.725 0.25*10/30];
74 TPlbl = [0 TPslid(2)-0.10 0.20 0.1];
75 Tnticks = 5;
76 Tlblmargin = 0.05;
77 Tlblvals = linspace(0.08,50,Tnticks);
78 for k = 1:Tnticks
79     Txpos = (Tlblmargin + (k-1)*(TPslid(3) - 2*Tlblmargin)/(Tnticks-1)) - TPlbl(3)/2;
80     TPlbl(1) = 0.025 + Txpos;
81     uicontrol('Parent',FP.tb1,...
82         'Units','normalized',...
83         'FontSize',6,...
84         'HorizontalAlignment','center',...
85         'Style','text',...
86         'Position',TPlbl,...
87         'String',sprintf('%.3f',Tlblvals(k)));
88 end
89
90 FP.zhedit1 = uicontrol('Parent',FP.tb1,...
91     'style','edit',...
92     'units','normalized',...
93     'position',[0.775 0.575-0.05 0.20 0.25*10/30],...
94     'string',num2str(FP.zhone,'%.3f'),...
95     'Tooltip','Write Desired High-pass Damping [ - ]',...
96     'callback',{@FPCallBack2,'zh'});
97
98 FP.fgglabel1 = uicontrol('Parent',FP.tb1,...
99     'Units','normalized',...
100    'FontSize',8,...
101    'HorizontalAlignment','center',...
102    'Style','text',...
103    'Position',[0.025 0.325-0.05 0.95 0.20],...
104    'String','Low-pass Cut Frequency [ Hz ]');
105
106 FP.fggSlider1 = uicontrol('Parent',FP.tb1,...
107     'style','slider',...
108     'units','normalized',...
109     'position',[0.025 0.35-0.05 0.725 0.25*10/30],...
110     'min',0.60,...
111     'max',FP.fggmaxone,...
112     'value',FP.fggone,...
113     'SliderStep',[0.01 0.01],...
114     'Tooltip','Select Desired Low-pass Cut Frequency [ Hz ]',...
115     'callback',{@FPCallBack1,'fgg'});
116
117 TPslid = [0.025 0.35-0.05 0.725 0.20];
118 TPlbl = [0 TPslid(2)-0.10 0.20 0.1];
119 Tnticks = 5;
120 Tlblmargin = 0.05;

```

```

121 Tlblvals = linspace(0.6,FP.fggmaxone,Tnticks);
122 for k = 1:Tnticks
123     Txpos = (Tlblmargin + (k-1)*(TPslid(3) - 2*Tlblmargin)/(Tnticks-1)) - TPlbl(3)/2;
124     TPlbl(1) = 0.025 + Txpos;
125     uicontrol('Parent',FP.tb1,...
126         'Units','normalized',...
127         'FontSize',6,...
128         'HorizontalAlignment','center',...
129         'Style','text',...
130         'Position',TPlbl,...
131         'String',sprintf('%.3f',Tlblvals(k)));
132 end
133
134 FP.fggedit1 = uicontrol('Parent',FP.tb1,...
135     'style','edit',...
136     'units','normalized',...
137     'position',[0.775 0.35-0.05 0.20 0.25*10/30],...
138     'string',num2str(FP.fggone,'%.3f'),...
139     'Tooltip','Write Desired Low-pass Cut Frequency [ Hz ]',...
140     'callback',{@FPCallBack2,'fgg'});
141
142 FP.zglabel1 = uicontrol('Parent',FP.tb1,...
143     'Units','normalized',...
144     'FontSize',8,...
145     'HorizontalAlignment','center',...
146     'Style','text',...
147     'Position',[0.025 0.10-0.05 0.95 0.20],...
148     'String','Low-pass Damping [ - ]');
149
150 FP.zgSlider1 = uicontrol('Parent',FP.tb1,...
151     'style','slider',...
152     'units','normalized',...
153     'position',[0.025 0.125-0.05 0.725 0.25*10/30],...
154     'min',0.019,...
155     'max',0.153,...
156     'value',FP.zgone,...
157     'SliderStep',[0.01 0.01],...
158     'Tooltip','Select Desired Low-pass Damping [ - ]',...
159     'callback',{@FPCallBack1,'zg'});
160
161 TPslid = [0.025 0.125-0.05 0.725 0.20];
162 TPlbl = [0 TPslid(2)-0.10 0.20 0.1];
163 Tnticks = 5;
164 Tlblmargin = 0.05;
165 Tlblvals = linspace(0.019,0.153,Tnticks);
166 for k = 1:Tnticks
167     Txpos = (Tlblmargin + (k-1)*(TPslid(3) - 2*Tlblmargin)/(Tnticks-1)) - TPlbl(3)/2;
168     TPlbl(1) = 0.025 + Txpos;
169     uicontrol('Parent',FP.tb1,...
170         'Units','normalized',...
171         'FontSize',6,...
172         'HorizontalAlignment','center',...
173         'Style','text',...
174         'Position',TPlbl,...
175         'String',sprintf('%.3f',Tlblvals(k)));
176 end
177
178 FP.zgedit1 = uicontrol('Parent',FP.tb1,...
179     'style','edit',...
180     'units','normalized',...
181     'position',[0.775 0.125-0.05 0.20 0.25*10/30],...

```

```

182         'string',num2str(FP.zgone,'%.3f'),...
183         'Tooltip','Write Desired Low-pass Damping [ - ]',...
184         'callback',{@FPCallBack2,'zg'});
185
186 FP.tb2 = uitab(FP.tg,'Title','User Values');
187
188 FP.fholabel2 = uicontrol('Parent',FP.tb2,...
189     'Units','normalized',...
190     'FontSize',8,...
191     'HorizontalAlignment','center',...
192     'Style','text',...
193     'Position',[0.025 0.775-0.05 0.95 0.20],...
194     'String','High-pass Filter Order [ - ]');
195
196 FP.fhoedit2 = uicontrol('Parent',FP.tb2,...
197     'style','edit',...
198     'units','normalized',...
199     'position',[0.40 0.80-0.05 0.20 0.25*0.10/0.30],...
200     'string',num2str(FP.fhotwo,'%.0f'),...
201     'Tooltip','Write Desired High-pass Filter Order [ - ]',...
202     'callback',{@FPCallBack3,'fho'});
203
204 FP.fhlabel2 = uicontrol('Parent',FP.tb2,...
205     'Units','normalized',...
206     'FontSize',8,...
207     'HorizontalAlignment','center',...
208     'Style','text',...
209     'Position',[0.025 0.55-0.05 0.95 0.20],...
210     'String','High-pass Cut Frequency [ Hz ]');
211
212 FP.fhedit2 = uicontrol('Parent',FP.tb2,...
213     'style','edit',...
214     'units','normalized',...
215     'position',[0.40 0.575-0.05 0.20 0.25*0.10/0.30],...
216     'string',num2str(FP.fhtwo,'%.3f'),...
217     'Tooltip','Write Desired High-pass Cut Frequency',...
218     'callback',{@FPCallBack3,'fh'});
219
220 FP.fggolabel2 = uicontrol('Parent',FP.tb2,...
221     'Units','normalized',...
222     'FontSize',8,...
223     'HorizontalAlignment','center',...
224     'Style','text',...
225     'Position',[0.025 0.325-0.05 0.95 0.20],...
226     'String','Low-pass Filter Order [ - ] ');
227
228 FP.fggoedit2 = uicontrol('Parent',FP.tb2,...
229     'style','edit',...
230     'units','normalized',...
231     'position',[0.40 0.35-0.05 0.20 0.25*0.10/0.30],...
232     'string',num2str(FP.fggotwo,'%.0f'),...
233     'Tooltip','Write Desired Low-pass Filter Order',...
234     'callback',{@FPCallBack3,'fggo'});
235
236 FP.fgglabel2 = uicontrol('Parent',FP.tb2,...
237     'Units','normalized',...
238     'FontSize',8,...
239     'HorizontalAlignment','center',...
240     'Style','text',...
241     'Position',[0.025 0.10-0.05 0.95 0.20],...
242     'String','Low-pass Cut Frequency [ Hz ]');

```

243

244

245

246

247

248

249

250

```
FP.fggedit2 = uicontrol('Parent',FP.tb2,...
                        'style','edit',...
                        'units','normalized',...
                        'position',[0.40 0.125-0.05 0.20 0.25*0.10/0.30],...
                        'string',num2str(FP.fggtwo,'%.3f'),...
                        'Tooltip','Write Desired Low-pass Cut Frequency',...
                        'callback',{@FPCallBack3,'fgg'});
```

Code 21: *FPpanel.m*: *Band-Pass Filter* panel of the *Synthetic Accelerogram* tab

M.pn2
Class: uipanel { FPpanel.m } Parent: M.tb1 { Main.m } Title: Band-pass Filter
Child: FP.tg (uitabgroup) { FPpanel.m } Callback: FPtgCallBack
<i>Description: Creates Band-Pass Filter Panel</i>

FP.tg
Class: uitabgroup { FPpanel.m } Parent: M.pn2 { FPpanel.m}
Child: FP.tb1 (uimenu) { FPpanel.m } Child: FP.tb2 (uimenu) { FPpanel.m }
<i>Description: Creates Band-Pass Filter panel tab group</i>

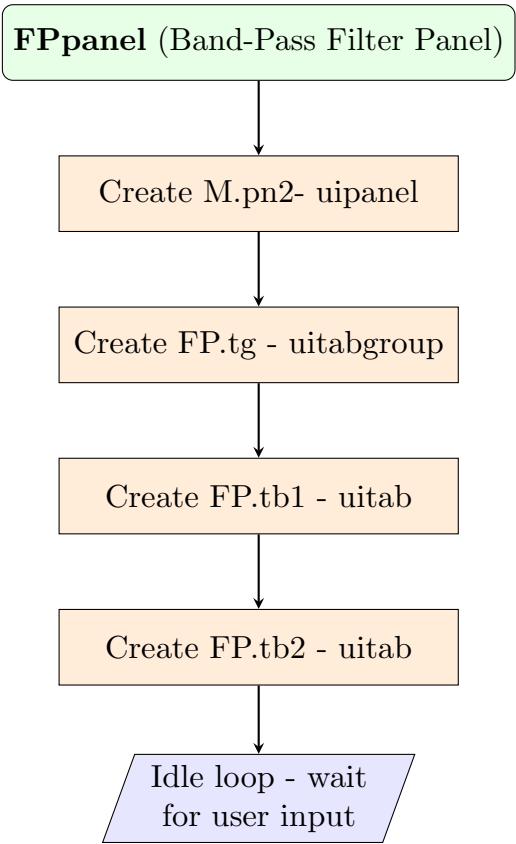
Hierarchy Diagram 6: Band-Pass Filter Panel

FP.tb1
Class: uitab { FPpanel.m } Parent: FP.tg{ FPpanel.m} Title: Observed Values
Child: FP.fhlabel1 (uicontrol{Text}) { FPpanel.m } Child: FP.fhSlider1 (uicontrol{Slider}) { FPpanel.m } Callback: FPCallBack1.fh Child: <None>(uicontrol{Text}) { FPpanel.m } Child: FP.fhedit1 (uicontrol{Edit}) { FPpanel.m } Callback: FPCallBack2.fh Child: FP.zhlabel1 (uicontrol{Text}) { FPpanel.m } Child: FP.zhSlider1 (uicontrol{Slider}) { FPpanel.m } Callback: FPCallBack1.zh Child: <None>(uicontrol{Text}) { FPpanel.m } Child: FP.zhedit1 (uicontrol){Edit} { FPpanel.m } Callback: FPCallBack2.zh Child: FP.fgglabel1 (uicontrol{Text}) { FPpanel.m } Child: FPCallBack1.fgg (uicontrol{Slider}) { FPpanel.m } Callback: FPCallBack1.fgg Child: <None>(uicontrol{Text}) { FPpanel.m } Child: FP.fggedit1 (uicontrol){Edit} { FPpanel.m } Callback: FPCallBack2.fgg Child: FP.zglabel1 (uicontrol{Text}) { FPpanel.m } Child: FP.zgSlider1 (uicontrol{Slider}) { FPpanel.m } Callback: FPCallBack1.zg Child: <None>(uicontrol{Text}) { FPpanel.m } Child: FP.zgedit1 (uicontrol){Edit} { FPpanel.m } Callback: FPCallBack2.zg
<i>Description: Creates Band-Pass Filter panel</i> <i>Observed Values tab</i>

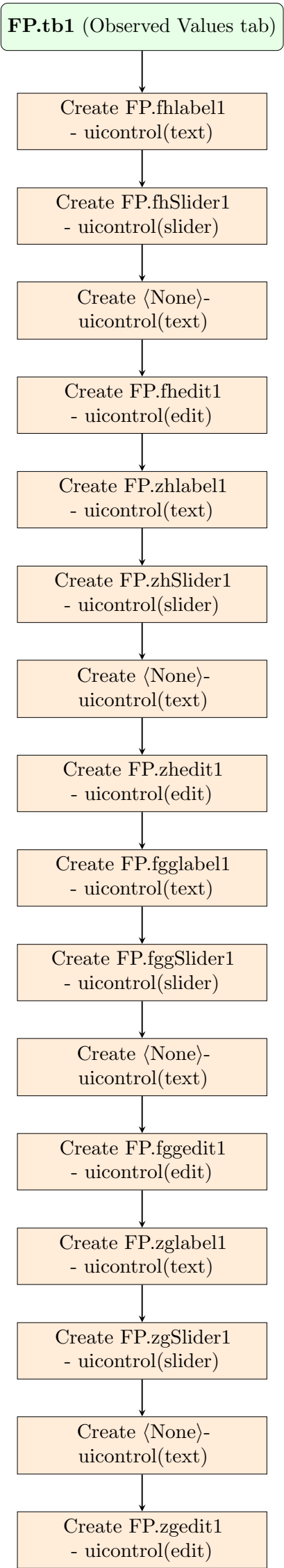
Hierarchy Diagram 7: Observed Values tab of the Band-Pass Filter Panel

FP.tb2
Class: uitab { FPpanel.m } Parent: FP.tg { FPpanel.m} Title: User Values
Child: FP.fholabel2 (uicontrol{Text}) { FPpanel.m } Child: FP.fhoedit2 (uicontrol){Edit} { FPpanel.m } Callback: FPCallBack3.fho Child: FP.fhlabel2 (uicontrol{Text}) { FPpanel.m } Child: FP.fhedit2 (uicontrol){Edit} { FPpanel.m } Callback: FPCallBack3.fh Child: FP.fggolabel2 (uicontrol{Text}) { FPpanel.m } Child: FP.fggoedit2 (uicontrol){Edit} { FPpanel.m } Callback: FPCallBack3.fggo Child: FP.fgglabel2 (uicontrol{Text}) { FPpanel.m } Child: FP.fggedit2 (uicontrol){Edit} { FPpanel.m } Callback: FPCallBack3.fgg'
<i>Description: Creates Band-Pass Filter panel</i> <i>User Values tab</i>

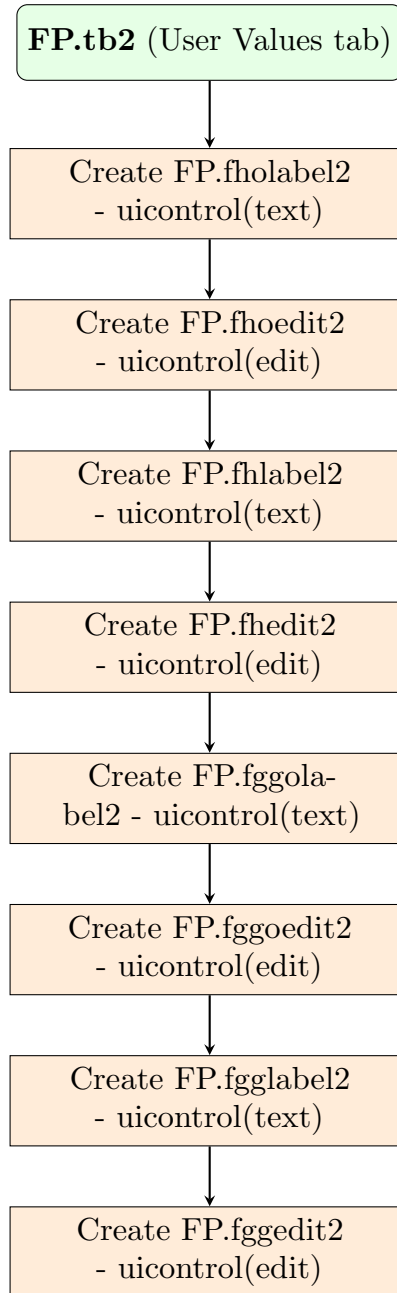
Hierarchy Diagram 8: User Values tab of the Band-Pass Filter Panel



Flowchart 14: Band-Pass Filter Panel



Flowchart 15: Observed Values tab of the Band-Pass Filter Panel



Flowchart 16: User Values tab of the Band-Pass Filter Panel

```

1 function FPtgCallBack(Source,EventData)
2
3     global M GP FP TP ZZ TN GG SP
4
5     t = EventData.NewValue.Title;
6
7     if strcmp(t,'Observed Values')
8         FP.Seltab = 1;
9         FP.whone = FP.fhone*2*pi;
10        FP.wgone = FP.fggone*2*pi;
11        FP.numeqone = conv([1 0],[2*FP.zgone*FP.wgone FP.wgone^2]);
12        FP.deneqone = conv([1 2*FP.zhone*FP.whone FP.whone^2],[1 2*FP.zgone*FP.wgone
13        ↪ FP.wgone^2]);
14        FP.Hone = tf(FP.numeqone,FP.deneqone);
15        FP.H = FP.Hone;
16        FP.GenAccone = lsim(FP.Hone,GP.RndAcc,GP.tt);
17        FP.NGenAcc = FP.GenAccone/max(abs(FP.GenAccone));
18    elseif strcmp(t,'User Values')
19        FP.Seltab = 2;
20        [FP.fhbtwo,FP.fhatwo] = butter(FP.fhotwo,FP.fhtwo/(GP.fs/2),'high');
21        [FP.fggbtwo,FP.fggatwo] = butter(FP.fggotwo,FP.fggtwo/(GP.fs/2),'low');
22        FP.btwo = conv(FP.fhbtwo,FP.fggbtwo);
23        FP.atwo = conv(FP.fhatwo,FP.fggatwo);
24        FP.Htwo = tf(FP.btwo,FP.atwo);

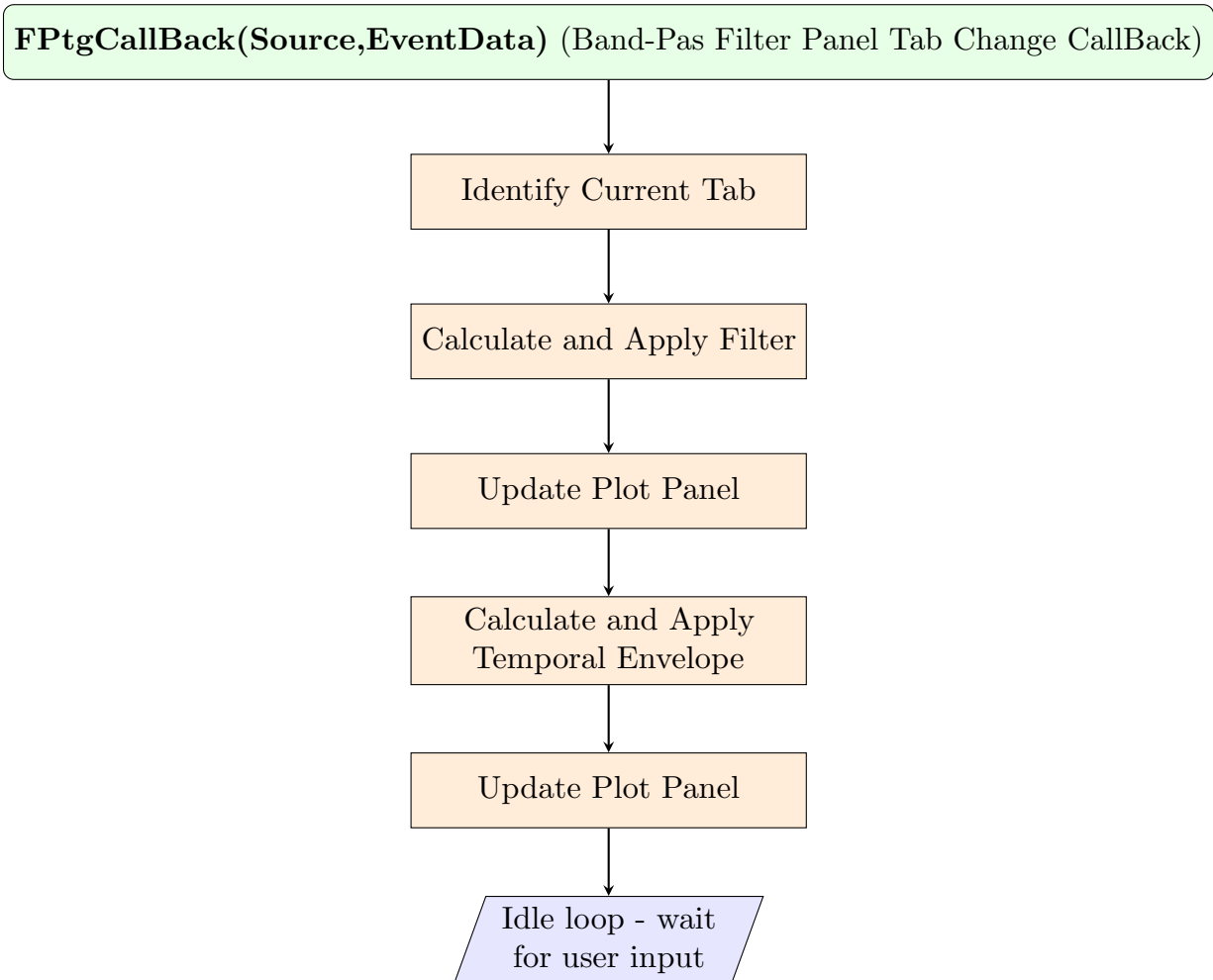
```

```

24     FP.H = FP.Htwo;
25     FP.GenAcctwo = filtfilt(FP.btwo,FP.atwo,GP.RndAcc);
26     FP.GenAcctwo = FP.GenAcctwo(:);
27     FP.NGenAcc = FP.GenAcctwo/max(abs(FP.GenAcctwo));
28     end
29     [FP.mag,FP.ph,FP.w] = bode(FP.H);
30
31     set(M.LN3,'XData',FP.w/2/pi,'YData',db(squeeze(FP.mag)));
32     set(M.LN4,'XData',FP.w/2/pi,'YData',squeeze(FP.ph));
33
34     if TP.Seltab == 1
35         TP.N01 = floor(TP.t1one*length(GP.tt)/100);
36         TP.n01 = TP.N01;
37         TP.win01 = ((1:TP.n01)/TP.n01).^TP.aone;
38         TP.win01 = TP.win01(:);
39         TP.N02 = ceil(TP.t2one*length(GP.tt)/100);
40         TP.N03 = length(GP.tt);
41         TP.n03 = TP.N03 - TP.N02 + 1;
42         TP.win03 = exp(TP.cone*(1:TP.n03)/TP.n03);
43         TP.win03 = TP.win03(:);
44         TP.n02 = length(GP.tt) - length(TP.win01) - length(TP.win03);
45         TP.win02 = ones(TP.n02,1);
46         TP.winNone = [TP.win01; TP.win02; TP.win03];
47         TP.winN = TP.winNone;
48     if TP.Seltab == 2
49         TP.N01 = floor(TP.t1two*length(GP.tt)/100);
50         TP.n01 = TP.N01;
51         TP.win01 = ((1:TP.n01)/TP.n01).^TP.atwo;
52         TP.win01 = TP.win01(:);
53         TP.N02 = ceil(TP.t2two*length(GP.tt)/100);
54         TP.N03 = length(GP.tt);
55         TP.n03 = TP.N03 - TP.N02 + 1;
56         TP.win03 = exp(TP.ctwo*(1:TP.n03)/TP.n03);
57         TP.win03 = TP.win03(:);
58         TP.n02 = length(GP.tt) - length(TP.win01) - length(TP.win03);
59         TP.win02 = ones(TP.n02,1);
60         TP.winNtwo = [TP.win01; TP.win02; TP.win03];
61         TP.winN = TP.winNtwo;
62     end
63     TP.mqS = FP.NGenAcc.*TP.winN;
64     TP.mqSN = TP.mqS/max(abs(TP.mqS));
65     [TP.psd,TP.ff] = pwelch(TP.mqSN*TP.PGA,GP.nfft,GP.noverlap,GP.window,GP.fs);
66
67     set(M.LN5,'XData',GP.tt/60,'YData',TP.mqSN*TP.PGA);
68     set(M.LN5.Parent,'YLim',[-1.1 1.1]*TP.PGA);
69     set(M.LN6,'XData',GP.tt/60,'YData',TP.winN*TP.PGA);
70     set(M.LN7,'XData',GP.tt/60,'YData',envelope(TP.mqSN*TP.PGA,floor(length(GP
    ↪ TP.mqSN)/100),'rms'));
71     set(M.LN8,'XData',TP.ff,'YData',db(TP.psd));
72
73     guidata(Source,M);
74 end

```

Code 22: *FPTgCallBack.m*: Callback for the change of tab in the *Band-pass Filter* panel



Flowchart 17: Callback for the change of tab in the *Band-pass Filter* panel

```

1 function FPcallbackfn1(Source,~,Param)
2
3     global M GP FP TP ZZ TN GG SP
4
5     M.change = 1;
6
7     FP.(Param) = get(Source, 'Value');
8
9     if strcmp(Param,'fh')
10         if FP.fh > GP.fs/2 - 0.01
11             FP.fh = GP.fs/2 - 0.01;
12         end
13         FP.fhone = FP.fh;
14         if FP.fhone > FP.fggone
15             FP.fggone = FP.fhone;
16         end
17     end
18
19     if strcmp(Param,'zh')
20         FP.zhone = FP.zh;
21     end
22
23     if strcmp(Param,'fgg')
24         FP.fggone = FP.fgg;
25         if FP.fhone > FP.fggone
26             FP.fhone = FP.fggone;
27         end
28     end
29
30     if strcmp(Param,'zg')
31         FP.zgone = FP.zg;
32     end
  
```

```

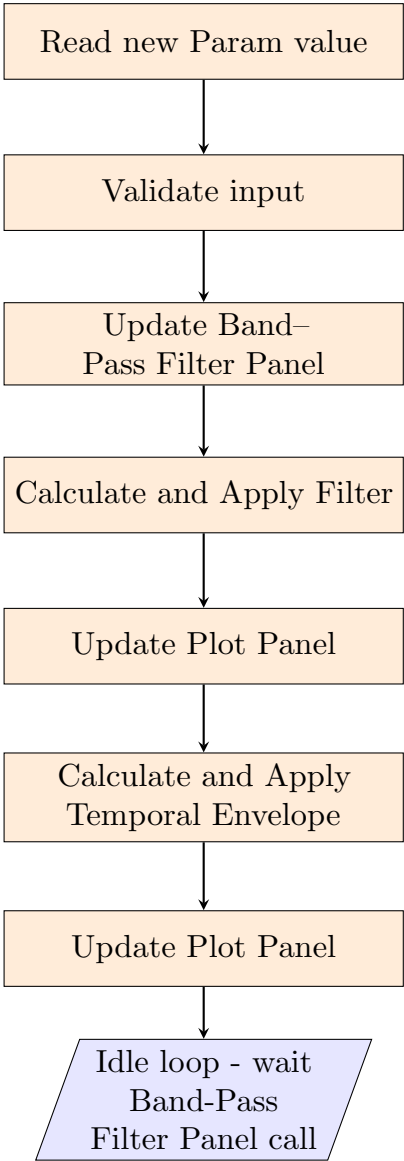
33
34 set(FP.fggedit1,'string',num2str(FP.fggone,'% .3f'));
35 set(FP.fhedit1,'string',num2str(FP.fhone,'% .3f'));
36 set(FP.zhedit1,'string',num2str(FP.zhone,'% .3f'));
37 set(FP.zgedit1,'string',num2str(FP.zgone,'% .3f'));
38 set(FP.fggSlider1,'value',FP.fggone);
39 set(FP.fhSlider1,'value',FP.fhone);
40
41 FP.whone = FP.fhone*2*pi;
42 FP.wgone = FP.fggone*2*pi;
43 FP.numeqone = conv([1 0],[2*FP.zgone*FP.wgone FP.wgone^2]);
44 FP.deneqone = conv([1 2*FP.zhone*FP.whone FP.whone^2],[1 2*FP.zgone*FP.wgone FP.wgone^2]);
45 FP.Hone = tf(FP.numeqone,FP.deneqone);
46 FP.H = FP.Hone;
47 FP.GenAccone = lsim(FP.Hone,GP.RndAcc,GP.tt);
48 FP.NGenAcc = FP.GenAccone/max(abs(FP.GenAccone));
49 [FP.mag,FP.ph,FP.w] = bode(FP.H);
50
51 set(M.LN3,'XData',FP.w/2/pi,'YData',db(squeeze(FP.mag)));
52 set(M.LN4,'XData',FP.w/2/pi,'YData',squeeze(FP.ph));
53
54 if TP.Seltab == 1
55     TP.N01 = floor(TP.tlone*length(GP.tt)/100);
56     TP.n01 = TP.N01;
57     TP.win01 = ((1:TP.n01)/TP.n01).^TP.aone;
58     TP.win01 = TP.win01(:);
59     TP.N02 = ceil(TP.t2one*length(GP.tt)/100);
60     TP.N03 = length(GP.tt);
61     TP.n03 = TP.N03 - TP.N02 + 1;
62     TP.win03 = exp(TP.cone*(1:TP.n03)/TP.n03);
63     TP.win03 = TP.win03(:);
64     TP.n02 = length(GP.tt) - length(TP.win01) - length(TP.win03);
65     TP.win02 = ones(TP.n02,1);
66     TP.winNone = [TP.win01; TP.win02; TP.win03];
67     TP.winN = TP.winNone;
68 if TP.Seltab == 2
69     TP.N01 = floor(TP.tltwo*length(GP.tt)/100);
70     TP.n01 = TP.N01;
71     TP.win01 = ((1:TP.n01)/TP.n01).^TP.atwo;
72     TP.win01 = TP.win01(:);
73     TP.N02 = ceil(TP.t2two*length(GP.tt)/100);
74     TP.N03 = length(GP.tt);
75     TP.n03 = TP.N03 - TP.N02 + 1;
76     TP.win03 = exp(TP.ctwo*(1:TP.n03)/TP.n03);
77     TP.win03 = TP.win03(:);
78     TP.n02 = length(GP.tt) - length(TP.win01) - length(TP.win03);
79     TP.win02 = ones(TP.n02,1);
80     TP.winNtwo = [TP.win01; TP.win02; TP.win03];
81     TP.winN = TP.winNtwo;
82 end
83 TP.mqS = FP.NGenAcc.*TP.winN;
84 TP.mqSN = TP.mqS/max(abs(TP.mqS));
85 [TP.psd,TP.ff] = pwelch(TP.mqSN*TP.PGA,GP.nfft,GP.noverlap,GP.window,GP.fs);
86
87 set(M.LN5,'XData',GP.tt/60,'YData',TP.mqSN*TP.PGA);
88 set(M.LN5.Parent,'YLim',[-1.1 1.1]*TP.PGA);
89 set(M.LN6,'XData',GP.tt/60,'YData',TP.winN*TP.PGA);
90 set(M.LN7,'XData',GP.tt/60,'YData',envelope(TP.mqSN*TP.PGA,floor(length(
↵ TP.mqSN)/100),'rms'));
91 set(M.LN8,'XData',TP.ff,'YData',db(TP.psd));
92

```

```
93     guidata(Source,M);
94
95 end
```

Code 23: *FPCallback1.m*: Callback for the slider controls of the *Observed Values* tab in the *Band-Pass Filter* panel

FPcallbackfn1(Source,Param) (Sliders CallBack)



Flowchart 18: Callback for the slider controls of the *Observed Values* tab in the *Band-Pass Filter* panel

```
1 function FPCallbackfn2(Source,~,Param)
2
3     global M GP FP TP ZZ TN GG SP
4
5     if isnan(str2double(get(Source,'String')))
6         if strcmp(Param,'fh')
7             set(Source,'string',num2str(FP.fhone,'%.3f'));
8         elseif strcmp(Param,'zh')
9             set(Source,'string',num2str(FP.zhone,'%.3f'));
10        elseif strcmp(Param,'fgg')
11            set(Source,'string',num2str(FP.fggone,'%.3f'));
12        elseif strcmp(Param,'zg')
13            set(Source,'string',num2str(FP.zgone,'%.3f'));
14        end
15    else
```

```

16 M.change = 1;
17
18 FP.(Param) = str2double(get(Source,'String'));
19
20 if strcmp(Param,'fh')
21     if FP.fh > GP.fs/2 - 0.01
22         FP.fh = GP.fs/2 - 0.01;
23     elseif FP.fh > 0.7
24         FP.fh = 0.7;
25     elseif FP.fh < 0.2
26         FP.fh = 0.2;
27     end
28     FP.fhone = FP.fh;
29     if FP.fhone > FP.fggone
30         FP.fggone = FP.fhone;
31     end
32 end
33
34 if strcmp(Param,'zh')
35     if FP.zh > 50
36         FP.zh = 50;
37     elseif FP.zh < 0.08
38         FP.zh = 0.08;
39     end
40     FP.zhone = FP.zh;
41 end
42
43 if strcmp(Param,'fgg')
44     if FP.fgg > FP.fggmaxone
45         FP.fgg = FP.fggmaxone;
46     elseif FP.fgg < 0.60
47         FP.fgg = 0.60;
48     end
49     FP.fggone = FP.fgg;
50     if FP.fhone > FP.fggone
51         FP.fhone = FP.fggone;
52     end
53 end
54
55 if strcmp(Param,'zg')
56     if FP.zg > 0.153
57         FP.zg = 0.153;
58     elseif FP.zg < 0.019
59         FP.zg = 0.019;
60     end
61     FP.zgone = FP.zg;
62 end
63
64 set(FP.zhSlider1,'value',FP.zhone);
65 set(FP.zhedit1,'string',num2str(FP.zhone,'%0.3f'));
66 set(FP.fggSlider1,'value',FP.fggone);
67 set(FP.fggedit1,'string',num2str(FP.fggone,'%0.3f'));
68 set(FP.fhSlider1,'value',FP.fhone);
69 set(FP.fhedit1,'string',num2str(FP.fhone,'%0.3f'));
70 set(FP.zgSlider1,'value',FP.zgone);
71 set(FP.zgedit1,'string',num2str(FP.zgone,'%0.3f'));
72
73 FP.whone = FP.fhone*2*pi;
74 FP.wgone = FP.fggone*2*pi;
75 FP.numeqone = conv([1 0],[2*FP.zgone*FP.wgone FP.wgone^2]);
76 FP.deneqone = conv([1 2*FP.zhone*FP.whone FP.whone^2],[1 2*FP.zgone*FP.wgone
    ↪ FP.wgone^2]);

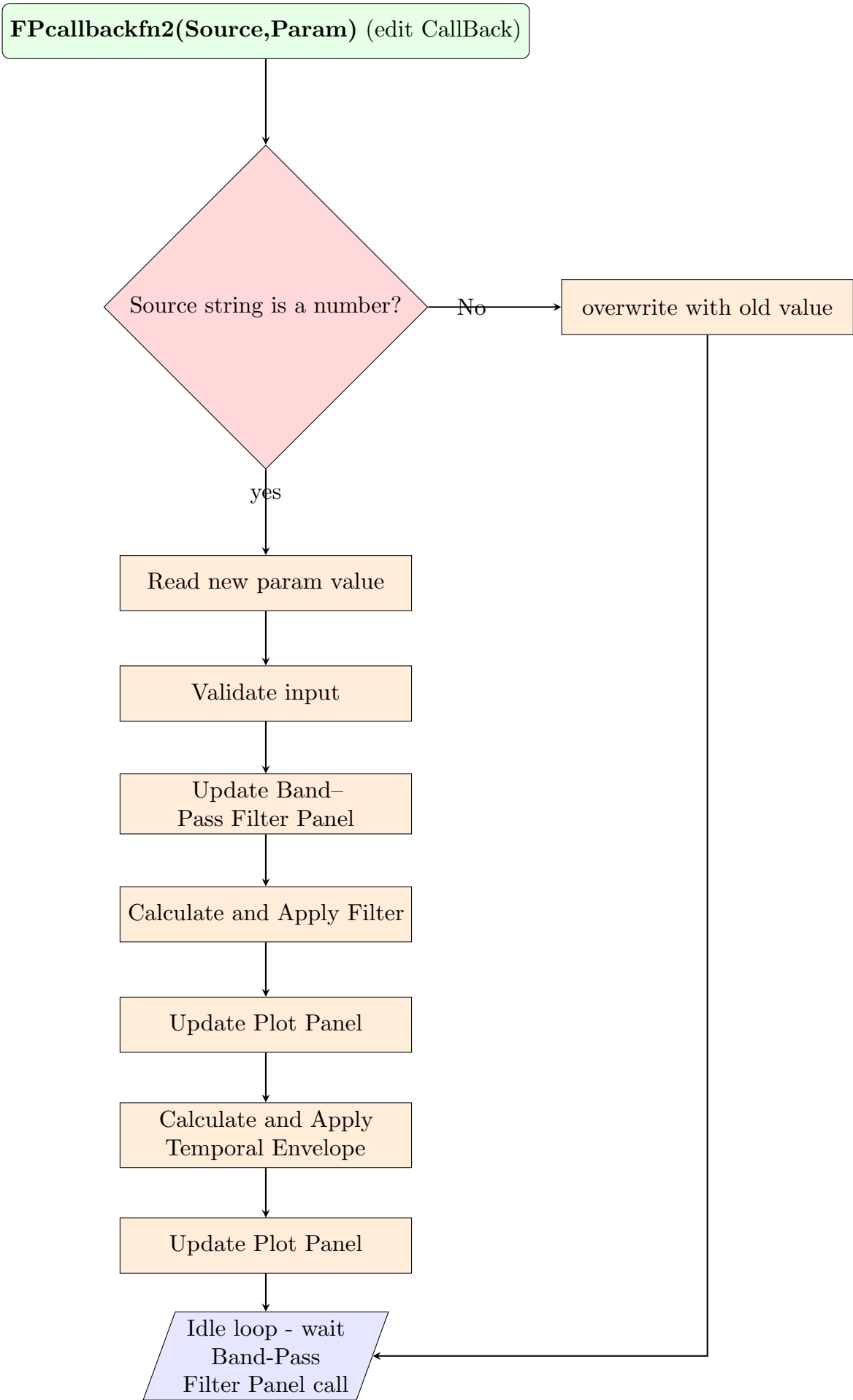
```

```

77     FP.Hone = tf(FP.numeqone,FP.deneqone);
78     FP.H = FP.Hone;
79     FP.GenAcccone = lsim(FP.Hone,GP.RndAcc,GP.tt);
80     FP.NGenAcc = FP.GenAcccone/max(abs(FP.GenAcccone));
81     [FP.mag,FP.ph,FP.w] = bode(FP.H);
82     set(M.LN3,'XData',FP.w/2/pi,'YData',db(squeeze(FP.mag)));
83     set(M.LN4,'XData',FP.w/2/pi,'YData',squeeze(FP.ph));
84
85     if TP.Selstab == 1
86         TP.N01 = floor(TP.t1one*length(GP.tt)/100);
87         TP.n01 = TP.N01;
88         TP.win01 = ((1:TP.n01)/TP.n01).^TP.aone;
89         TP.win01 = TP.win01(:);
90         TP.N02 = ceil(TP.t2one*length(GP.tt)/100);
91         TP.N03 = length(GP.tt);
92         TP.n03 = TP.N03 - TP.N02 + 1;
93         TP.win03 = exp(TP.cone*(1:TP.n03)/TP.n03);
94         TP.win03 = TP.win03(:);
95         TP.n02 = length(GP.tt) - length(TP.win01) - length(TP.win03);
96         TP.win02 = ones(TP.n02,1);
97         TP.winNone = [TP.win01; TP.win02; TP.win03];
98         TP.winN = TP.winNone;
99     if TP.Selstab == 2
100         TP.N01 = floor(TP.t1two*length(GP.tt)/100);
101         TP.n01 = TP.N01;
102         TP.win01 = ((1:TP.n01)/TP.n01).^TP.atwo;
103         TP.win01 = TP.win01(:);
104         TP.N02 = ceil(TP.t2two*length(GP.tt)/100);
105         TP.N03 = length(GP.tt);
106         TP.n03 = TP.N03 - TP.N02 + 1;
107         TP.win03 = exp(TP.ctwo*(1:TP.n03)/TP.n03);
108         TP.win03 = TP.win03(:);
109         TP.n02 = length(GP.tt) - length(TP.win01) - length(TP.win03);
110         TP.win02 = ones(TP.n02,1);
111         TP.winNtwo = [TP.win01; TP.win02; TP.win03];
112         TP.winN = TP.winNtwo;
113     end
114     TP.mqS = FP.NGenAcc.*TP.winN;
115     TP.mqSN = TP.mqS/max(abs(TP.mqS));
116     [TP.psd,TP.ff] = pwelch(TP.mqSN*TP.PGA,GP.nfft,GP.noverlap,GP.window,GP.fs);
117
118     set(M.LN5,'XData',GP.tt/60,'YData',TP.mqSN*TP.PGA);
119     set(M.LN5.Parent,'YLim',[-1.1 1.1]*TP.PGA);
120     set(M.LN6,'XData',GP.tt/60,'YData',TP.winN*TP.PGA);
121
122     ↪ set(M.LN7,'XData',GP.tt/60,'YData',envelope(TP.mqSN*TP.PGA,floor(length(TP.mqSN)/100),'rms'));
123     set(M.LN8,'XData',TP.ff,'YData',db(TP.psd));
124
125     guidata(Source,M);
126
127 end
end

```

Code 24: *FPCallback2.m*: Callback for the editable text box controls of the *Observed Values* tab in the *Band-Pass Filter* panel



Flowchart 19: Callback for the editable text box controls of the *Observed Values* tab in the *Band-Pass Filter* panel

```

1 function FPCallbackfn3(Source,~,Param)
2
3     global M GP FP TP MZ UV TN GG SP
4
5     if isnan(str2double(get(Source, 'String')))
6         if strcmp(Param,'fho')
7             set(Source,'string',num2str(FP.fhotwo,'%.0f'));
  
```

```

8     elseif strcmp(Param,'fh')
9         set(Source,'string',num2str(FP.fhtwo,'%3f'));
10    elseif strcmp(Param,'fggo')
11        set(Source,'string',num2str(FP.fggotwo,'%0f'));
12    elseif strcmp(Param,'fgg')
13        set(Source,'string',num2str(FP.fggtwo,'%3f'));
14    end
15 else
16     M.change = 1;
17
18     FP.(Param) = str2double(get(Source,'String'));
19
20     if strcmp(Param,'fh')
21         if FP.fh > GP.fs/2 - 0.01
22             FP.fh = GP.fs/2 - 0.01;
23         elseif FP.fh < 0.1
24             FP.fh = 0.1;
25         end
26         FP.fhtwo = FP.fh;
27     end
28
29     if strcmp(Param,'fho')
30         if FP.fho < 1
31             FP.fho = 1;
32         elseif FP.fho > 4
33             FP.fho = 4;
34         else
35             FP.fho = ceil(FP.fho);
36         end
37         FP.fhotwo = FP.fho;
38     end
39
40     if strcmp(Param,'fgg')
41         if FP.fgg > GP.fs/2 - 0.01
42             FP.fgg = GP.fs/2 - 0.01;
43         elseif FP.fgg < 0.01
44             FP.fgg = 0.01;
45         end
46         FP.fggtwo = FP.fgg;
47     end
48
49     if strcmp(Param,'fggo')
50         if FP.fggo < 1
51             FP.fggo = 1;
52         elseif FP.fggo > 4
53             FP.fggo = 4;
54         else
55             FP.fggo = ceil(FP.fggo);
56         end
57         FP.fggotwo = FP.fggo;
58     end
59
60     set(FP.fhoedit2,'string',num2str(FP.fhotwo,'%0f'));
61     set(FP.fggoedit2,'string',num2str(FP.fggotwo,'%0f'));
62     set(FP.fggedit2,'string',num2str(FP.fggtwo,'%3f'));
63     set(FP.fhedit2,'string',num2str(FP.fhtwo,'%3f'));
64
65     [FP.fhbtwo,FP.fhatwo] = butter(FP.fhotwo,FP.fhtwo/(GP.fs/2),'high');
66     [FP.fggbtwo,FP.fggatwo] = butter(FP.fggotwo,FP.fggtwo/(GP.fs/2),'low');
67     FP.btwo = conv(FP.fhbtwo,FP.fggbtwo);
68     FP.atwo = conv(FP.fhatwo,FP.fggatwo);

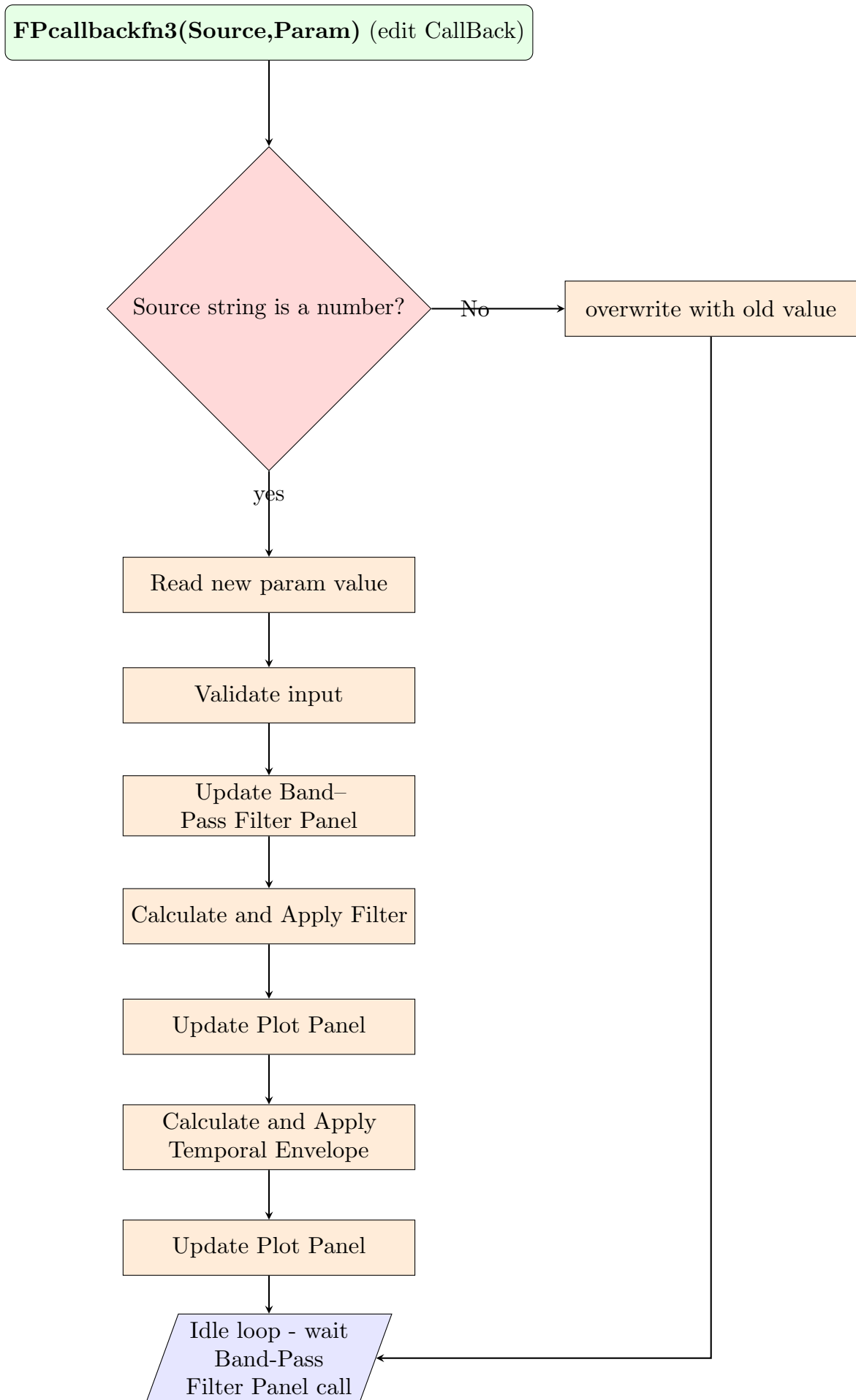
```

```

69     FP.Htwo = tf(FP.btwo,FP.atwo);
70     FP.H = FP.Htwo;
71     FP.GenAcctwo = filtfilt(FP.btwo,FP.atwo,GP.RndAcc);
72     FP.GenAcctwo = FP.GenAcctwo(:);
73     FP.NGenAcc = FP.GenAcctwo/max(abs(FP.GenAcctwo));
74     [FP.mag,FP.ph,FP.w] = bode(FP.H);
75
76     set(M.LN3,'XData',FP.w/2/pi,'YData',db(squeeze(FP.mag)));
77     set(M.LN4,'XData',FP.w/2/pi,'YData',squeeze(FP.ph));
78
79     if TP.Seltab == 1
80         TP.N01 = floor(TP.t1one*length(GP.tt)/100);
81         TP.n01 = TP.N01;
82         TP.win01 = ((1:TP.n01)/TP.n01).^TP.aone;
83         TP.win01 = TP.win01(:);
84         TP.N02 = ceil(TP.t2one*length(GP.tt)/100);
85         TP.N03 = length(GP.tt);
86         TP.n03 = TP.N03 - TP.N02 + 1;
87         TP.win03 = exp(TP.cone*(1:TP.n03)/TP.n03);
88         TP.win03 = TP.win03(:);
89         TP.n02 = length(GP.tt) - length(TP.win01) - length(TP.win03);
90         TP.win02 = ones(TP.n02,1);
91         TP.winNone = [TP.win01; TP.win02; TP.win03];
92         TP.winN = TP.winNone;
93     if TP.Seltab == 2
94         TP.N01 = floor(TP.t1two*length(GP.tt)/100);
95         TP.n01 = TP.N01;
96         TP.win01 = ((1:TP.n01)/TP.n01).^TP.atwo;
97         TP.win01 = TP.win01(:);
98         TP.N02 = ceil((100 - TP.t2two)*length(GP.tt)/100);
99         TP.N03 = length(GP.tt);
100        TP.n03 = TP.N03 - TP.N02 + 1;
101        TP.win03 = exp(TP.ctwo*(1:TP.n03)/TP.n03);
102        TP.win03 = TP.win03(:);
103        TP.n02 = length(GP.tt) - length(TP.win01) - length(TP.win03);
104        TP.win02 = ones(TP.n02,1);
105        TP.winNtwo = [TP.win01; TP.win02; TP.win03];
106        TP.winN = TP.winNtwo;
107    end
108    TP.mqS = FP.NGenAcc.*TP.winN;
109    TP.mqSN = TP.mqS/max(abs(TP.mqS));
110    [TP.psd,TP.ff] = pwelch(TP.mqSN*TP.PGA,GP.nfft,GP.noverlap,GP.window,GP.fs);
111
112    set(M.LN5,'XData',GP.tt/60,'YData',TP.mqSN*TP.PGA);
113    set(M.LN5.Parent,'YLim',[-1.1 1.1]*TP.PGA);
114    set(M.LN6,'XData',GP.tt/60,'YData',TP.winN*TP.PGA);
115    set(M.LN7,'XData',GP.tt/60,'YData',envelope(TP.mqSN*TP.PGA,floor(length(GP
    ↪ TP.mqSN)/100),'rms'));
116    set(M.LN8,'XData',TP.ff,'YData',db(TP.psd));
117
118    guidata(Source,M);
119
120    end
121 end

```

Code 25: *FPCallback3.m*: Callback for the editable text box controls of the *User Values* tab in the *Band-Pass Filter* panel



Flowchart 20: Callback for the editable text box controls of the *User Values* tab in the *Band-Pass Filter* panel

5.2.5 Temporal Envelope Panel

```

1 M.pn3 = uipanel('Parent',M.tb1,...
2             'Units','normalized',...
3             'Position',[0.025 0.010 0.25 0.5595], ...
4             'Title','Temporal Envelope');
5

```

```

6 TP.pn1 = uipanel('Parent',M.pn3,...
7             'Units','normalized',...
8             'Position',[0.025 0.75 0.95 0.255]);
9
10 axes('Parent',TP.pn1,...
11     'Units','normalized',...
12     'Position',[0 0 1 1]);
13
14 imagepath = ['Images',symb,'temporalWindow.jpg'];
15 image = imread(imagepath);
16 [h, w, ~] = size(image);
17 imgzoompan(gcf, 'ImgWidth', w, 'ImgHeight', h);
18 imshow(image);
19
20 TP.pn2 = uipanel('Parent',M.pn3,...
21             'Units','normalized',...
22             'Position',[0.025 0.025 0.95 0.70]);
23
24 TP.tg = uitabgroup(TP.pn2,'SelectionChangedFcn',@TPtgCallBack);
25
26 TP.tb1 = uitab(TP.tg,'Title','Obseved Values');
27
28 TP.buttonGroup = uibuttongroup('Parent',TP.tb1,...
29                             'Units','normalized',...
30                             'FontSize',8,...
31                             'Position',[0.025 0.825 0.95 0.17],...
32                             'Title', 'Select Desired Moment Magnitude Range');
33
34 TP.radioButton1 = uicontrol('Parent',TP.buttonGroup,...
35                             'units','normalized',...
36                             'Style','radiobutton',...
37                             'FontSize',8,...
38                             'Position',[0.05, 1/3, 0.90/3, 0.5],...
39                             'String', '2.7 to 3.2',...
40                             'Callback',@RadioButtonCallBack);
41
42 TP.radioButton2 = uicontrol('Parent',TP.buttonGroup,...
43                             'units','normalized',...
44                             'Style','radiobutton',...
45                             'FontSize',8,...
46                             'Position',[0.05+0.90/3, 1/3, 0.90/3, 0.5],...
47                             'String','3.3 to 3.6',...
48                             'Callback',@RadioButtonCallBack);
49
50 TP.radioButton3 = uicontrol('Parent',TP.buttonGroup,...
51                             'units','normalized',...
52                             'Style','radiobutton',...
53                             'FontSize',8,...
54                             'Position',[0.05+0.90/3*2, 1/3, 0.90/3, 0.5],...
55                             'String','3.7 to 4.1',...
56                             'Callback',@RadioButtonCallBack);
57
58 TP.t1label = uicontrol('Parent',TP.tb1,...
59                     'Units','normalized',...
60                     'FontSize',8,...
61                     'HorizontalAlignment','center',...
62                     'Style','text',...
63                     'Position',[0.025 0.70 0.95 0.1],...
64                     'String','Percentage Rise Time t1 [ % ]');
65
66 TP.t1Slider = uicontrol('Parent',TP.tb1,...

```

```

67         'style','slider',...
68         'units','normalized',...
69         'position',[0.025 0.70-0.05 0.725 0.25*0.10/0.5445/.7],...
70         'min',TP.t1min,...
71         'max',TP.t1max,...
72         'value',TP.t1,...
73         'SliderStep',[0.01 0.01],...
74         'Tooltip','Select Desired Percentage Rise Time t1 [ % ]',...
75         'callback',{@TPCallBack1,'t1'});
76
77 TPslid = [0.025 0.70-0.05 0.725 0.045];
78 TPlbl = [0 TPslid(2)-0.03 0.15 0.03];
79 Tnticks = 5;
80 Tlblmargin = 0.05;
81 Tlblvals = linspace(TP.t1min,TP.t1max,Tnticks);
82 for k = 1:Tnticks
83     Txpos = (Tlblmargin + (k-1)*(TPslid(3) - 2*Tlblmargin)/(Tnticks-1)) - TPlbl(3)/2;
84     TPlbl(1) = 0.025 + Txpos;
85     uicontrol('Parent',TP.tb1,...
86             'Units','normalized',...
87             'FontSize',6,...
88             'HorizontalAlignment','center',...
89             'Style','text',...
90             'Position',TPlbl,...
91             'String',sprintf('%.3f',Tlblvals(k)),...
92             'Tag','t1Slider1ticklabel');
93 end
94
95 TP.t1edit = uicontrol('Parent',TP.tb1,...
96         'style','edit',...
97         'units','normalized',...
98         'position',[0.775 0.70-0.05 0.20 0.25*0.10/0.5445/.7],...
99         'String',num2str(TP.t1,'%.3f'),...
100        'SliderStep',[0.01 0.01],...
101        'Tooltip','Write Desired Percentage Rise Time t1 [ % ]',...
102        'callback',{@TPCallBack2,'t1'});
103
104 TP.alabel = uicontrol('Parent',TP.tb1,...
105         'Units','normalized',...
106         'FontSize',8,...
107         'HorizontalAlignment','center',...
108         'Style','text',...
109         'Position',[0.025 0.50 0.95 0.1],...
110         'String','Rise Exponent a [ - ]');
111
112 TP.aSlider = uicontrol('Parent',TP.tb1,...
113         'style','slider',...
114         'units','normalized',...
115         'position',[0.025 0.50-0.05 0.725 0.25*0.10/0.5445/.7],...
116         'min',TP.amin,...
117         'max',TP.amax,...
118         'value',TP.a,...
119         'SliderStep',[0.01 0.01],...
120         'Tooltip','Select Desired Rise Exponent a [ - ]',...
121         'callback',{@TPCallBack1,'a'});
122
123 TPslid = [0.025 0.50-0.05 0.725 0.045];
124 TPlbl = [0 TPslid(2)-0.03 0.15 0.03];
125 Tnticks = 5;
126 Tlblmargin = 0.05;
127 Tlblvals = linspace(TP.amin,TP.amax,Tnticks);

```

```

128 for k = 1:Tnticks
129     Txpos = (Tlblmargin + (k-1)*(TPslid(3) - 2*Tlblmargin)/(Tnticks-1)) - TPlbl(3)/2;
130     TPlbl(1) = 0.025 + Txpos;
131     uicontrol('Parent',TP.tb1,...
132             'Units','normalized',...
133             'FontSize',6,...
134             'HorizontalAlignment','center',...
135             'Style','text',...
136             'Position',TPlbl,...
137             'String',sprintf('%.3f',Tlblvals(k)),...
138             'Tag','aslider1ticklabel');
139 end
140
141 TP.aedit = uicontrol('Parent',TP.tb1,...
142                     'style','edit',...
143                     'units','normalized',...
144                     'position',[0.775 0.50-0.05 0.20 0.25*0.10/0.5445/.7],...
145                     'String',num2str(TP.a,'%.3f'),...
146                     'SliderStep',[0.01 0.01],...
147                     'Tooltip','Write Desired Rise Exponent a [ - ]',...
148                     'callback',{@TPCallBack2,'a'});
149
150 TP.t2label = uicontrol('Parent',TP.tb1,...
151                       'Units','normalized',...
152                       'FontSize',8,...
153                       'HorizontalAlignment','center',...
154                       'Style','text',...
155                       'Position',[0.025 0.30 0.95 0.1],...
156                       'String','Percentage Fall Time t2 [ % ]');
157
158 TP.t2Slider = uicontrol('Parent',TP.tb1,...
159                        'style','slider',...
160                        'units','normalized',...
161                        'position',[0.025 0.30-0.05 0.725 0.25*0.10/0.5445/.7],...
162                        'min',TP.t2min,...
163                        'max',TP.t2max,...
164                        'value',TP.t2,...
165                        'SliderStep',[0.01 0.01],...
166                        'Tooltip','Select Desired Percentage Fall Time t2 [ % ]',...
167                        'callback',{@TPCallBack1,'t2'});
168
169 TPslid = [0.025 0.30-0.05 0.725 0.045];
170 TPlbl = [0 TPslid(2)-0.03 0.15 0.03];
171 Tnticks = 5;
172 Tlblmargin = 0.05;
173 Tlblvals = linspace(TP.t2min,TP.t2max,Tnticks);
174 for k = 1:Tnticks
175     Txpos = (Tlblmargin + (k-1)*(TPslid(3) - 2*Tlblmargin)/(Tnticks-1)) - TPlbl(3)/2;
176     TPlbl(1) = 0.025 + Txpos;
177     uicontrol('Parent',TP.tb1,...
178             'Units','normalized',...
179             'FontSize',6,...
180             'HorizontalAlignment','center',...
181             'Style','text',...
182             'Position',TPlbl,...
183             'String',sprintf('%.3f',Tlblvals(k)),...
184             'Tag','t2slider1ticklabel');
185 end
186
187 TP.t2edit = uicontrol('Parent',TP.tb1,...
188                     'style','edit',...

```

```

189         'units','normalized',...
190         'position',[0.775 0.30-0.05 0.20 0.25*0.10/0.5445/.7],...
191         'string',num2str(TP.t2,'%3f'),...
192         'SliderStep',[0.01 0.01],...
193         'Tooltip','Write Desired Percentage Fall Time t2 [ % ]',...
194         'callback',{@TPCallBack2,'t2'});
195
196 TP.clabel = uicontrol('Parent',TP.tb1,...
197     'Units','normalized',...
198     'FontSize',8,...
199     'HorizontalAlignment','center',...
200     'Style','text',...
201     'Position',[0.025 0.10 0.95 0.1],...
202     'String','Fall Exponent c [ - ]');
203
204 TP.cSlider = uicontrol('Parent',TP.tb1,...
205     'style','slider',...
206     'units','normalized',...
207     'position',[0.025 0.10-0.05 0.725 0.25*0.10/0.5445/.7],...
208     'min',TP.cmin,...
209     'max',TP.cmax,...
210     'value',TP.c,...
211     'SliderStep',[0.01 0.01],...
212     'Tooltip','Select Desired Fall Exponent c [ - ]',...
213     'callback',{@TPCallBack1,'c'});
214
215 TPslid = [0.025 0.10-0.05 0.725 0.045];
216 TPlbl = [0 TPslid(2)-0.03 0.15 0.03];
217 Tnticks = 5;
218 Tlblmargin = 0.05;
219 Tlblvals = linspace(TP.cmin,TP.cmax,Tnticks);
220 for k = 1:Tnticks
221     Txpos = (Tlblmargin + (k-1)*(TPslid(3) - 2*Tlblmargin)/(Tnticks-1)) - TPlbl(3)/2;
222     TPlbl(1) = 0.025 + Txpos;
223     uicontrol('Parent',TP.tb1,...
224         'Units','normalized',...
225         'FontSize',6,...
226         'HorizontalAlignment','center',...
227         'Style','text',...
228         'Position',TPlbl,...
229         'String',sprintf('%3f',Tlblvals(k)),...
230         'Tag','cslider1ticklabel');
231 end
232
233 TP.cedit = uicontrol('Parent',TP.tb1,...
234     'style','edit',...
235     'units','normalized',...
236     'position',[0.775 0.10-0.05 0.20 0.25*0.10/0.5445/.7],...
237     'string',num2str(TP.c,'%3f'),...
238     'SliderStep',[0.01 0.01],...
239     'Tooltip','Write Desired Fall Exponent c [ - ]',...
240     'callback',{@TPCallBack2,'c'});
241
242 TP.tb2 = uitab(TP.tg,'Title','User Values');
243
244 TP.t1label2 = uicontrol('Parent',TP.tb2,...
245     'Units','normalized',...
246     'FontSize',8,...
247     'HorizontalAlignment','center',...
248     'Style','text',...
249     'Position',[0.025 0.80 0.95 0.1],...

```

```

250         'String','Percentage Rise Time t1 [ % ]');
251
252 TP.t1edit2 = uicontrol('Parent',TP.tb2,...
253     'style','edit',...
254     'units','normalized',...
255     'position',[0.4 0.80-0.05 0.20 0.25*0.10/0.5445/.7],...
256     'String',num2str(TP.t1,'%.3f'),...
257     'SliderStep',[0.01 0.01],...
258     'Tooltip','Write Desired Percentage Rise Time t1 [ % ]',...
259     'callback',{@TPCallBack3,'t1'});
260
261 TP.alabel2 = uicontrol('Parent',TP.tb2,...
262     'Units','normalized',...
263     'FontSize',8,...
264     'HorizontalAlignment','center',...
265     'Style','text',...
266     'Position',[0.025 0.60 0.95 0.1],...
267     'String','Rise Exponent a [ - ]');
268
269 TP.aedit2 = uicontrol('Parent',TP.tb2,...
270     'style','edit',...
271     'units','normalized',...
272     'position',[0.4 0.60-0.05 0.20 0.25*0.10/0.5445/.7],...
273     'String',num2str(TP.a,'%.3f'),...
274     'SliderStep',[0.01 0.01],...
275     'Tooltip','Write Desired Rise Exponent a [ - ]',...
276     'callback',{@TPCallBack3,'a'});
277
278 TP.t2label2 = uicontrol('Parent',TP.tb2,...
279     'Units','normalized',...
280     'FontSize',8,...
281     'HorizontalAlignment','center',...
282     'Style','text',...
283     'Position',[0.025 0.40 0.95 0.1],...
284     'String','Percentage Fall Time t2 [ % ]');
285
286 TP.t2edit2 = uicontrol('Parent',TP.tb2,...
287     'style','edit',...
288     'units','normalized',...
289     'position',[0.4 0.40-0.05 0.20 0.25*0.10/0.5445/.7],...
290     'string',num2str(TP.t2,'%.3f'),...
291     'SliderStep',[0.01 0.01],...
292     'Tooltip','Write Desired Percentage Fall Time t2 [ % ]',...
293     'callback',{@TPCallBack3,'t2'});
294
295 TP.clabel2 = uicontrol('Parent',TP.tb2,...
296     'Units','normalized',...
297     'FontSize',8,...
298     'HorizontalAlignment','center',...
299     'Style','text',...
300     'Position',[0.025 0.20 0.95 0.1],...
301     'String','Fall Exponent c [ - ]');
302
303 TP.cedit2 = uicontrol('Parent',TP.tb2,...
304     'style','edit',...
305     'units','normalized',...
306     'position',[0.4 0.20-0.05 0.20 0.25*0.10/0.5445/.7],...
307     'string',num2str(TP.c,'%.3f'),...
308     'SliderStep',[0.01 0.01],...
309     'Tooltip','Write Desired Fall Exponent c [ - ]',...
310     'callback',{@TPCallBack3,'c'});

```

Code 26: *TPpanel.m*: Temporal Envelope panel of the *Synthetic Accelerogram* tab

M.pn3
Class: uipanel { TPpanel.m } Parent: M.tb1 { Main.m } Title: Temporal Envelope
Child: TP.pn1 (uitabgroup) { FPpanel.m } Child: TP.pn1 (uitabgroup) { FPpanel.m }
<i>Description: Creates Temporal Envelope Panel</i>

TP.pn1
Class: uipanel { TPpanel.m } Parent: M.pn3 { TPpanel.m }
Child: <None>(axes) { TPpanel.m } Callback: imgzoompan
<i>Description: Creates Figure Panel and Axes</i>

TP.pn2
Class: uipanel { TPpanel.m } Parent: M.pn3 { TPpanel.m }
Child: TP.tg (uitabgroup) { TPpanel.m } Callback: TPtgCallBack
<i>Description: Creates Controls Tab Group</i>

TP.tg
Class: uitabgroup { TPpanel.m } Parent: TP.pn2 { TPpanel.m }
Child: TP.tb1 (uitab) { TPpanel.m } Child: TP.tb2 (uitab) { TPpanel.m }
<i>Description: Creates Figure Panel and Axes</i>

Hierarchy Diagram 9: Temporal Envelope Panel

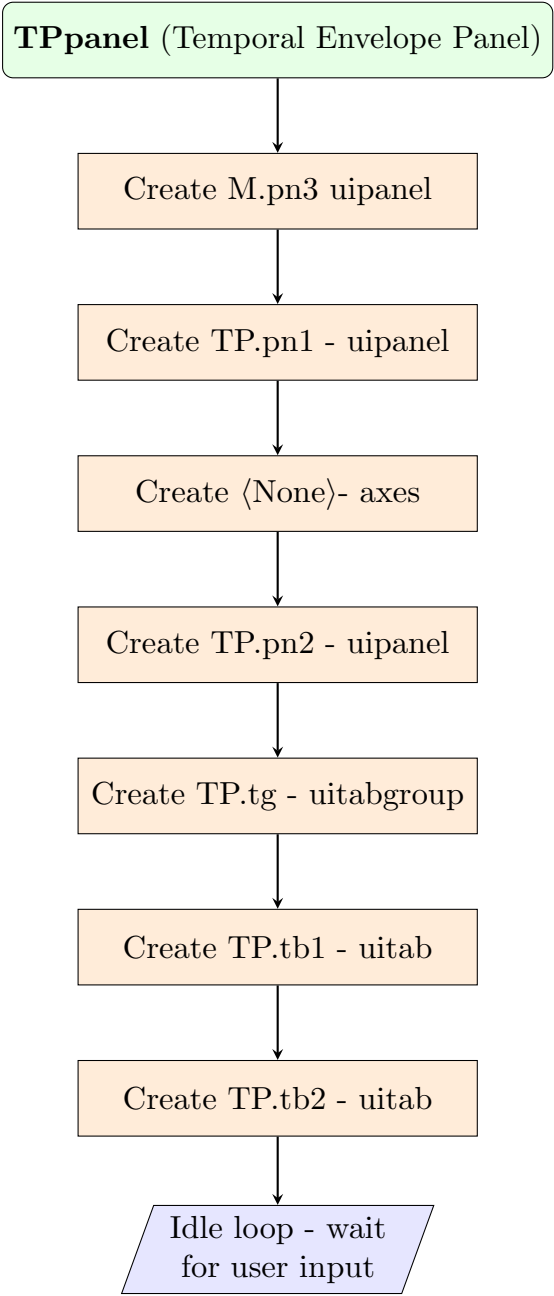
TP.tb1
Class: uitab { TPpanel.m } Parent: TP.tg{ TPpanel.m} Title: Observed Values
Child: TP.buttonGroup (uibuttongroup) { TPpanel.m } Child: TP.t1label (uicontrol{Text}) { TPpanel.m } Child: TP.t1Slider (uicontrol{Slider}) { TPpanel.m } Callback: TPCallBack1.t1 Child: <None>(uicontrol{Text}) { TPpanel.m } Child: TP.t1edit (uicontrol){Edit} { TPpanel.m } Callback: TPCallBack2.t1 Child: TP.alabel (uicontrol{Text}) { TPpanel.m } Child: TP.aSlider (uicontrol{Slider}) { TPpanel.m } Callback: TPCallBack1.a Child: <None>(uicontrol{Text}) { TPpanel.m } Child: TP.aedit (uicontrol){Edit} { TPpanel.m } Callback: TPCallBack2.a Child: TP.t2label (uicontrol{Text}) { TPpanel.m } Child: TP.t2Slider (uicontrol{Slider}) { TPpanel.m } Callback: TPCallBack1.t2 Child: <None>(uicontrol{Text}) { TPpanel.m } Child: TP.t2edit (uicontrol){Edit} { TPpanel.m } Callback: TPCallBack2.t2 Child: TP.clabel (uicontrol{Text}) { TPpanel.m } Child: TP.cSlider (uicontrol{Slider}) { TPpanel.m } Callback: TPCallBack1.c Child: <None>(uicontrol{Text}) { TPpanel.m } Child: TP.cedit (uicontrol){Edit} { TPpanel.m } Callback: TPCallBack2.c <i>Description: Creates Temporal Envelope panel Observed Values tab</i>

TP.buttonGroup
Class: uibuttongroup { TPpanel.m } Parent: TP.tb1{ TPpanel.m }
Child: TP.radioButton1 (uicontrol{radiobutton}) { TPpanel.m } Callback: RadioButtonCallBack Child: TP.radioButton2 (uicontrol{radiobutton}) { TPpanel.m } Callback: RadioButtonCallBack Child: TP.radioButton3 (uicontrol{radiobutton}) { TPpanel.m } Callback: RadioButtonCallBack <i>Description: Creates Radio Button Group</i>

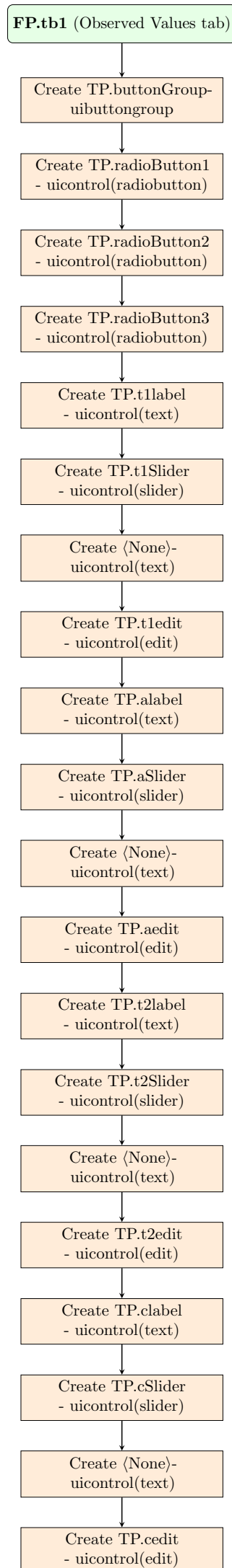
Hierarchy Diagram 10: Observed Values tab of the Temporal Envelope Panel

TP.tb2
Class: uitab { TPpanel.m } Parent: TP.tg { TPpanel.m} Title: User Values
Child: TP.t1label2 (uicontrol{Text}) { TPpanel.m } Child: TP.t1edit2 (uicontrol){Edit} { TPpanel.m } Callback: TPCallBack3.t1 Child: TP.alabel2 (uicontrol{Text}) { TPpanel.m } Child: TP.aedit2 (uicontrol){Edit} { TPpanel.m } Callback: TPCallBack3.a Child: TP.t2label2 (uicontrol{Text}) { TPpanel.m } Child: TP.t2edit2 (uicontrol){Edit} { TPpanel.m } Callback: TPCallBack3.t2 Child: TP.clabel2 (uicontrol{Text}) { TPpanel.m } Child: TP.cedit2 (uicontrol){Edit} { TPpanel.m } Callback: TPCallBack3.c
<i>Description: Creates Temporal Envelope panel User Values tab</i>

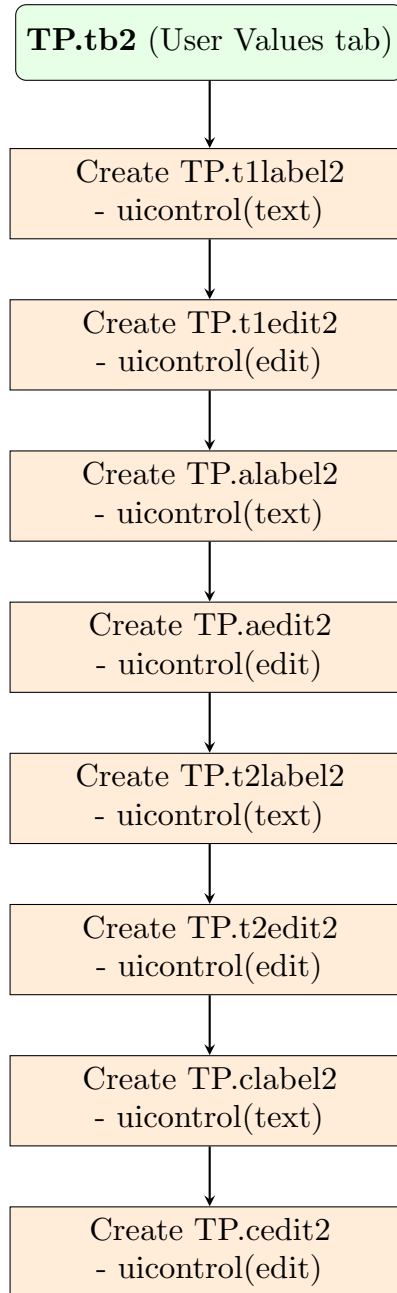
Hierarchy Diagram 11: User Values tab of the Temporal Envelope Panel



Flowchart 21: Temporal Envelope Panel



Flowchart 22: Observed Values tab of the Temporal Envelope Panel



Flowchart 23: User Values tab of the Temporal Envelope Panel

```

1 function imgzoompan(hfig, varargin)
2     if isempty(findobj('type','figure'))
3         fprintf('%s -- finds no open figure windows. Quitting.\n', mfilename)
4         return
5     end
6     if nargin==0 || isempty(hfig) || ~isa(hfig,'matlab.ui.Figure')
7         hfig = gcf;
8     end
9     p = inputParser;
10    p.addOptional('Magnify', 1.1, @isnumeric);
11    p.addOptional('XMagnify', 1.0, @isnumeric);
12    p.addOptional('YMagnify', 1.0, @isnumeric);
13    p.addOptional('ChangeMagnify', 1.1, @isnumeric);
14    p.addOptional('IncreaseChange', 1.1, @isnumeric);
15    p.addOptional('MinValue', 1.1, @isnumeric);
16    p.addOptional('MaxZoomScrollCount', 30, @isnumeric);
17    p.addOptional('ImgWidth', 0, @isnumeric);
18    p.addOptional('ImgHeight', 0, @isnumeric);
19    p.addOptional('PanMouseButton', 2, @isnumeric);
20    p.addOptional('ResetMouseButton', 3, @isnumeric);
21    p.addOptional('ButtonDownFcn', @(~,~) 0);
22    p.addOptional('ButtonUpFcn', @(~,~) 0);
23    parse(p, varargin{:});
24    opt = p.Results;

```

```

25 if opt.Magnify<opt.MinValue
26     opt.Magnify=opt.MinValue;
27 end
28 if opt.ChangeMagnify<opt.MinValue
29     opt.ChangeMagnify=opt.MinValue;
30 end
31 if opt.IncreaseChange<opt.MinValue
32     opt.IncreaseChange=opt.MinValue;
33 end
34 set(hfig,'WindowScrollWheelFcn',@zoom_fcn);
35 set(hfig,'WindowButtonDownFcn',@down_fcn);
36 set(hfig,'WindowButtonUpFcn',@up_fcn);
37 zoomScrollCount = 0;
38 orig.h=[];
39 orig.XLim=[];
40 orig.YLim=[];
41 function zoom_fcn(src, cbdata)
42     scrollChange = cbdata.VerticalScrollCount;
43     if ((zoomScrollCount - scrollChange) <= opt.MaxZoomScrollCount)
44         axish = gca;
45         if (isempty(orig.h) || axish ~= orig.h)
46             orig.h = axish;
47             orig.XLim = axish.XLim;
48             orig.YLim = axish.YLim;
49         end
50         cpaxes = mean(axish.CurrentPoint);
51         newXLim = (axish.XLim - cpaxes(1)) * (opt.Magnify * opt.XMagnify)^scrollChange +
52             ↪ cpaxes(1);
53         newYLim = (axish.YLim - cpaxes(2)) * (opt.Magnify * opt.YMagnify)^scrollChange +
54             ↪ cpaxes(2);
55         newXLim = floor(newXLim);
56         newYLim = floor(newYLim);
57         if (opt.ImgWidth > 0)
58             if (newXLim(1) >= 0 && newXLim(2) <= opt.ImgWidth && newYLim(1) >= 0 &&
59                 ↪ newYLim(2) <= opt.ImgHeight)
60                 axish.XLim = newXLim;
61                 axish.YLim = newYLim;
62                 zoomScrollCount = zoomScrollCount - scrollChange;
63             else
64                 axish.XLim = orig.XLim;
65                 axish.YLim = orig.YLim;
66                 zoomScrollCount = 0;
67             end
68         else
69             axish.XLim = newXLim;
70             axish.YLim = newYLim;
71             zoomScrollCount = zoomScrollCount - scrollChange;
72         end
73     end
74 end
75 function down_fcn(hObject, evt)
76     opt.ButtonDownFcn(hObject, evt);
77     clickType = evt.Source.SelectionType;
78     panBt = opt.PanMouseButton;
79     if (panBt > 0)
80         if (panBt == 1 && strcmp(clickType, 'normal')) || ...
81             (panBt == 2 && strcmp(clickType, 'alt')) || ...
82             (panBt == 3 && strcmp(clickType, 'extend'))
83             guiArea = hittest(hObject);
84             parentAxes = ancestor(guiArea,'axes');
85             if ~isempty(parentAxes)

```

```

83         startPan(parentAxes)
84     else
85         setptr(evt.Source,'forbidden')
86     end
87 end
88 end
89 end
90 function up_fcn(hObject, evt)
91     opt.ButtonUpFcn(hObject, evt);
92     clickType = evt.Source.SelectionType;
93     resBt = opt.ResetMouseButton;
94     if (resBt > 0 && ~isempty(orig.XLim))
95         if (resBt == 1 && strcmp(clickType, 'normal')) || ...
96             (resBt == 2 && strcmp(clickType, 'alt')) || ...
97             (resBt == 3 && strcmp(clickType, 'extend'))
98             guiArea = hittest(hObject);
99             parentAxes = ancestor(guiArea,'axes');
100             parentAxes.XLim=orig.XLim;
101             parentAxes.YLim=orig.YLim;
102         end
103     end
104     stopPan
105 end
106 function startPan(hAx)
107     hFig = ancestor(hAx, 'Figure', 'toplevel');
108     seedPt = get(hAx, 'CurrentPoint');
109     seedPt = seedPt(1, :);
110     hAx.XLimMode = 'manual';
111     hAx.YLimMode = 'manual';
112     set(hFig,'WindowButtonMotionFcn',{@panningFcn,hAx,seedPt});
113     setptr(hFig, 'hand');
114 end
115 function stopPan
116     set(gcbf,'WindowButtonMotionFcn',[]);
117     setptr(gcbf,'arrow');
118 end
119 function panningFcn(~,~,hAx,seedPt)
120     currPt = get(hAx,'CurrentPoint');
121     XLim = hAx.XLim;
122     YLim = hAx.YLim;
123     x_seed = (seedPt(1)-XLim(1))/(XLim(2)-XLim(1));
124     y_seed = (seedPt(2)-YLim(1))/(YLim(2)-YLim(1));
125     x_curr = (currPt(1,1)-XLim(1))/(XLim(2)-XLim(1));
126     y_curr = (currPt(1,2)-YLim(1))/(YLim(2)-YLim(1));
127     deltaX = x_curr-x_seed;
128     deltaY = y_curr-y_seed;
129     newXLims(1) = -deltaX*diff(XLim)+XLim(1);
130     newXLims(2) = newXLims(1)+diff(XLim);
131     newYLims(1) = -deltaY*diff(YLim)+YLim(1);
132     newYLims(2) = newYLims(1)+diff(YLim);
133     newXLims = round(newXLims);
134     newYLims = round(newYLims);
135     if (newXLims(1) > 0.0 && newXLims(2) < opt.ImgWidth)
136         set(hAx,'Xlim',newXLims);
137     end
138     if (newYLims(1) > 0.0 && newYLims(2) < opt.ImgHeight)
139         set(hAx,'Ylim',newYLims);
140     end
141 end
142 end
143 end

```

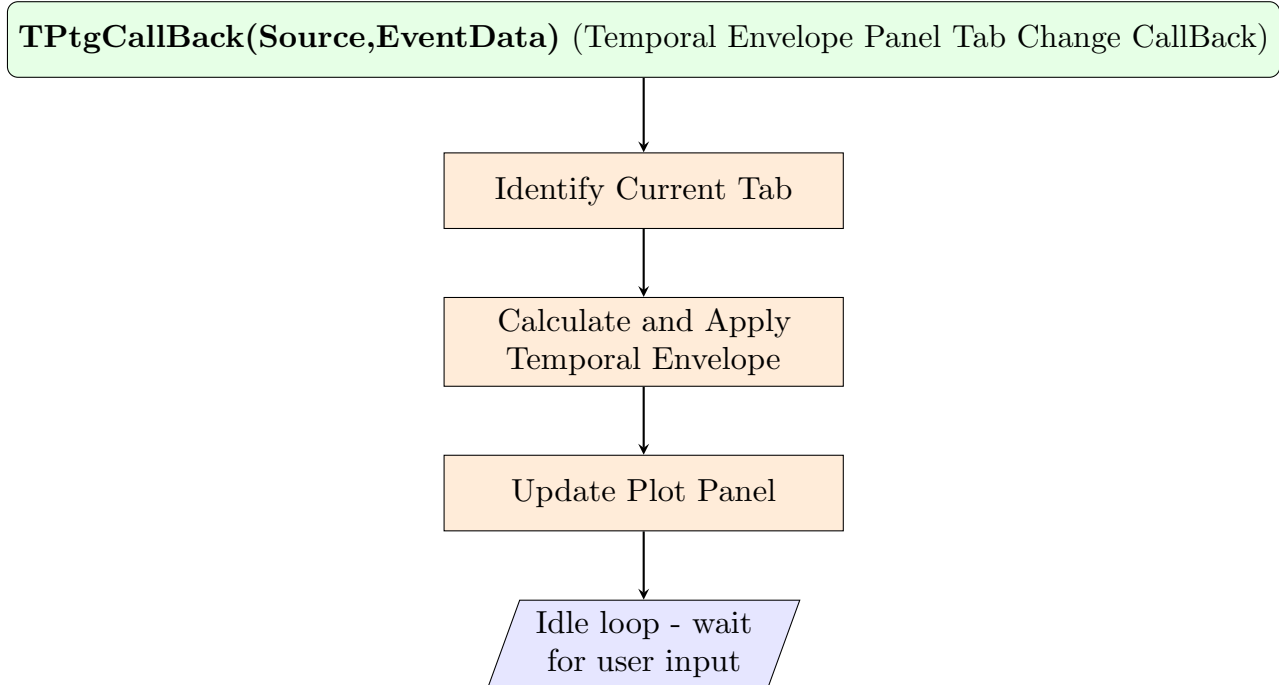
Code 27: *imgzoompan.m*: Function that implements panning and zooming of the *Temporal Envelope* panel image

```

1 function TPtgCallBack(Source,EventData)
2
3     global M GP FP TP ZZ TN GG SP
4
5     t = EventData.NewValue.Title;
6     if strcmp(t,'Obseved Values')
7         TP.N01 = floor(TP.t1one*length(GP.tt)/100);
8         TP.n01 = TP.N01;
9         TP.win01 = ((1:TP.n01)/TP.n01).^TP.aone;
10        TP.win01 = TP.win01(:);
11        TP.N02 = ceil(TP.t2one*length(GP.tt)/100);
12        TP.N03 = length(GP.tt);
13        TP.n03 = TP.N03 - TP.N02 + 1;
14        TP.win03 = exp(TP.cone*(1:TP.n03)/TP.n03);
15        TP.win03 = TP.win03(:);
16        TP.n02 = length(GP.tt) - length(TP.win01) - length(TP.win03);
17        TP.win02 = ones(TP.n02,1);
18        TP.winNone = [TP.win01; TP.win02; TP.win03];
19        TP.winN = TP.winNone;
20    elseif strcmp(t,'User Values')
21        TP.N01 = floor(TP.t1two*length(GP.tt)/100);
22        TP.n01 = TP.N01;
23        TP.win01 = ((1:TP.n01)/TP.n01).^TP.atwo;
24        TP.win01 = TP.win01(:);
25        TP.N02 = ceil(TP.t2two*length(GP.tt)/100);
26        TP.N03 = length(GP.tt);
27        TP.n03 = TP.N03 - TP.N02 + 1;
28        TP.win03 = exp(TP.ctwo*(1:TP.n03)/TP.n03);
29        TP.win03 = TP.win03(:);
30        TP.n02 = length(GP.tt) - length(TP.win01) - length(TP.win03);
31        TP.win02 = ones(TP.n02,1);
32        TP.winNtwo = [TP.win01; TP.win02; TP.win03];
33        TP.winN = TP.winNtwo;
34    end
35
36    TP.mqS = FP.NGenAcc.*TP.winN;
37    TP.mqSN = TP.mqS/max(abs(TP.mqS));
38    [TP.psd,TP.ff] = pwelch(TP.mqSN*TP.PGA,GP.nfft,GP.noverlap,GP.window,GP.fs);
39
40    set(M.LN5,'XData',GP.tt/60,'YData',TP.mqSN*TP.PGA);
41    set(M.LN5.Parent,'YLim',[-1.1 1.1]*TP.PGA);
42    set(M.LN6,'XData',GP.tt/60,'YData',TP.winN*TP.PGA);
43    set(M.LN7,'XData',GP.tt/60,'YData',envelope(TP.mqSN*TP.PGA,floor(length(
44        ↪ TP.mqSN)/100),'rms'));
45    set(M.LN8,'XData',TP.ff,'YData',db(TP.psd));
46
47    guidata(Source,M);
48 end

```

Code 28: *TPtgCallBack.m*: Callback for the change of tab in the *Temporal Envelope* panel



Flowchart 24: Callback for the change of tab in the Temporal Envelope Panel

```

1 function RadioButtonCallBack(Source,~, ~)
2
3     global M GP FP TP ZZ TN GG SP
4
5     M.change = 1;
6
7     isChecked1 = get(TP.radioButton1, 'Value');
8     isChecked2 = get(TP.radioButton2, 'Value');
9     isChecked3 = get(TP.radioButton3, 'Value');
10
11     if isChecked1
12         TP.Checked = 1;
13         TP.t1minone = 4.9112;
14         TP.t1one = 10.3561;
15         TP.t1maxone = 19.9832;
16         TP.aminone = 0.1879;
17         TP.aone = 1.6230;
18         TP.amaxone = 4.3490;
19         TP.t2minone = 4.9164;
20         TP.t2one = 12.2103;
21         TP.t2maxone = 22.6608;
22         TP.cminone = -4.8428;
23         TP.cone = -3.1044;
24         TP.cmaxone = -1.7363;
25     elseif isChecked2
26         TP.Checked = 2;
27         TP.t1minone = 4.8709;
28         TP.t1one = 8.4829;
29         TP.t1maxone = 14.1745;
30         TP.aminone = 0.3714;
31         TP.aone = 0.7460;
32         TP.amaxone = 2.2694;
33         TP.t2minone = 4.8756;
34         TP.t2one = 9.5416;
35         TP.t2maxone = 16.3818;
36         TP.cminone = -5.0450;
37         TP.cone = -3.2158;
38         TP.cmaxone = -1.7919;
39     elseif isChecked3
40         TP.Checked = 3;

```

```

41     TP.t1minone = 3.7868;
42     TP.t1one = 6.0933;
43     TP.t1maxone = 7.8382;
44     TP.aminone = 0.3811;
45     TP.aone = 1.0063;
46     TP.amaxone = 1.7402;
47     TP.t2minone = 5.2963;
48     TP.t2one = 8.7476;
49     TP.t2maxone = 11.2301;
50     TP.cminone = -6.8821;
51     TP.cone = -4.8535;
52     TP.cmaxone = -3.8256;
53 end
54 set(TP.t1Slider,'min',TP.t1minone,'max',TP.t1maxone,'value',TP.t1one);
55 set(TP.aSlider,'min',TP.aminone,'max',TP.amaxone,'value',TP.aone);
56 set(TP.t2Slider,'min',TP.t2minone,'max',TP.t2maxone,'value',TP.t2one);
57 set(TP.cSlider,'min',TP.cminone,'max',TP.cmaxone,'value',TP.cone);
58 set(TP.t1edit,'string',num2str(TP.t1one,'%3f'));
59 set(TP.aedit,'string',num2str(TP.aone,'%3f'));
60 set(TP.t2edit,'string',num2str(TP.t2one,'%3f'));
61 set(TP.cedit,'string',num2str(TP.cone,'%3f'));
62
63 TPslid = [0.025 0.70-0.05 0.725 0.045];
64 TP1bl = [0 TPslid(2)-0.03 0.15 0.03];
65 Tnticks = 5;
66 Tlblmargin = 0.05;
67 Tlblvals = linspace(TP.t1minone,TP.t1maxone,Tnticks);
68 for k = 1:Tnticks
69     Txpos = (Tlblmargin + (k-1)*(TPslid(3) - 2*Tlblmargin)/(Tnticks-1)) - TP1bl(3)/2;
70     TP1bl(1) = 0.025 + Txpos;
71     uicontrol('Parent',TP.tb1,...
72             'Units','normalized',...
73             'FontSize',6,...
74             'HorizontalAlignment','center',...
75             'Style','text',...
76             'Position',TP1bl,...
77             'String',sprintf('%3f',Tlblvals(k)),...
78             'Tag','t1Slider1ticklabel');
79 end
80
81 TPslid = [0.025 0.50-0.05 0.725 0.045];
82 TP1bl = [0 TPslid(2)-0.03 0.15 0.03];
83 Tnticks = 5;
84 Tlblmargin = 0.05;
85 Tlblvals = linspace(TP.aminone,TP.amaxone,Tnticks);
86 for k = 1:Tnticks
87     Txpos = (Tlblmargin + (k-1)*(TPslid(3) - 2*Tlblmargin)/(Tnticks-1)) - TP1bl(3)/2;
88     TP1bl(1) = 0.025 + Txpos;
89     uicontrol('Parent',TP.tb1,...
90             'Units','normalized',...
91             'FontSize',6,...
92             'HorizontalAlignment','center',...
93             'Style','text',...
94             'Position',TP1bl,...
95             'String',sprintf('%3f',Tlblvals(k)),...
96             'Tag','aSlider1ticklabel');
97 end
98
99 TPslid = [0.025 0.30-0.05 0.725 0.045];
100 TP1bl = [0 TPslid(2)-0.03 0.15 0.03];
101 Tnticks = 5;

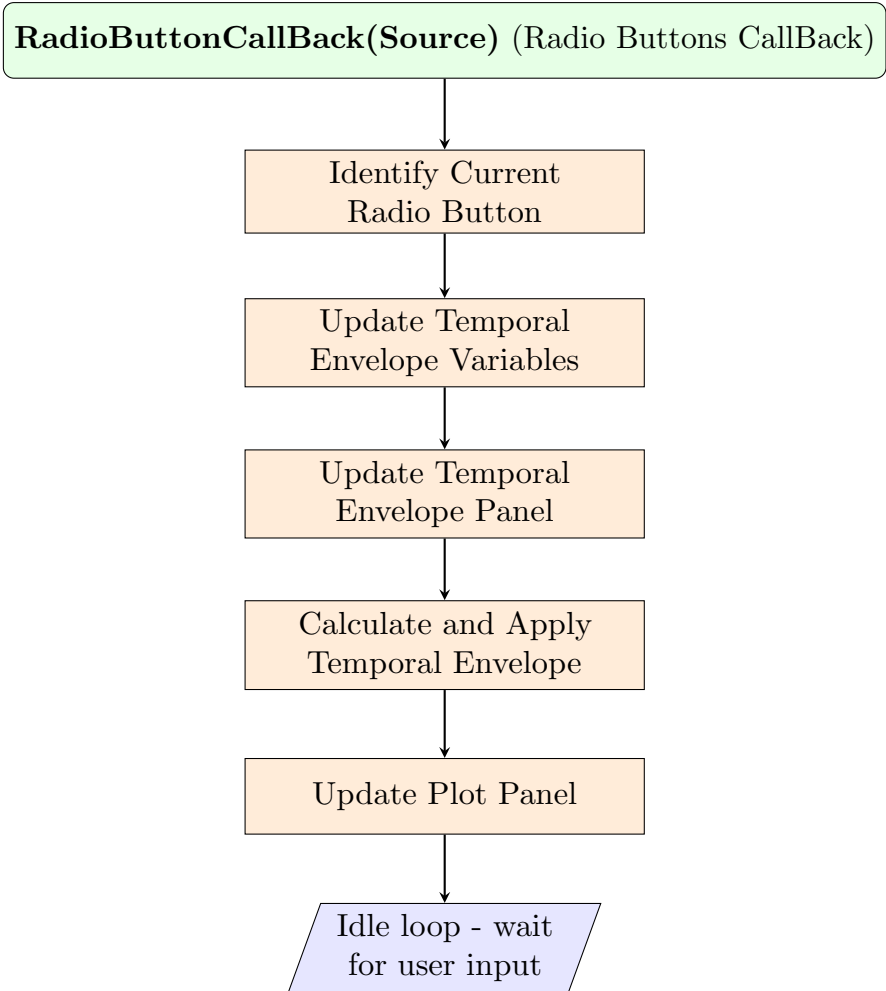
```

```

102 Tlblmargin = 0.05;
103 Tlblvals = linspace(TP.t2minone,TP.t2maxone,Tnticks);
104 for k = 1:Tnticks
105     Txpos = (Tlblmargin + (k-1)*(TPslid(3) - 2*Tlblmargin)/(Tnticks-1)) - TPlbl(3)/2;
106     TPlbl(1) = 0.025 + Txpos;
107     uicontrol('Parent',TP.tb1,...
108               'Units','normalized',...
109               'FontSize',6,...
110               'HorizontalAlignment','center',...
111               'Style','text',...
112               'Position',TPlbl,...
113               'String',sprintf('%.3f',Tlblvals(k)),...
114               'Tag','t2slider1ticklabel');
115 end
116
117 TPslid = [0.025 0.10-0.05 0.725 0.045];
118 TPlbl = [0 TPslid(2)-0.03 0.15 0.03];
119 Tnticks = 5;
120 Tlblmargin = 0.05;
121 Tlblvals = linspace(TP.cminone,TP.cmaxone,Tnticks);
122 for k = 1:Tnticks
123     Txpos = (Tlblmargin + (k-1)*(TPslid(3) - 2*Tlblmargin)/(Tnticks-1)) - TPlbl(3)/2;
124     TPlbl(1) = 0.025 + Txpos;
125     uicontrol('Parent',TP.tb1,...
126               'Units','normalized',...
127               'FontSize',6,...
128               'HorizontalAlignment','center',...
129               'Style','text',...
130               'Position',TPlbl,...
131               'String',sprintf('%.3f',Tlblvals(k)),...
132               'Tag','cslider1ticklabel');
133 end
134
135 TP.N01 = floor(TP.t1one*length(GP.tt)/100);
136 TP.n01 = TP.N01;
137 TP.win01 = ((1:TP.n01)/TP.n01).^TP.aone;
138 TP.win01 = TP.win01(:);
139 TP.N02 = ceil(TP.t2one*length(GP.tt)/100);
140 TP.N03 = length(GP.tt);
141 TP.n03 = TP.N03 - TP.N02 + 1;
142 TP.win03 = exp(TP.cone*(1:TP.n03)/TP.n03);
143 TP.win03 = TP.win03(:);
144 TP.n02 = length(GP.tt) - length(TP.win01) - length(TP.win03);
145 TP.win02 = ones(TP.n02,1);
146 TP.winNone = [TP.win01; TP.win02; TP.win03];
147 TP.winN = TP.winNone;
148
149 TP.mqS = FP.NGenAcc.*TP.winN;
150 TP.mqSN = TP.mqS/max(abs(TP.mqS));
151 [TP.psd,TP.ff] = pwelch(TP.mqSN*TP.PGA,GP.nfft,GP.noverlap,GP.window,GP.fs);
152
153 set(M.LN5,'XData',GP.tt/60,'YData',TP.mqSN*TP.PGA);
154 set(M.LN5.Parent,'YLim',[-1.1 1.1]*TP.PGA);
155 set(M.LN6,'XData',GP.tt/60,'YData',TP.winN*TP.PGA);
156 set(M.LN7,'XData',GP.tt/60,'YData',envelope(TP.mqSN*TP.PGA,floor(length(GP.tt)/
    ↪ TP.mqSN)/100),'rms'));
157 set(M.LN8,'XData',TP.ff,'YData',db(TP.psd));
158
159 guidata(Source,M);
160 end

```

Code 29: *RadioButtonCallBack.m*: Callback of the radio button group of the *Temporal Envelope* panel



Flowchart 25: Callback for the radio button group in the Temporal Envelope Panel

```
1 function TPCallBack1(Source,~,Param)
2
3     global M GP FP TP ZZ TN GG SP
4
5     M.change = 1;
6
7     TP.(Param) = get(Source, 'Value');
8
9     if strcmp(Param,'t1')
10         TP.t1one = TP.t1;
11         if TP.t2one <= TP.t1one
12             TP.t2one = TP.t1one + 0.004;
13         end
14     end
15
16     if strcmp(Param,'a')
17         TP.aone = TP.a;
18     end
19
20     if strcmp(Param,'t2')
21         TP.t2one = TP.t2;
22         if TP.t2one <= TP.t1one
23             TP.t1one = TP.t2one - 0.004;
24         end
25     end
26
27     if strcmp(Param,'c')
28         TP.cone = TP.c;
29     end
```

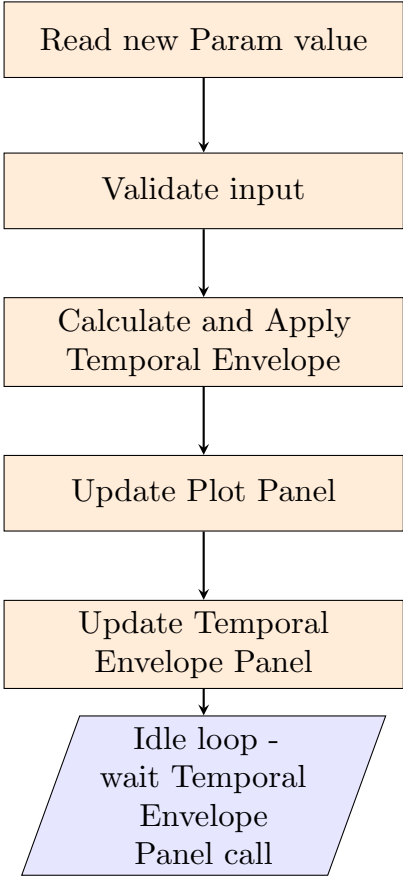
```

30
31     TP.N01 = floor(TP.t1one*length(GP.tt)/100);
32     TP.n01 = TP.N01;
33     TP.win01 = ((1:TP.n01)/TP.n01).^TP.aone;
34     TP.win01 = TP.win01(:);
35     TP.N02 = ceil(TP.t2one*length(GP.tt)/100);
36     TP.N03 = length(GP.tt);
37     TP.n03 = TP.N03 - TP.N02 + 1;
38     TP.win03 = exp(TP.cone*(1:TP.n03)/TP.n03);
39     TP.win03 = TP.win03(:);
40     TP.n02 = length(GP.tt) - length(TP.win01) - length(TP.win03);
41     TP.win02 = ones(TP.n02,1);
42     TP.winNone = [TP.win01; TP.win02; TP.win03];
43     TP.winN = TP.winNone;
44
45     TP.mqS = FP.NGenAcc.*TP.winN;
46     TP.mqSN = TP.mqS/max(abs(TP.mqS));
47     [TP.psd,TP.ff] = pwelch(TP.mqSN*TP.PGA,GP.nfft,GP.noverlap,GP.window,GP.fs);
48
49     set(M.LN5,'XData',GP.tt/60,'YData',TP.mqSN*TP.PGA);
50     set(M.LN5.Parent,'YLim',[-1.1 1.1]*TP.PGA);
51     set(M.LN6,'XData',GP.tt/60,'YData',TP.winN*TP.PGA);
52     set(M.LN7,'XData',GP.tt/60,'YData',envelope(TP.mqSN*TP.PGA,floor(length(GP.tt)/
    ↪ TP.mqSN)/100),'rms'));
53     set(M.LN8,'XData',TP.ff,'YData',db(TP.psd));
54
55     set(TP.t1Slider,'Value',TP.t1one);
56     set(TP.aSlider,'Value',TP.aone);
57     set(TP.t2Slider,'Value',TP.t2one);
58     set(TP.cSlider,'Value',TP.cone);
59     set(TP.t1edit,'string',num2str(TP.t1one,'%3f'));
60     set(TP.aedit,'string',num2str(TP.aone,'%3f'));
61     set(TP.t2edit,'string',num2str(TP.t2one,'%3f'));
62     set(TP.cedit,'string',num2str(TP.cone,'%3f'));
63
64     guidata(Source,M);
65
66 end

```

Code 30: *TPCallback1.m*: Callback for the slider controls of the *Observed Values* tab in the *Temporal Envelope* panel

TPCallback1(Source,Param) (Sliders CallBack)



Flowchart 26: Callback for the sliders controls

```

1 function TPCallBack2(Source,~,Param)
2
3     global M GP FP TP ZZ TN GG SP
4
5     if isnan(str2double(get(Source,'String')))
6         if strcmp(Param,'t1')
7             set(Source,'string',num2str(TP.t1one,'%.3f'));
8         elseif strcmp(Param,'a')
9             set(Source,'string',num2str(TP.aone,'%.3f'));
10        elseif strcmp(Param,'t2')
11            set(Source,'string',num2str(TP.t2one,'%.3f'));
12        elseif strcmp(Param,'c')
13            set(Source,'string',num2str(TP.cone,'%.3f'));
14        end
15    else
16        M.change = 1;
17
18        TP.(Param) = str2double(get(Source,'String'));
19
20    if strcmp(Param,'t1')
21        if TP.Checked == 1
22            if TP.t1 < 4.9112
23                TP.t1 = 4.9112;
24            elseif TP.t1 > 19.9832
25                TP.t1 = 19.9832;
26            end
27        elseif TP.Checked == 2
28            if TP.t1 < 4.8709
29                TP.t1 = 4.8709;
30            elseif TP.t1 > 14.1745
31                TP.t1 = 14.1745;
  
```

```

32     end
33 elseif TP.Checked == 3
34     if TP.t1 < 3.7868
35         TP.t1 = 3.7868;
36     elseif TP.t1 > 7.8382
37         TP.t1 = 7.8382;
38     end
39 end
40 TP.t1one = TP.t1;
41 if TP.t2one <= TP.t1one
42     TP.t2one = TP.t1one + 0.004;
43 end
44 end
45
46 if strcmp(Param,'a')
47     if TP.Checked == 1
48         if TP.a <= 0.1879
49             TP.a = 0.1879;
50         elseif TP.a > 4.3490
51             TP.a = 4.3490;
52         end
53     elseif TP.Checked == 2
54         if TP.a <= 0.3714
55             TP.a = 0.3714;
56         elseif TP.a > 2.2694
57             TP.a = 2.2694;
58         end
59     elseif TP.Checked == 3
60         if TP.a <= 0.3811
61             TP.a = 0.3811;
62         elseif TP.a > 1.7402
63             TP.a = 1.7402;
64         end
65     end
66
67     TP.aone = TP.a;
68 end
69
70 if strcmp(Param,'t2')
71     if TP.Checked == 1
72         if TP.t2 < 4.9164
73             TP.t2 = 4.9164;
74         elseif TP.t2 > 22.6608
75             TP.t2 = 22.6608;
76         end
77     elseif TP.Checked == 2
78         if TP.t2 < 4.8756
79             TP.t2 = 4.8756;
80         elseif TP.t2 > 16.3818
81             TP.t2 = 16.3818;
82         end
83     elseif TP.Checked == 3
84         if TP.t2 < 5.2963
85             TP.t2 = 5.2963;
86         elseif TP.t2 > 11.2301
87             TP.t2 = 11.2301;
88         end
89     end
90     TP.t2one = TP.t2;
91     if TP.t2one <= TP.t1one
92         TP.t1one = TP.t2one - 0.004;

```

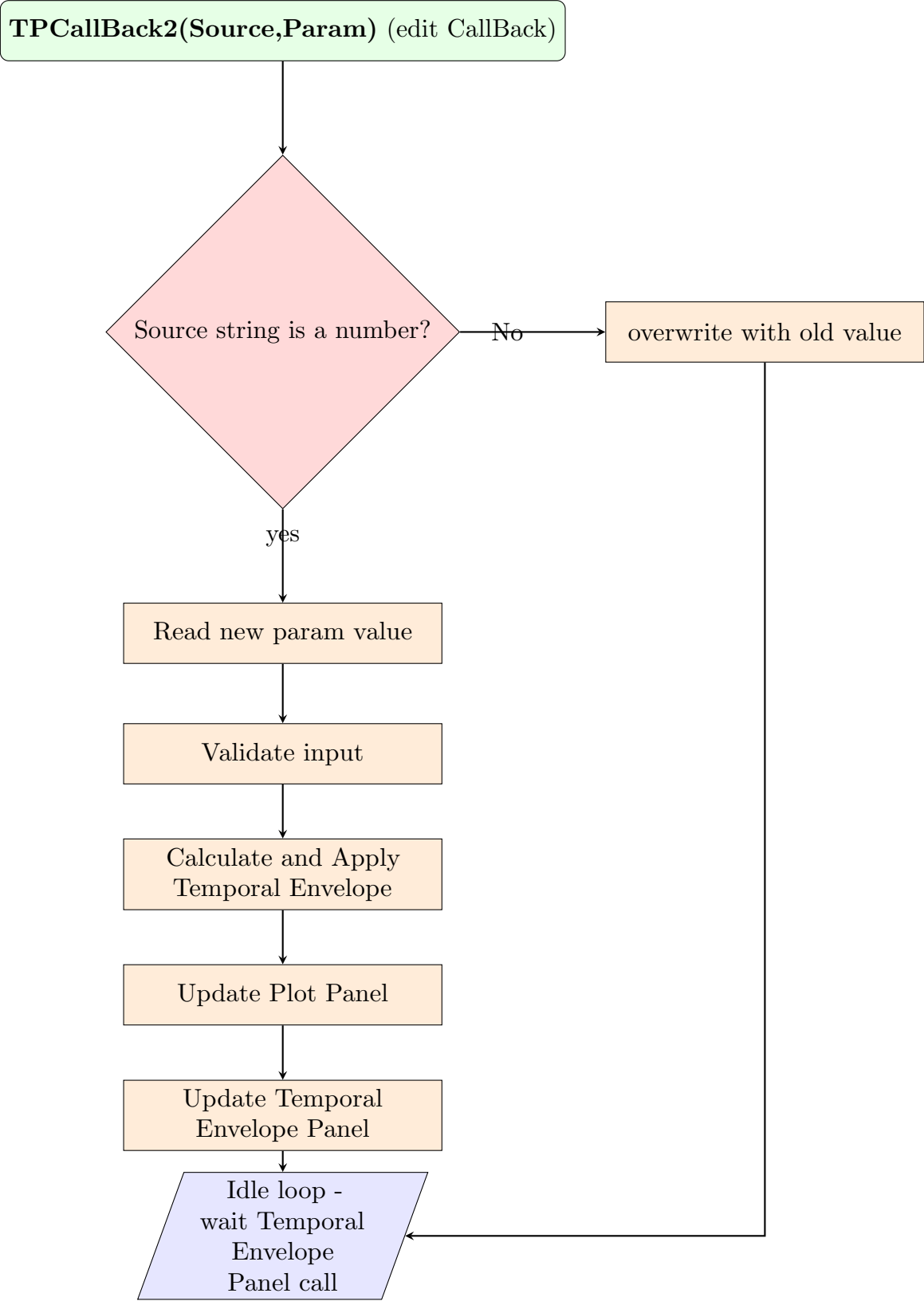
```

93     end
94 end
95
96 if strcmp(Param,'c')
97     if TP.Checked == 1
98         if TP.c < -4.8428
99             TP.c = -4.8428;
100         elseif TP.c > -1.7363
101             TP.c = -1.7363;
102         end
103     elseif TP.Checked == 2
104         if TP.c < -5.0450
105             TP.c = -5.0450;
106         elseif TP.c > -1.7919
107             TP.c = -1.7919;
108         end
109     elseif TP.Checked == 3
110         if TP.c < -6.8821
111             TP.c = -6.8821;
112         elseif TP.c > -3.8256
113             TP.c = -3.8256;
114         end
115     end
116     TP.cone = TP.c;
117 end
118
119 TP.N01 = floor(TP.t1one*length(GP.tt)/100);
120 TP.n01 = TP.N01;
121 TP.win01 = ((1:TP.n01)/TP.n01).^TP.aone;
122 TP.win01 = TP.win01(:);
123 TP.N02 = ceil(TP.t2one*length(GP.tt)/100);
124 TP.N03 = length(GP.tt);
125 TP.n03 = TP.N03 - TP.N02 + 1;
126 TP.win03 = exp(TP.cone*(1:TP.n03)/TP.n03);
127 TP.win03 = TP.win03(:);
128 TP.n02 = length(GP.tt) - length(TP.win01) - length(TP.win03);
129 TP.win02 = ones(TP.n02,1);
130 TP.winNone = [TP.win01; TP.win02; TP.win03];
131 TP.winN = TP.winNone;
132
133 TP.mqS = FP.NGenAcc.*TP.winN;
134 TP.mqSN = TP.mqS/max(abs(TP.mqS));
135 [TP.psd,TP.ff] = pwelch(TP.mqSN*TP.PGA,GP.nfft,GP.noverlap,GP.window,GP.fs);
136
137 set(M.LN5,'XData',GP.tt/60,'YData',TP.mqSN*TP.PGA);
138 set(M.LN5.Parent,'YLim',[-1.1 1.1]*TP.PGA);
139 set(M.LN6,'XData',GP.tt/60,'YData',TP.winN*TP.PGA);
140 set(M.LN7,'XData',GP.tt/60,'YData',envelope(TP.mqSN*TP.PGA,floor(length(GP.tt)/
    ↪ TP.mqSN)/100),'rms');
141 set(M.LN8,'XData',TP.ff,'YData',db(TP.psd));
142
143 set(TP.t1Slider,'Value',TP.t1one);
144 set(TP.aSlider,'Value',TP.aone);
145 set(TP.t2Slider,'Value',TP.t2one);
146 set(TP.cSlider,'Value',TP.cone);
147 set(TP.t1edit,'string',num2str(TP.t1one,'%.3f'));
148 set(TP.aedit,'string',num2str(TP.aone,'%.3f'));
149 set(TP.t2edit,'string',num2str(TP.t2one,'%.3f'));
150 set(TP.cedit,'string',num2str(TP.cone,'%.3f'));
151
152 guidata(Source,M);

```

153
154 end

Code 31: *TPCallback2.m*: Callback for the editable text box controls of the *Observed Values* tab in the *Temporal Envelope* panel



Flowchart 27: Callback for the text edit controls of the Observed Values tab

```
1 function TPCallBack3(Source,~,Param)
2
3     global M GP FP TP ZZ TN GG SP
4
5     if isnan(str2double(get(Source, 'String')))
6         set(Source,'string',num2str(TP.(Param), '%.3f'));
7     else
8         M.change = 1;
```

```

9
10 TP.(Param) = str2double(get(Source,'String'));
11
12 if strcmp(Param,'t1')
13     if TP.t1 < 1
14         TP.t1 = 1;
15     elseif TP.t1 > 98
16         TP.t1 = 98;
17     end
18     TP.t1two = TP.t1;
19     if TP.t2two <= TP.t1two
20         TP.t2two = TP.t1two + 0.004;
21     end
22 end
23
24 if strcmp(Param,'a')
25     if TP.a < 0.01
26         TP.a = 0.01;
27     elseif TP.a > 10
28         TP.a = 10;
29     end
30     TP.atwo = TP.a;
31 end
32
33 if strcmp(Param,'t2')
34     if TP.t2 < 2
35         TP.fh = 2;
36     elseif TP.t2 > 99
37         TP.t2 = 99;
38     end
39     TP.t2two = TP.t2;
40     if TP.t2two <= TP.t1two
41         TP.t1two = TP.t2two - 0.004;
42     end
43 end
44
45 if strcmp(Param,'c')
46     if TP.c < -10
47         TP.c = -10;
48     elseif TP.c > -1
49         TP.c = -1;
50     end
51     TP.ctwo = TP.c;
52 end
53
54 TP.N01 = floor(TP.t1two*length(GP.tt)/100);
55 TP.n01 = TP.N01;
56 TP.win01 = ((1:TP.n01)/TP.n01).^TP.atwo;
57 TP.win01 = TP.win01(:);
58 TP.N02 = ceil(TP.t2two*length(GP.tt)/100);
59 TP.N03 = length(GP.tt);
60 TP.n03 = TP.N03 - TP.N02 + 1;
61 TP.win03 = exp(TP.ctwo*(1:TP.n03)/TP.n03);
62 TP.win03 = TP.win03(:);
63 TP.n02 = length(GP.tt) - length(TP.win01) - length(TP.win03);
64 TP.win02 = ones(TP.n02,1);
65 TP.winNtwo = [TP.win01; TP.win02; TP.win03];
66 TP.winN = TP.winNtwo;
67
68 TP.mqS = FP.NGenAcc.*TP.winN;
69 TP.mqSN = TP.mqS/max(abs(TP.mqS));

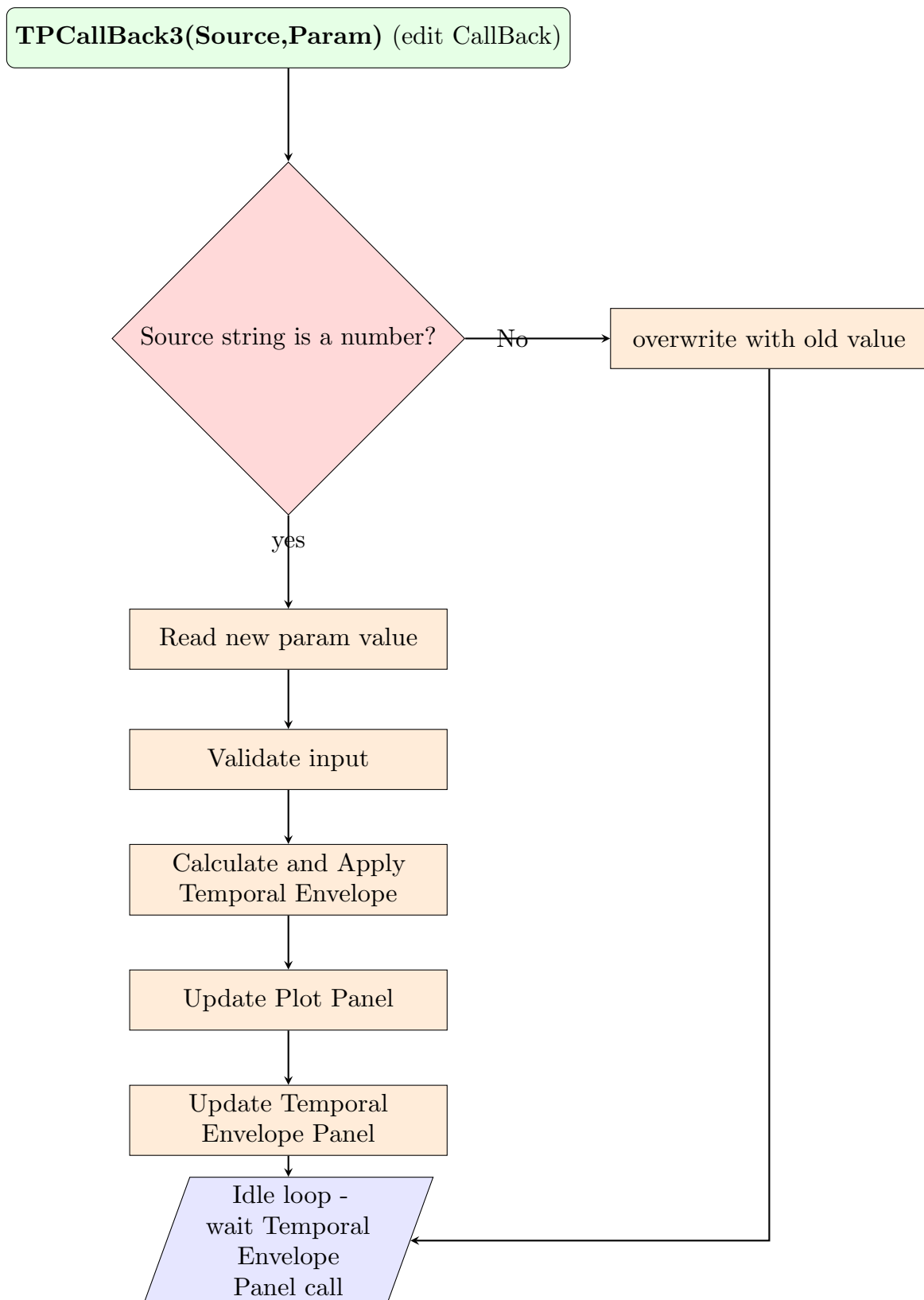
```

```

70     [TP.psd,TP.ff] = pwelch(TP.mqSN*TP.PGA,GP.nfft,GP.noverlap,GP.window,GP.fs);
71
72     set(M.LN5,'XData',GP.tt/60,'YData',TP.mqSN*TP.PGA);
73     set(M.LN5.Parent,'YLim',[-1.1 1.1]*TP.PGA);
74     set(M.LN6,'XData',GP.tt/60,'YData',TP.winN*TP.PGA);
75     set(M.LN7,'XData',GP.tt/60,'YData',envelope(TP.mqSN*TP.PGA,floor(length(
↵ TP.mqSN)/100),'rms'));
76     set(M.LN8,'XData',TP.ff,'YData',db(TP.psd));
77
78     set(TP.t1edit2,'string',num2str(TP.t1two,'%.3f'));
79     set(TP.aedit2,'string',num2str(TP.atwo,'%.3f'));
80     set(TP.t2edit2,'string',num2str(TP.t2two,'%.3f'));
81     set(TP.cedit2,'string',num2str(TP.ctwo,'%.3f'));
82 end
83
84 guidata(Source,M);
85
86 end

```

Code 32: *TPCallback3.m*: Callback for the editable text box controls of the *User Values* tab in the *Temporal Envelope* panel



Flowchart 28: Callback for the text edit controls of the User Values tab

5.2.6 PGA Panel

```

1 M.pn4 = uipanel('Parent',M.tb1,...
2     'Units','normalized',...
3     'Position',[0.30 0.010 0.20 0.065],...
4     'Title','Peak Ground Acceleration');
5
6 TP.PGAlabel = uicontrol('Parent',M.pn4,...
7     'units','normalized',...
8     'FontSize',8,...
9     'HorizontalAlignment','left',...
10    'Style','text',...
11    'Position',[0.10, 0.20, 0.40, 0.5],...
12    'String','Record PGA:');
13
14 TP.PGAedit = uicontrol('Parent',M.pn4,...

```

```

15         'units','normalized',...
16         'Style','edit',...
17         'String',num2str(TP.PGA,'% .3f'),...
18         'Position', [0.40, 0.25, 0.20, 0.5],...
19         'Tooltip','Write Desired Record PGA [ m/s/s ]',...
20         'Callback', @PGACallBack);
21
22 TP.PGAlabel2 = uicontrol('Parent',M.pn4,...
23         'units','normalized',...
24         'FontSize',8,...
25         'HorizontalAlignment','left',...
26         'Style','text',...
27         'Position',[0.65, 0.20, 0.20, 0.5],...
28         'String','[ m/s/s ]');

```

Code 33: *PGApanel.m*: *Peak Ground Acceleration* panel of the *Synthetic Accelerogram* tab

```

1 function PGACallBack(Source,~,~)
2
3     global M GP FP TP ZZ TN GG SP
4
5     inputValue = str2double(get(Source,'String'));
6
7     if isnan(inputValue)
8         set(TP.PGAedit,'String',num2str(TP.PGA,'% .3f'));
9     else
10         M.change = 1;
11
12         if inputValue < 0.001
13             inputValue = 0.001;
14         elseif inputValue > 100
15             inputValue = 100
16         end
17
18         TP.PGA = inputValue;
19
20         set(TP.PGAedit,'String',num2str(TP.PGA,'% .3f'));
21
22         [TP.psd,TP.ff] = pwelch(TP.mqSN*TP.PGA,GP.nfft,GP.noverlap,GP.window,GP.fs);
23
24         set(M.LN5,'XData',GP.tt/60,'YData',TP.mqSN*TP.PGA);
25         set(M.LN5.Parent,'YLim',[-1.1 1.1]*TP.PGA);
26         set(M.LN6,'XData',GP.tt/60,'YData',TP.winN*TP.PGA);
27         set(M.LN7,'XData',GP.tt/60,'YData',envelope(TP.mqSN*TP.PGA,floor(length(
28             ↪ TP.mqSN)/100),'rms'));
29         set(M.LN8,'XData',TP.ff,'YData',db(TP.psd));
30     end
31
32     guidata(Source,M);
33 end

```

Code 34: *PGACallBack.m*: Callback for the editable text box control of the *Peak Ground Acceleration* panel

5.2.7 Synthetic Accelerogram plot Panel

```

1 M.pn5 = uipanel('Parent',M.tb1,...
2         'Units','normalized',...
3         'Position',[0.30 0.10 0.675 0.875]);

```

```

4
5 M.ax1 = subplot(3,2,1,'Parent',M.pn5);
6 M.LN1 = plot(M.ax1,GP.tt/60,GP.NGenAcc);
7 axtoolbar(M.ax1,{'zoomin','zoomout','pan','datacursor','restoreview','export'});
8 grid on;
9 axis('tight');
10 ylim([-1.1 1.1]);
11 xlabel('Time [ min ]','fontsize',8);
12 ylabel('Norm. Acc. [ - ]','fontsize',8);
13 title('White Noise - Accelerogram','fontsize',10);
14
15 M.ax2 = subplot(3,2,2,'Parent',M.pn5);
16 M.LN2 = semilogx(M.ax2,GP.ff,db(GP.psd),'LineWidth',2);
17 axtoolbar(M.ax2,{'zoomin','zoomout','pan','datacursor','restoreview','export'});
18 grid on;
19 axis('tight');
20 ylim([-100 0]);
21 xlabel('Frequency [ Hz ]','fontsize',8);
22 ylabel('PSD [ dB/Hz ]','fontsize',8);
23 title('White Noise - PSD','fontsize',10);
24
25 M.ax3 = subplot(3,2,3,'Parent',M.pn5);
26 M.LN3 = semilogx(M.ax3,FP.w/2/pi,db(squeeze(FP.mag)),'LineWidth',2,'Color',[0.8500 0.3250
↵ 0.0980]);
27 axtoolbar(M.ax3,{'zoomin','zoomout','pan','datacursor','restoreview','export'});
28 grid on;
29 axis('tight');
30 xlabel('Frequency [ Hz ]','fontsize',8);
31 ylabel('Magnitude [ dB ]','fontsize',8);
32 title('Filter Bode - Amplitude','fontsize',10);
33
34 M.ax4 = subplot(3,2,4,'Parent',M.pn5);
35 M.LN4 = semilogx(M.ax4,FP.w/2/pi,squeeze(FP.ph),'LineWidth',2,'Color',[0.8500 0.3250 0.0980]);
36 axtoolbar(M.ax4,{'zoomin','zoomout','pan','datacursor','restoreview','export'});
37 grid on;
38 axis('tight');
39 xlabel('Frequency [ Hz ]','fontsize',8);
40 ylabel('Phase [ Deg ]','fontsize',8);
41 title('Filter Bode - Phase','fontsize',10);
42
43 M.ax5 = subplot(3,2,5,'Parent',M.pn5);
44 M.LN5 = plot(M.ax5,GP.tt/60,TP.mqSN*TP.PGA,'Color',[0.9290 0.6940 0.1250]);
45 hold on;
46 M.LN6 = plot(M.ax5,GP.tt/60,TP.winN*TP.PGA,'LineWidth',2,'Color',[0.4940 0.1840 0.5560]);
47 M.LN7 = plot(M.ax5,GP.tt/60,envelope(TP.mqSN*TP.PGA,floor(length(
↵ TP.mqSN)/100),'rms'),'LineWidth',2,'Color',[0.4660 0.6740
↵ 0.1880]);
48 hold off;
49 axtoolbar(M.ax5,{'zoomin','zoomout','pan','datacursor','restoreview','export'});
50 grid on;
51 axis('tight');
52 ylim([-1.1 1.1]*TP.PGA);
53 xlabel('Time [ min ]','fontsize',8);
54 ylabel('Acceleration [ m/s/s ]','fontsize',8);
55 title('Synthetic Moonquake - Accelerogram','fontsize',10);
56
57 M.ax6 = subplot(3,2,6,'Parent',M.pn5);
58 M.LN8 = semilogx(M.ax6,TP.ff,db(TP.psd),'LineWidth',2,'Color',[0.9290 0.6940 0.1250]);
59 axtoolbar(M.ax6,{'zoomin','zoomout','pan','datacursor','restoreview','export'});
60 grid on;
61 axis('tight')

```

```

62 xlabel('Frequency [ Hz ]','fontsize',8);
63 ylabel('PSD [ dB/Hz ]','fontsize',8);
64 title('Synthetic Moonquake - PSD','fontsize',10);

```

Code 35: *PlotAcc.m* Plot panel of the *Synthetic Accelerogram* tab

5.2.8 Export Synthetic Accelerogram Figures Button

```

1 uicontrol('Parent',M.tb1,...
2         'units','normalized',...
3         'pos',[0.750 0.025 0.100 .050],...
4         'FontSize',8,...
5         'HorizontalAlignment','center',...
6         'Style','Push',...
7         'String','Export Figures', ...
8         'callback',@ExportAccFigCallBack);

```

Code 36: *ExportAccFig.m*: *Export Figures* button of the *Synthetic Accelerogram* tab

```

1 function ExportAccFigCallBack(~,~,~)
2
3 global M GP FP TP ZZ TN GG SP
4
5     outputFileName = 'Figures.fig';
6     defaultFileName = fullfile('*.','outputFileName');
7     [EfileName, EfilePath] = uiputfile(defaultFileName,'Export as figures ''MATLAB .fig''
8     ↪ files');
9
10    if(isunix)
11        symb = '/';
12    else
13        symb = '\';
14    end
15
16    if isequal(EfilePath, 0)
17        msgbox('File save was canceled.');
```

```

17        imagepath = ['Images',symb,'logo_M.gif'];
18        jframe = get(gcf,'javaframe');
19        Icon = javax.swing.ImageIcon(imagepath);
20        jframe.setFigureIcon(Icon);
21    else
22        set(M.fg, 'pointer', 'watch')
23        drawnow;
24
25        handles.f = M.fg;
26        handles.axes = M.ax1;
27        fignew = figure('Visible','off');
28        newAxes = copyobj(handles.axes,fignew);
29        set(newAxes,'Position',get(groot,'DefaultAxesPosition'));
30        set(fignew,'CreateFcn','set(gcf,'Visible','on')));
31        EsaveFilePath = fullfile(EfilePath,'White Noise - Accelerogram.fig');
32        savefig(fignew,ESaveFilePath);
33        delete(fignew);
34
35        handles.f = M.fg;
36        handles.axes = M.ax2;
37        fignew = figure('Visible','off');
38        newAxes = copyobj(handles.axes,fignew);
39        set(newAxes,'Position',get(groot,'DefaultAxesPosition'));

```

```

40     set(fignew,'CreateFcn','set(gcf,'Visible','on')));
41     EsaveFilePath = fullfile(EfilePath,'White Noise - PSD.fig');
42     savefig(fignew,EsaveFilePath);
43     delete(fignew);
44
45     handles.f = M.fig;
46     handles.axes = M.ax3;
47     fignew = figure('Visible','off');
48     newAxes = copyobj(handles.axes,fignew);
49     set(newAxes,'Position',get(groot,'DefaultAxesPosition'));
50     set(fignew,'CreateFcn','set(gcf,'Visible','on')));
51     EsaveFilePath = fullfile(EfilePath,'Filter Bode - Amplitude.fig');
52     savefig(fignew,EsaveFilePath);
53     delete(fignew);
54
55     handles.f = M.fig;
56     handles.axes = M.ax4;
57     fignew = figure('Visible','off');
58     newAxes = copyobj(handles.axes,fignew);
59     set(newAxes,'Position',get(groot,'DefaultAxesPosition'));
60     set(fignew,'CreateFcn','set(gcf,'Visible','on')));
61     EsaveFilePath = fullfile(EfilePath,'Filter Bode - Phase.fig');
62     savefig(fignew,EsaveFilePath);
63     delete(fignew);
64
65     handles.f = M.fig;
66     handles.axes = M.ax5;
67     fignew = figure('Visible','off');
68     newAxes = copyobj(handles.axes,fignew);
69     set(newAxes,'Position',get(groot,'DefaultAxesPosition'));
70     set(fignew,'CreateFcn','set(gcf,'Visible','on')));
71     EsaveFilePath = fullfile(EfilePath,'Synthetic Moonquake - Accelerogram.fig');
72     savefig(fignew,EsaveFilePath);
73     delete(fignew);
74
75     handles.f = M.fig;
76     handles.axes = M.ax6;
77     fignew = figure('Visible','off');
78     newAxes = copyobj(handles.axes,fignew);
79     set(newAxes,'Position',get(groot,'DefaultAxesPosition'));
80     set(fignew,'CreateFcn','set(gcf,'Visible','on')));
81     EsaveFilePath = fullfile(EfilePath,'Synthetic Moonquake - PSD.fig');
82     savefig(fignew,EsaveFilePath);
83     delete(fignew);
84
85     set(M.fig, 'pointer', 'arrow');
86
87     msgbox(['Figures Exported to: ',EfilePath]);
88     imagepath = ['Images',symb,'logo_M.gif'];
89     jframe = get(gcf,'javaframe');
90     Icon = javax.swing.ImageIcon(imagepath);
91     jframe.setFigureIcon(Icon);
92 end
93
94 end

```

Code 37: *ExportAccFigCallback.m*: Callback for the *Export Figures* button of the *Synthetic Accelerogram* tab

5.2.9 Export Synthetic Accelerogram Button

```
1 uicontrol('Parent',M.tb1,...
2         'units','normalized',...
3         'pos',[0.875 0.025 0.100 .050],...
4         'FontSize',8,...
5         'HorizontalAlignment','center',...
6         'Style','Push',...
7         'String','Export Accelerogram', ...
8         'callback',@ExportAccCallBack);
```

Code 38: *ExportAcc.m*: *Export Accelerogram* button of the *Synthetic Accelerogram* tab

```
1 function ExportAccCallBack(~,~,~)
2
3     global M GP FP TP ZZ TN GG SP
4
5     [EfileName, EfilePath] = uiputfile('*.txt', 'Export as '*.txt' file');
6
7     if(isunix)
8         symb = '/';
9     else
10        symb = '\';
11    end
12
13    if isequal(EfileName, 0) || isequal(EfilePath, 0)
14        msgbox('File save was canceled. ');
15        imagepath = ['Images',symb,'logo_M.gif'];
16        jframe = get(gcf,'javaframe');
17        Icon = javax.swing.ImageIcon(imagepath);
18        jframe.setFigureIcon(Icon);
19    else
20        set(M.fg, 'pointer', 'watch')
21        drawnow;
22
23        EsaveFilePath = fullfile(EfilePath, EfileName);
24        fid = fopen(EsaveFilePath,'wt');
25        fprintf(fid,'%s\n',['% Date: ' datestr(now, 'dd/mmmmm/yyyy-HH:MM:SS')]);
26        fprintf(fid,'%s\n','%Accelerogram records');
27        fprintf(fid,'%s\n',['% File: ' EfileName]);
28        fprintf(fid,'%s\n',['% Durantion: ' num2str(GP.T,'% .3f') ' min']);
29        fprintf(fid,'%s\n',['% Sampling Frequency: ' num2str(GP.fs,'% .3f') ' Hz']);
30        fprintf(fid,'%s\n',['% PGA: ' num2str(TP.PGA,'% .3f') ' m/s/s']);
31        fprintf(fid,'%s\n','% First Column: Time (s)');
32        fprintf(fid,'%s\n','% Second Column: Synthetic Moonquake Acc. (m/s/s)');
33        fprintf(fid,'%s\n','% Matlab Loading Method:');
34        fprintf(fid,'%s\n',['% fileID = fopen('' EfileName '');']);
35        fprintf(fid,'%s\n','% Data = cell2mat(textscan(fileID,'%f
36        ↵ %f','headerlines',14));');
37        fprintf(fid,'%s\n','% fclose(fileID);');
38        fprintf(fid,'\n');
39        A = [GP.tt TP.mqSN*TP.PGA];
40        for ii = 1:size(A,1)
41            fprintf(fid,'%6.6f\t%.6e',A(ii,:));
42            fprintf(fid,'\n');
43        end
44        fclose(fid);
45
46        set(M.fg, 'pointer', 'arrow');
47
```

```

48     msgbox(['Synthetic Moonquake Acc. Exported to: ', EsaveFilePath]);
49     imagepath = ['Images',symb,'logo_M.gif'];
50     jframe = get(gcf,'javaframe');
51     Icon = javax.swing.ImageIcon(imagepath);
52     jframe.setFigureIcon(Icon);
53 end
54
55 end

```

Code 39: *ExportAccCallBack.m*: Callback for the *Export Accelerogram* button of the *Synthetic Accelerogram* tab

5.2.10 Damping Ratio Panel

```

1  M.pn6 = uipanel('Parent',M.tb2,...
2      'Units','normalized',...
3      'Position',[0.025 0.9105 0.250 0.070],...
4      'Title','Damping Ratio');
5
6  ZZ.Zlabel = uicontrol('Parent',M.pn6,...
7      'Units','normalized',...
8      'FontSize',8,...
9      'HorizontalAlignment','left',...
10     'Style','text',...
11     'Position',[0.075 0.225*10/7 0.950 0.25*10/7],...
12     'String','Damping Ratio:');
13
14  ZZ.Zedit = uicontrol('Parent',M.pn6,...
15      'style','edit',...
16      'FontSize',8,...
17      'units','normalized',...
18      'position',[0.425 0.225*10/7 0.200 1.10*0.25*10/7],...
19      'string',num2str(ZZ.Z,'%3f'),...
20      'Tooltip','Write Desired Damping Ratio [ - ]',...
21      'callback',{@ZZCallBack,'Z'});
22
23  ZZ.Zlabel2 = uicontrol('Parent',M.pn6,...
24      'Units','normalized',...
25      'FontSize',8,...
26      'HorizontalAlignment','left',...
27      'Style','text',...
28      'Position',[0.650 0.225*10/7 0.950 0.25*10/7],...
29      'String','[ % ]');

```

Code 40: *ZZpanel.m*: *Damping Ratio* panel of the *Response Spectrum* tab

```

1  function ZZCallBack(Source,~,Param)
2
3      global M GP FP TP ZZ TN GG SP
4
5      if isnan(str2double(get(Source,'String')))
6          set(Source,'string',num2str(ZZ.(Param),'%.3f'));
7      else
8
9          ZZ.(Param) = str2double(get(Source,'String'));
10
11         if strcmp(Param,'Z')
12             if ZZ.Z < 0
13                 ZZ.Z = 0;

```

```

14         elseif ZZ.Z > 99.999
15             ZZ.Z = 99.999;
16         end
17     end
18
19     set(ZZ.Zedit,'string',num2str(ZZ.Z, '%.3f'));
20
21     SpecMQ
22
23     set(M.LN9,'XData',TN.Tn,'YData',SP.ResD);
24     set(M.LN10,'XData',TN.Tn,'YData',SP.ResV);
25     set(M.LN11,'XData',TN.Tn,'YData',SP.ResA);
26     set(M.LN12,'XData',TN.Tn,'YData',SP.ResSV);
27     set(M.LN13,'XData',TN.Tn,'YData',SP.ResSA);
28
29 end
30
31 guidata(Source,M);
32 end

```

Code 41: *ZZCallback.m*: Callback for the editable text box control of the *Damping Ratio* panel

5.2.11 Response Spectrum Periods Panel

```

1 M.pn8 = uipanel('Parent',M.tb2,...
2             'Units','normalized',...
3             'Position',[0.025 0.7550 0.25 0.15],...
4             'Title','Response Spectrum Periods');
5
6 TN.Tminlabel = uicontrol('Parent',M.pn8,...
7             'Units','normalized',...
8             'FontSize',8,...
9             'HorizontalAlignment','left',...
10            'Style','text',...
11            'Position',[0.075 0.225*2/3+0.600 0.95 1/6],...
12            'String','Minimum Period:');
13
14 TN.Tminedit = uicontrol('Parent',M.pn8,...
15            'style','edit',...
16            'FontSize',8,...
17            'units','normalized',...
18            'position',[0.425 0.225*2/3+0.600 0.20 1/6],...
19            'string',num2str(TN.Tmin, '%.3f'),...
20            'Tooltip','Write Desired Minimum Response Spectrum Period [ s ]',...
21            'callback',{@TNCallBack,'Tmin'});
22
23 TN.Tminlabel2 = uicontrol('Parent',M.pn8,...
24            'Units','normalized',...
25            'FontSize',8,...
26            'HorizontalAlignment','left',...
27            'Style','text',...
28            'Position',[0.65 0.225*2/3+0.600 0.95 1/6],...
29            'String','[ s ]');
30
31 TN.Tmaxlabel = uicontrol('Parent',M.pn8,...
32            'Units','normalized',...
33            'FontSize',8,...
34            'HorizontalAlignment','left',...
35            'Style','text',...
36            'Position',[0.075 0.225*2/3+0.300 0.95 1/6],...

```

```

37         'String','Maximum Period:');
38
39 TN.Tmaxedit = uicontrol('Parent',M.pn8,...
40         'style','edit',...
41         'FontSize',8,...
42         'units','normalized',...
43         'position',[0.425 0.225*2/3+0.300 0.20 1/6],...
44         'string',num2str(TN.Tmax,'%3f'),...
45         'Tooltip','Write Desired Maximum Response Spectrum Period [ s ]',...
46         'callback',{@TNCallBack,'Tmax'});
47
48 TN.Tmaxlabel2 = uicontrol('Parent',M.pn8,...
49         'Units','normalized',...
50         'FontSize',8,...
51         'HorizontalAlignment','left',...
52         'Style','text',...
53         'Position',[0.65 0.225*2/3+0.300 0.95 1/6],...
54         'String','[ s ]');
55
56 TN.Tnumlabel = uicontrol('Parent',M.pn8,...
57         'Units','normalized',...
58         'FontSize',8,...
59         'HorizontalAlignment','left',...
60         'Style','text',...
61         'Position',[0.075 0.225*2/3 0.95 1/6],...
62         'String','Number of Periods:');
63
64 TN.Tnumedit = uicontrol('Parent',M.pn8,...
65         'style','edit',...
66         'FontSize',8,...
67         'units','normalized',...
68         'position',[0.425 0.225*2/3 0.20 1/6],...
69         'string',num2str(TN.Tnum,'%0f'),...
70         'Tooltip','Write Desired Response Spectrum Number of Periods [ - ]',...
71         'callback',{@TNCallBack,'Tnum'});
72
73 TN.Tnumlabel2 = uicontrol('Parent',M.pn8,...
74         'Units','normalized',...
75         'FontSize',8,...
76         'HorizontalAlignment','left',...
77         'Style','text',...
78         'Position',[0.65 0.225*2/3 0.95 1/6],...
79         'String','[ - ]');

```

Code 42: *TNpanel.m*: Response Spectrum Periods panel of the *Response Spectrum* tab

```

1 function TNCallBack(Source,~,Param)
2
3     global M GP FP TP ZZ TN GG SP
4
5     if isnan(str2double(get(Source,'String')))
6         if strcmp(Param,'Tmin') || strcmp(Param,'Tmax')
7             set(Source,'string',num2str(TN.(Param),'%3f'));
8         elseif strcmp(Param,'Tnum')
9             set(Source,'string',num2str(TN.Tnum,'%0f'));
10        end
11    else
12        TN.(Param) = str2double(get(Source,'String'));
13
14        if strcmp(Param,'Tmin')
15            if TN.Tmin < 0.001

```

```

16         TN.Tmin = 0.001;
17     elseif TN.Tmin > 99.999
18         TN.Tmin = 99.999;
19     end
20 end
21
22 if strcmp(Param,'Tmax')
23     if TN.Tmax < 0.002
24         TN.Tmax = 0.002;
25     elseif TN.Tmax > 100
26         TN.Tmax = 100;
27     end
28 end
29
30 if (TN.Tmin >= TN.Tmax)
31     TN.Tmin = 0.01;
32     TN.Tmax = 10;
33 end
34
35 if strcmp(Param,'Tnum')
36     if TN.Tnum < 10
37         TN.Tnum = 10;
38     elseif TN.Tnum >= 10000
39         TN.Tnum = 10000;
40     else
41         TN.Tnum = ceil(TN.Tnum);
42     end
43 end
44
45 TN.Tn=logspace(log10(TN.Tmin),log10(TN.Tmax),TN.Tnum);
46
47 set(TN.Tminedit,'string',num2str(TN.Tmin,'%.3f'));
48 set(TN.Tmaxedit,'string',num2str(TN.Tmax,'%.3f'));
49 set(TN.Tnumedit,'string',num2str(TN.Tnum,'%.0f'));
50
51 SpecMQ
52
53 set(M.LN9,'XData',TN.Tn,'YData',SP.ResD);
54 set(M.LN10,'XData',TN.Tn,'YData',SP.ResV);
55 set(M.LN11,'XData',TN.Tn,'YData',SP.ResA);
56 set(M.LN12,'XData',TN.Tn,'YData',SP.ResSV);
57 set(M.LN13,'XData',TN.Tn,'YData',SP.ResSA);
58
59 end
60
61 guidata(Source,M);
62 end

```

Code 43: *TNCallback.m*: Callback for the editable text box controls of the *Response Spectrum Periods* panel

5.2.12 Gravitational Acceleration Magnitude Panel

```

1 M.pn9 = uipanel('Parent',M.tb2,...
2     'Units','normalized',...
3     'Position',[0.025 0.6795 0.250 0.070],...
4     'Title','Gravitational Acceleration Magnitude');
5
6 GG.gglabel = uicontrol('Parent',M.pn9,...
7     'Units','normalized',...

```

```

8         'FontSize',8,...
9         'HorizontalAlignment','left',...
10        'Style','text',...
11        'Position',[0.075 0.225*10/7 0.950 0.25*10/7],...
12        'String','Gravitational Acc.:');
13
14 GG.ggedit = uicontrol('Parent',M.pn9,...
15         'style','edit',...
16         'FontSize',8,...
17         'units','normalized',...
18         'position',[0.425 0.225*10/7 0.200 1.10*0.25*10/7],...
19         'string',num2str(GG.gg,'%.3f'),...
20         'Tooltip','Write Desired Gravitational Acceleration [ m/s/s ]',...
21         'callback',{@GGCallBack,'gg'});
22
23 GG.gglabel2 = uicontrol('Parent',M.pn9,...
24         'Units','normalized',...
25         'FontSize',8,...
26         'HorizontalAlignment','left',...
27         'Style','text',...
28         'Position',[0.650 0.225*10/7 0.950 0.25*10/7],...
29         'String','[ m/s/s ]');

```

Code 44: *GGpanel.m*: Gravitational Acceleration Magnitude panel of the *Response Spectrum* tab

```

1 function GGCallBack(Source,~,Param)
2
3     global M GP FP TP ZZ TN GG SP
4
5     if isnan(str2double(get(Source,'String')))
6         set(Source,'string',num2str(GG.(Param),'%.3f'));
7     else
8         GG.(Param) = str2double(get(Source,'String'));
9
10        if strcmp(Param,'gg')
11            if GG.gg < 0.001
12                GG.gg = 0.001;
13            end
14        end
15
16        set(GG.ggedit,'string',num2str(GG.gg,'%.3f'));
17
18        SpecMQ
19
20        set(M.LN9,'XData',TN.Tn,'YData',SP.ResD);
21        set(M.LN10,'XData',TN.Tn,'YData',SP.ResV);
22        set(M.LN11,'XData',TN.Tn,'YData',SP.ResA);
23        set(M.LN12,'XData',TN.Tn,'YData',SP.ResSV);
24        set(M.LN13,'XData',TN.Tn,'YData',SP.ResSA);
25    end
26
27    guidata(Source,M);
28 end

```

Code 45: *GGCallBack.m*: Callback for the editable text box controls of the *Gravitational Acceleration Magnitude* panel

5.2.13 *SpecMQ* Function for the calculus or the seismic response spectra

```

1 function SpecMQ()
2
3 global M GP FP TP ZZ TN GG SP
4
5     set(M.fg, 'pointer', 'watch')
6     drawnow;
7
8     SP.ResD=zeros;
9     SP.ResV=zeros;
10    SP.ResA=zeros;
11    SP.ResSV=zeros;
12    SP.ResSA=zeros;
13    for ii=1:length(TN.Tn)
14        SP.tn=TN.Tn(ii);
15        SP.wn=2*pi/SP.tn;
16        SP.c=2*ZZ.Z/100*SP.wn;
17        SP.k=SP.wn^2;
18        [SP.u,SP.v,SP.a]=interpolation(1,SP.c,SP.k,-TP.mqSN*TP.PGA/GG.gg,0,0,1/GP.fs,length(
        ↪ TP.mqSN));
19        SP.D=max(abs(SP.u));
20        SP.V=max(abs(SP.v));
21        SP.A=max(abs(SP.a));
22        SP.SV=SP.wn*SP.D;
23        SP.SA=SP.wn*SP.wn*SP.D;
24        SP.ResD(ii)=SP.D;
25        SP.ResV(ii)=SP.V;
26        SP.ResA(ii)=SP.A;
27        SP.ResSV(ii)=SP.SV;
28        SP.ResSA(ii)=SP.SA;
29    end
30
31    set(M.fg, 'pointer', 'arrow');
32
33    function [u,v,a]=interpolation(m,c,k,pt,uo,vo,dt,N)
34        u=zeros;
35        v=zeros;
36        a=zeros;
37        wn=sqrt(k/m);
38        zita= c/(2*m*wn);
39        wd=wn*sqrt(1- zita^2);
40        p=pt;
41        u(1)=uo;
42        v(1)=vo;
43        a(1)=(p(1)/m)-(2*zita*wn*v(1))-(wn^2)*u(1);
44        A=exp(-zita*wn*dt)*(((zita*sin(wd*dt))/sqrt(1-(zita)^2))+cos(wd*dt));
45        B=exp(-zita*wn*dt)*(sin(wd*dt)/wd);
46        C=(1/k)* (((2*zita)/ (wn*dt)) + (exp(-zita*wn*dt)*((((1- 2*(zita^2))/(wn*dt)) -
        ↪ (zita/sqrt(1 -(zita^2))))*sin(wd*dt)) -((1 +((2*zita)/(wn*dt))*cos(wd*dt)))));
47        D=(1/k)* ( (1-(2*zita/(wn*dt))) +
        ↪ (exp(-zita*wn*dt)*((-1+2*(zita^2))/(wn*dt))*sin(wd*dt)
        ↪ +(2*zita/(wn*dt))*cos(wd*dt))));
48        Apr=-exp(-zita*wn*dt)*(wn*sin(wd*dt)/sqrt(1 - zita^2));
49        Bpr=exp(-zita*wn*dt)*(-zita*sin(wd*dt)/sqrt(1 - zita^2) + cos(wd*dt));
50        Cpr=(1/k)* ((-1/ dt) + exp(-zita*wn*dt)*(((wn/sqrt(1 - zita^2)) + zita/(dt*sqrt(1 -
        ↪ zita^2)))*sin(wd*dt) +(1/dt)*cos(wd*dt)));
51        Dpr=(1/(k*dt))*(1-exp(-zita*wn*dt)*((zita/sqrt(1 - zita^2))*sin(wd*dt) +cos(wd*dt)));
52        for i=1:N-1
53            u(i+1)= A*u(i)+B*v(i)+C*p(i)+D*p(i+1);
54            v(i+1)= Apr*u(i)+Bpr*v(i)+Cpr*p(i)+Dpr*p(i+1);

```

```

55         a(i+1)=(p(i+1)/m)-2*zita*wn*v(i+1)-(wn^2)*u(i+1);
56     end
57 end
58 end

```

Code 46: *SpecMQ.m*: Function for the calculus or the seismic response spectra

5.2.14 Response Spectrum plot Panel

```

1  M.pn10 = uipanel('Parent',M.tb2,...
2          'Units','normalized',...
3          'Position',[0.30 0.10 0.675 0.875]);
4
5  M.ax7 = subplot(3,2,1,'Parent',M.pn10);
6  M.LN9 = plot(M.ax7,TN.Tn,SP.ResD,'LineWidth',2);
7  axtoolbar(M.ax7,{'zoomin','zoomout','pan','datacursor','restoreview','export'});
8  grid on;
9  axis('tight');
10 xlabel('Period [ s ]','fontsize',8);
11 ylabel('Displacement [ g*s*s ]','fontsize',8);
12 title('Displacement - Response Spectrum','fontsize',10);
13
14 M.ax8 = subplot(3,2,2,'Parent',M.pn10);
15 M.LN10 = plot(M.ax8,TN.Tn,SP.ResV,'LineWidth',2,'Color',[0.8500 0.3250 0.0980]);
16 axtoolbar(M.ax8,{'zoomin','zoomout','pan','datacursor','restoreview','export'});
17 grid on;
18 axis('tight');
19 xlabel('Period [ s ]','fontsize',8);
20 ylabel('Velocity [ g*s ]','fontsize',8);
21 title('Velocity - Response Spectrum','fontsize',10);
22
23 M.ax9 = subplot(3,2,3,'Parent',M.pn10);
24 M.LN11 = plot(M.ax9,TN.Tn,SP.ResA,'LineWidth',2,'Color',[0.9290 0.6940 0.1250]);
25 axtoolbar(M.ax9,{'zoomin','zoomout','pan','datacursor','restoreview','export'});
26 grid on;
27 axis('tight');
28 xlabel('Period [ s ]','fontsize',8);
29 ylabel('Acceleration [ g ]','fontsize',8);
30 title('Acceleration - Response Spectrum','fontsize',10);
31
32 M.ax10 = subplot(3,2,4,'Parent',M.pn10);
33 M.LN12 = plot(M.ax10,TN.Tn,SP.ResSV,'LineWidth',2,'Color',[0.4940 0.1840 0.5560]);
34 axtoolbar(M.ax10,{'zoomin','zoomout','pan','datacursor','restoreview','export'});
35 grid on;
36 axis('tight');
37 xlabel('Period [ s ]','fontsize',8);
38 ylabel('Pseudo-Velocity [ g*s ]','fontsize',8);
39 title('Pseudo-Velocity - Response Spectrum','fontsize',10);
40
41 M.ax11 = subplot(3,2,[5 6],'Parent',M.pn10);
42 M.LN13 = plot(M.ax11,TN.Tn,SP.ResSA,'LineWidth',2,'Color',[0.4660 0.6740 0.1880]);
43 axtoolbar(M.ax11,{'zoomin','zoomout','pan','datacursor','restoreview','export'});
44 grid on;
45 axis('tight');
46 xlabel('Period [ s ]','fontsize',8);
47 ylabel('Pseudo-Acceleration [ g ]','fontsize',8);
48 title('Pseudo-Acceleration - Response Spectrum','fontsize',10);

```

Code 47: *PlotSpec.m*: Plot panel of the *Response Spectrum* tab

5.2.15 Export Response Spectrum Figures Button

```
1 uicontrol('Parent',M.tb2,...
2         'units','normalized',...
3         'pos',[0.750 0.025 0.100 .050],...
4         'FontSize',8,...
5         'HorizontalAlignment','center',...
6         'Style','Push',...
7         'String','Export Figures', ...
8         'callback',@ExportSpecFigCallBack);
```

Code 48: *ExportSpecFig.m*: *Export Figures* button of the *Response Spectrum* tab

```
1 function ExportSpecFigCallBack(~,~,~)
2
3 global M GP FP TP ZZ TN GG SP
4
5     outputFileName = 'Figures.fig';
6     defaultFileName = fullfile('*.fig',outputFileName);
7     [EfileName, EfilePath] = uiputfile(defaultFileName,'Export as figures 'MATLAB .fig'
8     ↪ files');
9
10    if(isunix)
11        symb = '/';
12    else
13        symb = '\';
14    end
15
16    if isequal(EfilePath, 0)
17        msgbox('File save was canceled.');
```

```
17     imagepath = ['Images',symb,'logo_M.gif'];
18     jframe = get(gcf,'javaframe');
19     Icon = javax.swing.ImageIcon(imagepath);
20     jframe.setFigureIcon(Icon);
21 else
22     set(M.fg, 'pointer', 'watch')
23     drawnow;
24
25     handles.f = M.fg;
26     handles.axes = M.ax7;
27     fignew = figure('Visible','off');
28     newAxes = copyobj(handles.axes,fignew);
29     set(newAxes,'Position',get(groot,'DefaultAxesPosition'));
30     set(fignew,'CreateFcn','set(gcf,'Visible','on')));
31     EsaveFilePath = fullfile(EfilePath,'Displacement - Response Spectrum.fig');
32     savefig(fignew,EsaveFilePath);
33     delete(fignew);
34
35     handles.f = M.fg;
36     handles.axes = M.ax8;
37     fignew = figure('Visible','off');
38     newAxes = copyobj(handles.axes,fignew);
39     set(newAxes,'Position',get(groot,'DefaultAxesPosition'));
40     set(fignew,'CreateFcn','set(gcf,'Visible','on')));
41     EsaveFilePath = fullfile(EfilePath,'Velocity - Response Spectrum.fig');
42     savefig(fignew,EsaveFilePath);
43     delete(fignew);
44
45     handles.f = M.fg;
46     handles.axes = M.ax9;
47     fignew = figure('Visible','off');
```

```

48     newAxes = copyobj(handles.axes,fignew);
49     set(newAxes,'Position',get(groot,'DefaultAxesPosition'));
50     set(fignew,'CreateFcn','set(gcf,'Visible','on')));
51     EsaveFilePath = fullfile(EfilePath,'Acceleration - Response Spectrum.fig');
52     savefig(fignew,EsaveFilePath);
53     delete(fignew);
54
55     handles.f = M.fg;
56     handles.axes = M.ax10;
57     fignew = figure('Visible','off');
58     newAxes = copyobj(handles.axes,fignew);
59     set(newAxes,'Position',get(groot,'DefaultAxesPosition'));
60     set(fignew,'CreateFcn','set(gcf,'Visible','on')));
61     EsaveFilePath = fullfile(EfilePath,'Pseudo-Velocity - Response Spectrum.fig');
62     savefig(fignew,EsaveFilePath);
63     delete(fignew);
64
65     handles.f = M.fg;
66     handles.axes = M.ax11;
67     fignew = figure('Visible','off');
68     newAxes = copyobj(handles.axes,fignew);
69     set(newAxes,'Position',get(groot,'DefaultAxesPosition'));
70     set(fignew,'CreateFcn','set(gcf,'Visible','on')));
71     EsaveFilePath = fullfile(EfilePath,'Pseudo-Acceleration - Response Spectrum.fig');
72     savefig(fignew,EsaveFilePath);
73     delete(fignew);
74
75     set(M.fg, 'pointer', 'arrow');
76
77     msgbox(['Figures Exported to: ',EfilePath]);
78     imagepath = ['Images',symb,'logo_M.gif'];
79     jframe = get(gcf,'javaframe');
80     Icon = javax.swing.ImageIcon(imagepath);
81     jframe.setFigureIcon(Icon);
82 end
83
84 end

```

Code 49: *ExportSpecFigCallBack.m*: Callback for the *Export Figures* button of the *Response Spectrum* tab

5.2.16 Export Response Spectrum Button

```

1 uicontrol('Parent',M.tb2,...
2         'units','normalized',...
3         'pos',[0.875 0.025 0.100 .050],...
4         'FontSize',8,...
5         'HorizontalAlignment','center',...
6         'Style','Push',...
7         'String','Export Spectra', ...
8         'callback',@ExportSpecCallBack);

```

Code 50: *ExportSpec.m*: *Export Spectra* button of the *Response Spectrum* tab

```

1 function ExportSpecCallBack(~,~,~)
2
3     global M GP FP TP ZZ TN GG SP
4
5     [EfileName, EfilePath] = uiputfile('*.txt', 'Export as '*.txt' file');

```

```

6
7     if(isunix)
8         symb = '/';
9     else
10        symb = '\\';
11    end
12
13    if isequal(EfileName, 0) || isequal(EfilePath, 0)
14        msgbox('File save was canceled.');
```

```

15        imagepath = ['Images',symb,'logo_M.gif'];
16        jframe = get(gcf,'javaframe');
17        Icon = javax.swing.ImageIcon(imagepath);
18        jframe.setFigureIcon(Icon);
19    else
20        set(M.fg, 'pointer', 'watch')
21        drawnow;
22
23        EsaveFilePath = fullfile(EfilePath, EfileName);
24        fid = fopen(EsaveFilePath,'wt');
```

```

25        fprintf(fid,'%s\n',['% Date: ' datestr(now, 'dd/mmmmm/yyyy-HH:MM:SS')]);
26        fprintf(fid,'%s\n','% Synthetic Moonquake Response Spectrum records');
27        fprintf(fid,'%s\n','% File: ' EfileName]);
28        fprintf(fid,'%s\n',['% Structural Damping Ratio: ' num2str(ZZ.Z,'%.3f') ' %']);
29        fprintf(fid,'%s\n',['% Minimum Period: ' num2str(TN.Tmin,'%.3f') ' s']);
30        fprintf(fid,'%s\n',['% Maximum Period: ' num2str(TN.Tmax,'%.3f') ' s']);
31        fprintf(fid,'%s\n',['% Number of Periods: ' num2str(TN.Tnum,'%.0f') ' -']);
32        fprintf(fid,'%s\n',['% Gravitational Acceleration: ' num2str(GG.gg,'%.3f') ' m/s/s']);
33        fprintf(fid,'%s\n','% First Column: Period (s)');
34        fprintf(fid,'%s\n','% Second Column: Displacement (g*s*s)');
35        fprintf(fid,'%s\n','% Third Column: Velocity (g*s)');
36        fprintf(fid,'%s\n','% Fourth Column: Acceleration (g)');
37        fprintf(fid,'%s\n','% Fifth Column: Pseudo-Velocity (g*s)');
38        fprintf(fid,'%s\n','% Sixth Column: Pseudo-Acceleration (g)');
39        fprintf(fid,'%s\n','% Matlab Loading Method:');
40        fprintf(fid,'%s\n',['% fileID = fopen('' EfileName '');']);
41        fprintf(fid,'%s\n','% Data = cell2mat(textscan(fileID,'%f
↳ %f','headerlines',20));');
42        fprintf(fid,'%s\n','% fclose(fileID);');
43        fprintf(fid,'\n');
44        fprintf(fid,'\n');
```

```

45        A = [TN.Tn(:) SP.ResD(:) SP.ResV(:) SP.ResA(:) SP.ResSV(:) SP.ResSA(:)];
46        for ii = 1:size(A,1)
47            fprintf(fid,'%6.6f\t%.6e\t%.6e\t%.6e\t%.6e\t%.6e',A(ii,:));
48            fprintf(fid,'\n');
```

```

49        end
50        fclose(fid);
51
52        set(M.fg, 'pointer', 'arrow');
```

```

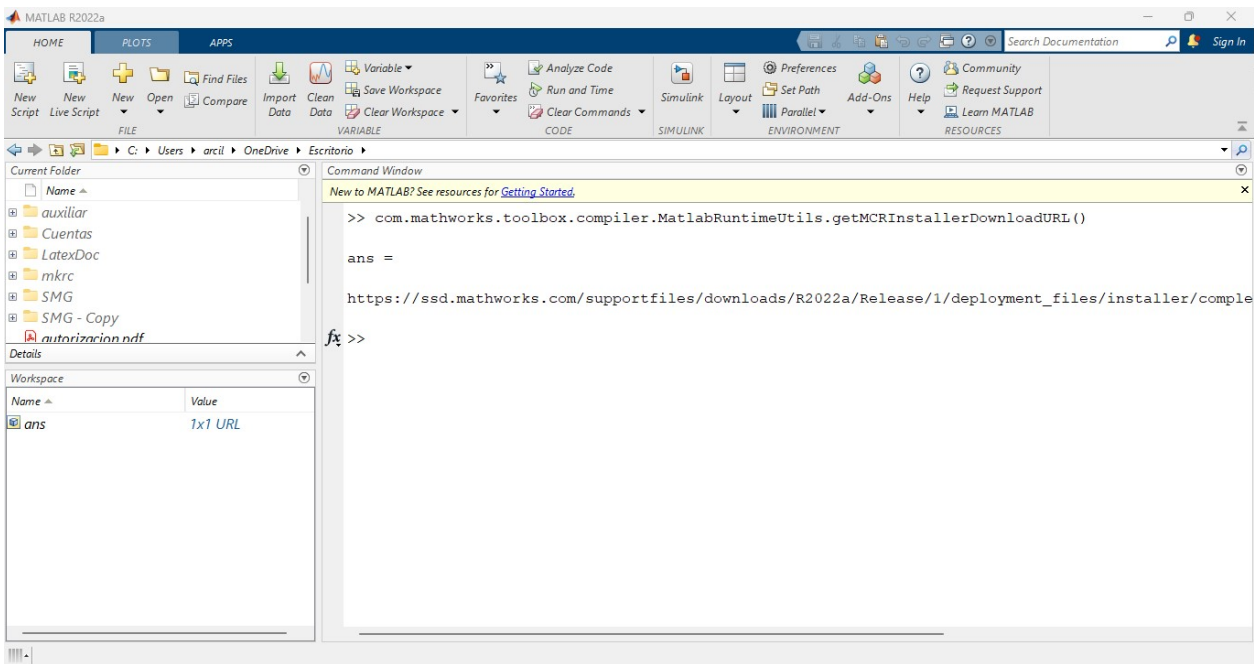
53
54        msgbox(['Response Spectra Exported to: ', EsaveFilePath]);
55        imagepath = ['Images',symb,'logo_M.gif'];
56        jframe = get(gcf,'javaframe');
57        Icon = javax.swing.ImageIcon(imagepath);
58        jframe.setFigureIcon(Icon);
59    end
60
61 end

```

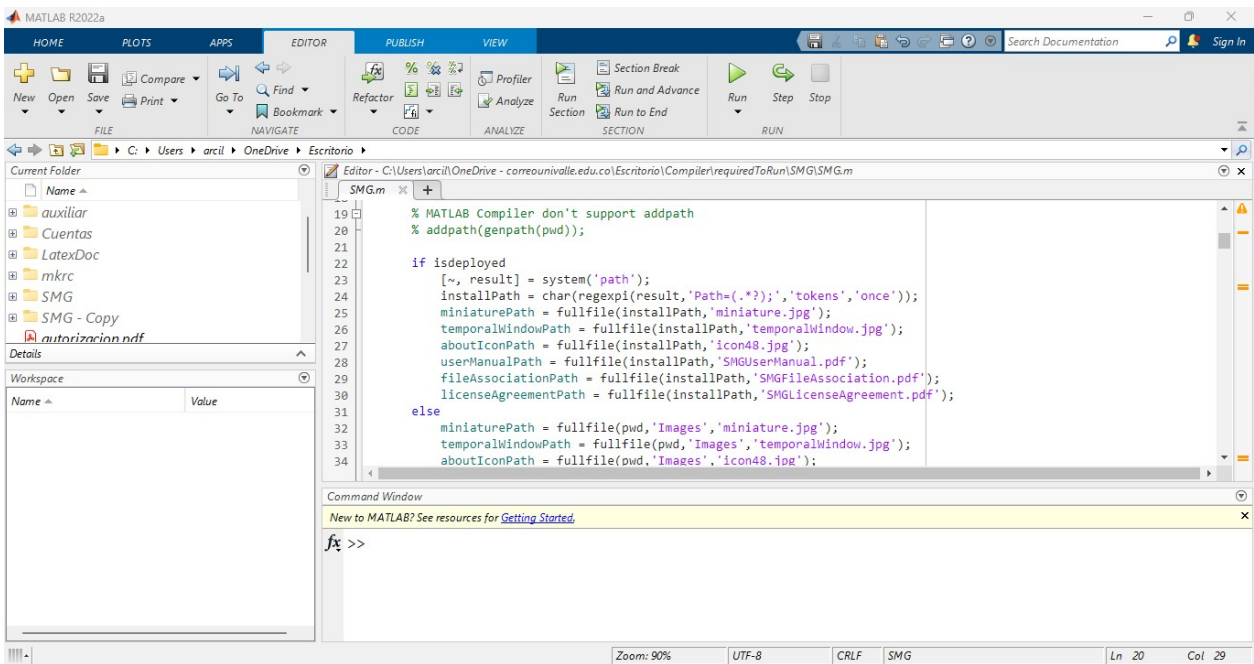
Code 51: *ExportSpecCallback.m*: Callback for the *Export Spectra* button of the *Response Spectrum* tab

6 Compilation

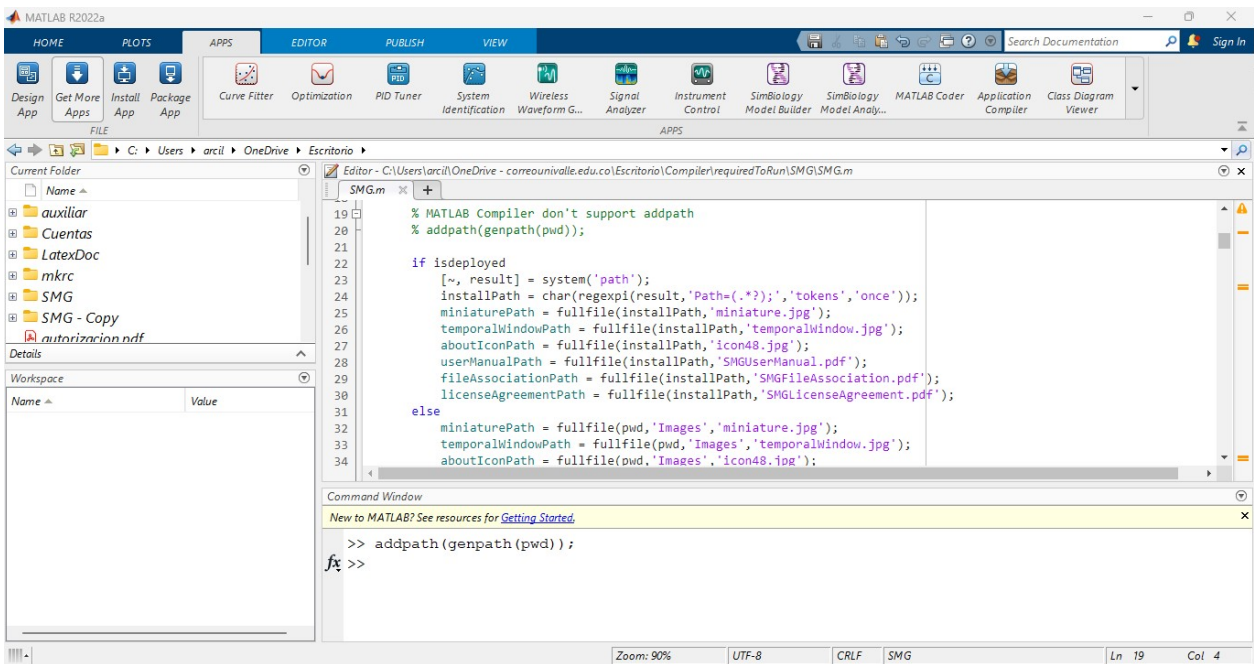
- Open MATLAB and execute the command `com.mathworks.toolbox.compiler.MatlabRuntimeUtils.getMCRInstallerDownloadURL()` to obtain the link to download the correct version of MATLAB RunTime.



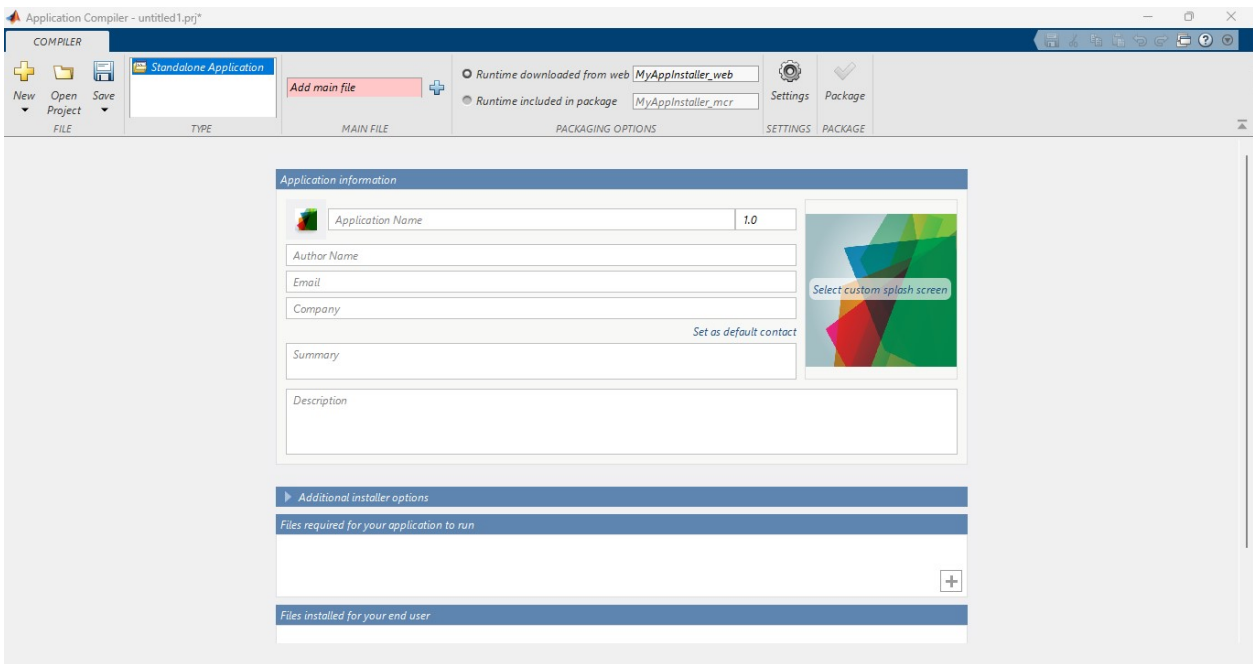
- Open `SMG.exe` and comment out the line `addpath(genpath(pwd));`.



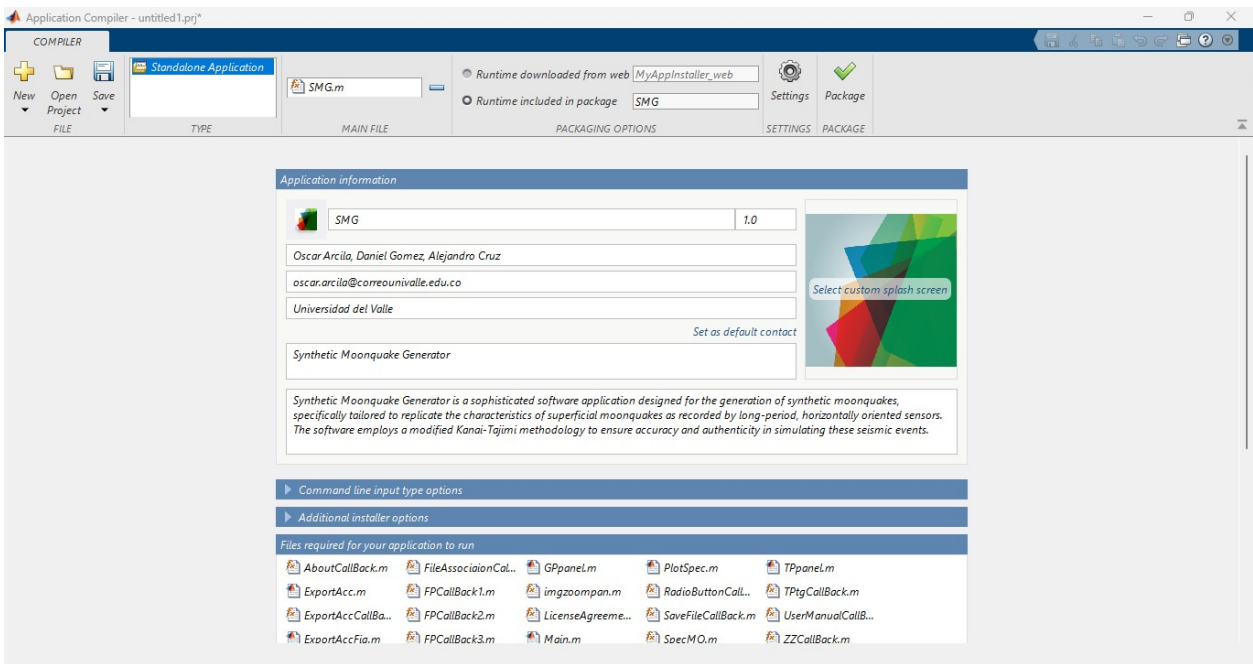
- Execute the command `addpath(genpath(pwd));` on the interpreter.



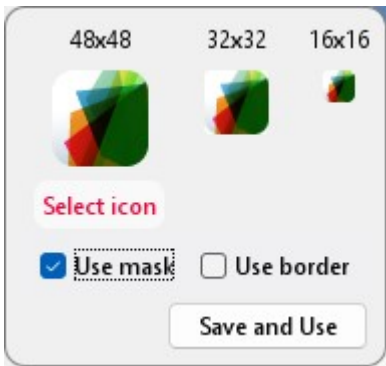
- Open *MATLAB Compiler*, located in the Apps tab.



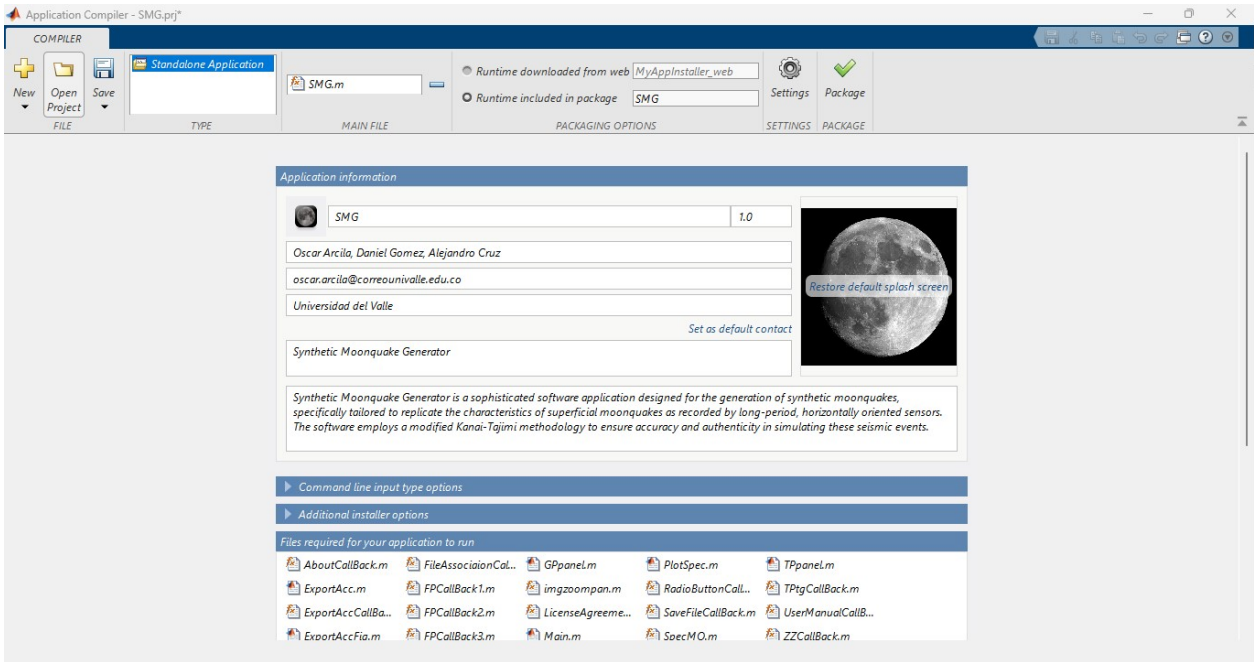
- Fill the MAIN FILE, PACKAGING OPTIONS, and Application information fields as shown in the figure.



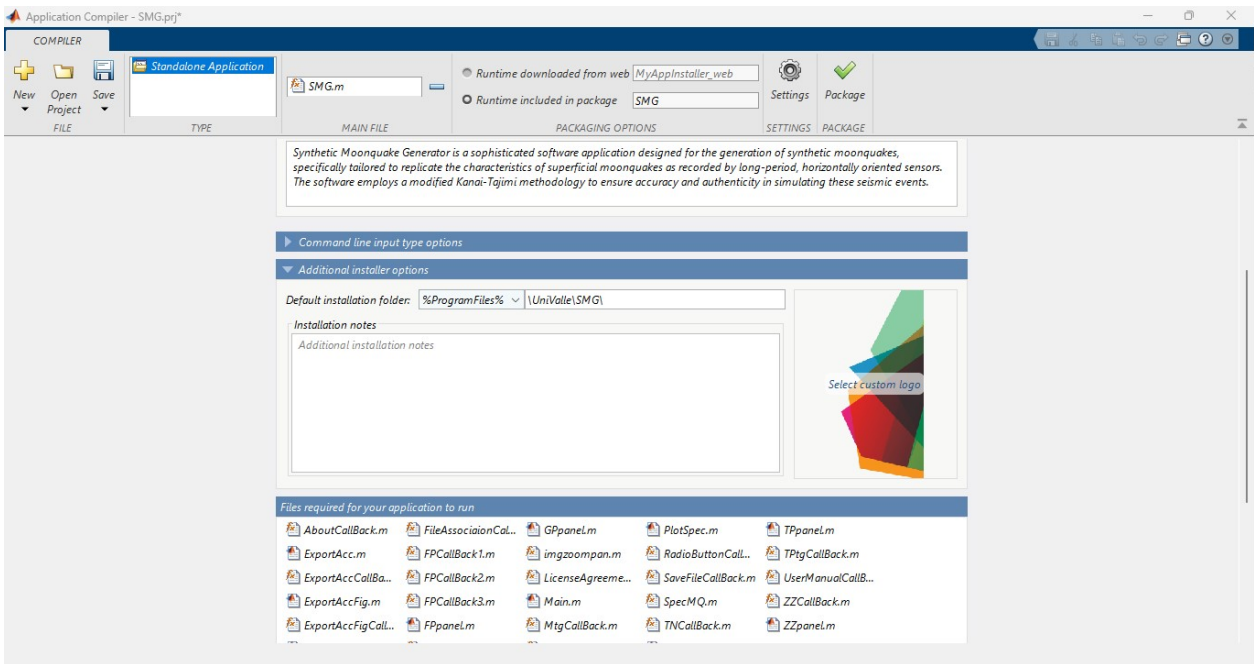
- Click the icon on the left and select the file *icon48.jpg* on the Images folder.



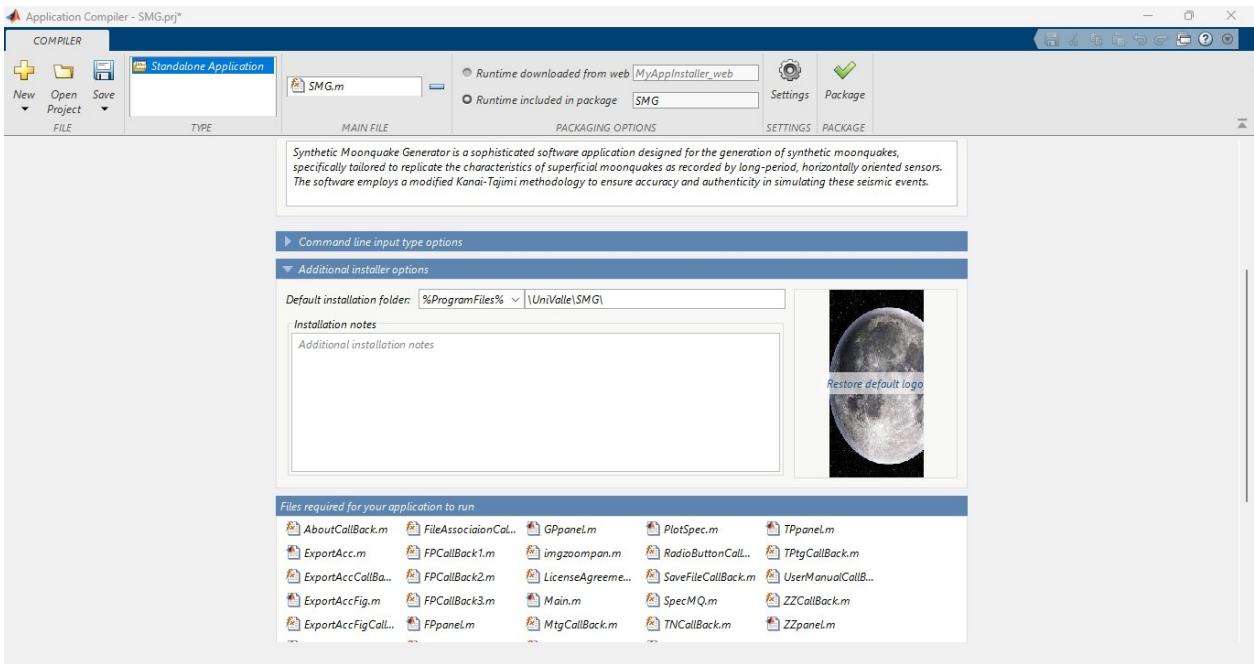
- Click the icon on the right and select the file *Splash.jpeg* on the Images folder.



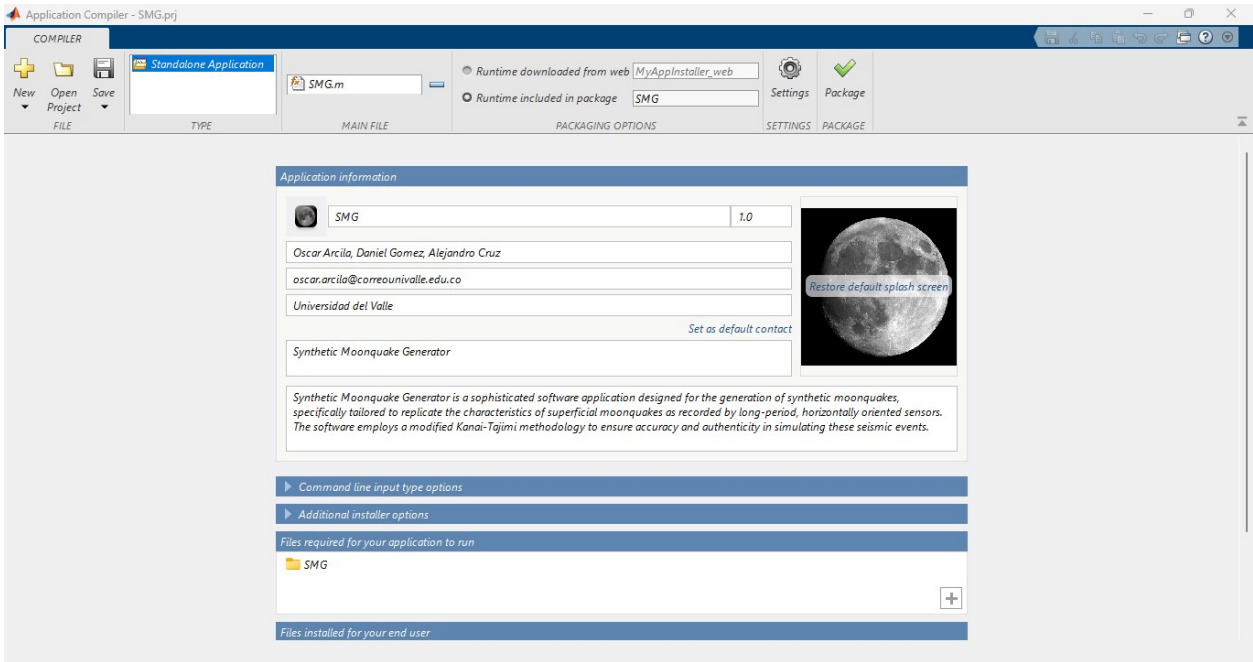
- Modify the Default installation folder whit `|UniValle\SMG|`.



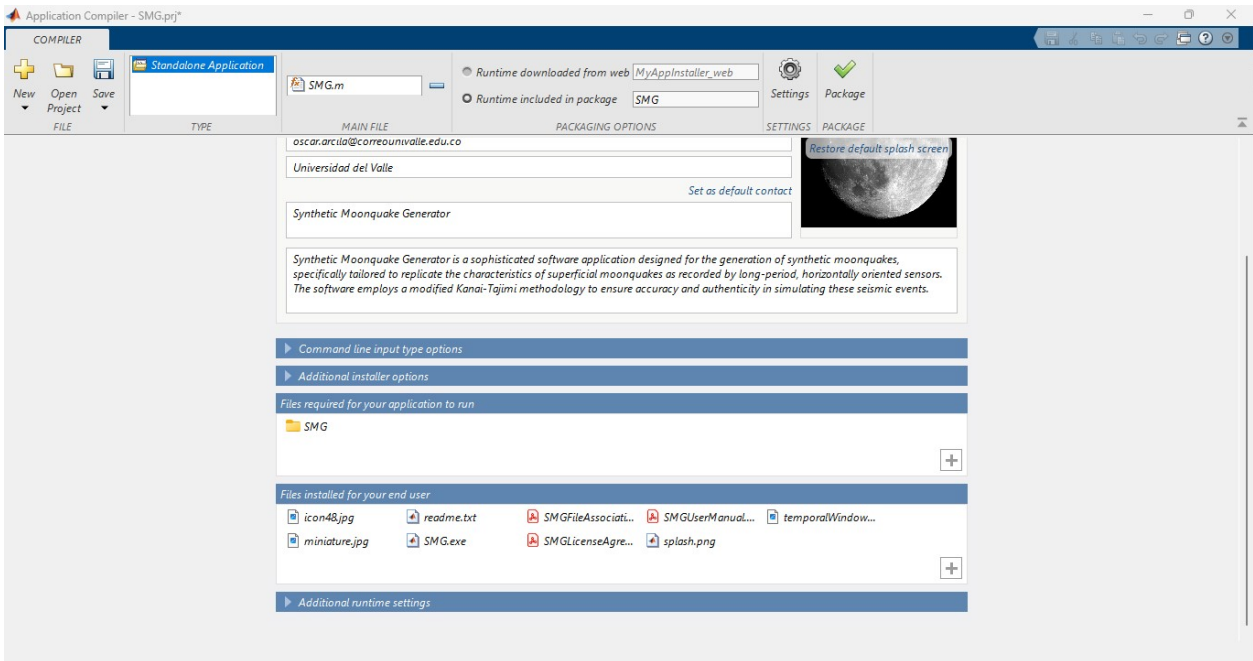
- Click the icon on the right (Select Custom Logo) and select the file `logo.jpeg` on the Images folder.



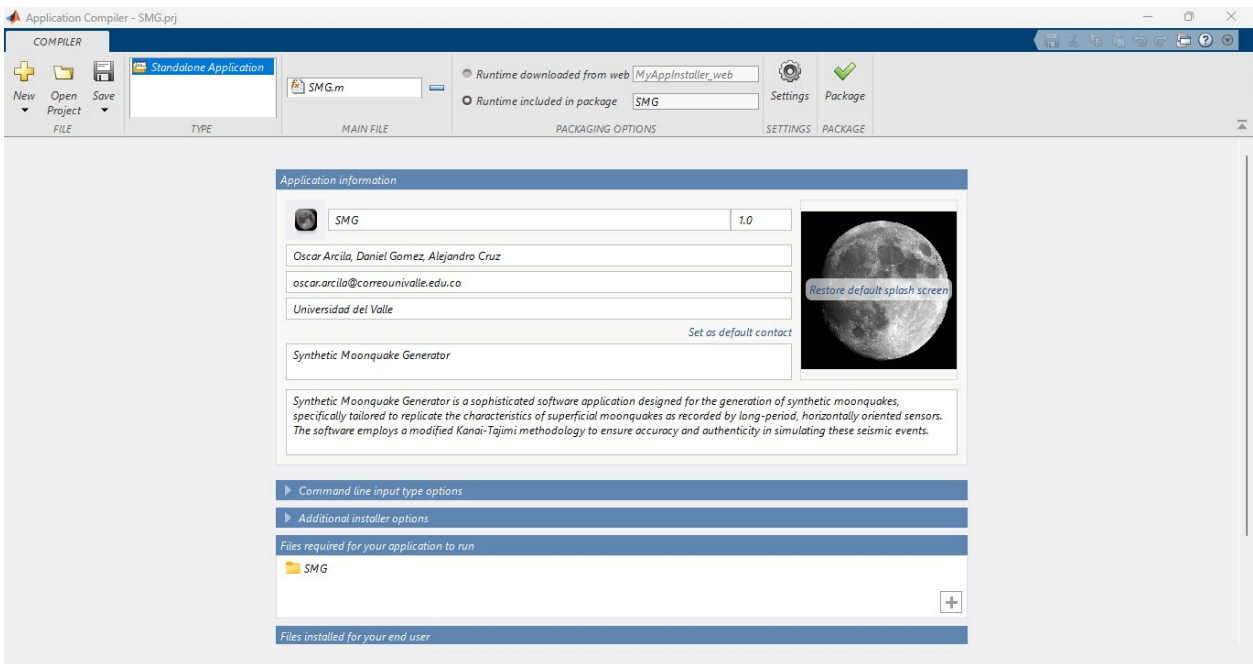
- In the Files required for your application to run add the code repository folder.



- In the Files installed for your end user add all the files of the *endUser* folder.



- Click on the package icon.



7 Requirements and Installation

7.1 Hardware Requirements

- Processor: 64 bits
- RAM: 8 GB or more
- DISK: 5 GB or more

7.2 Software Requirements

- OS: Windows 10 or posterior
- PDF file reader
- MATLAB is not necessary

7.3 Installation Process

- Find the installer *SMG.exe* and make right-click on it.



Figure 27: Executable icon: *SMG.exe*

- Select the option *Run as administrator*.

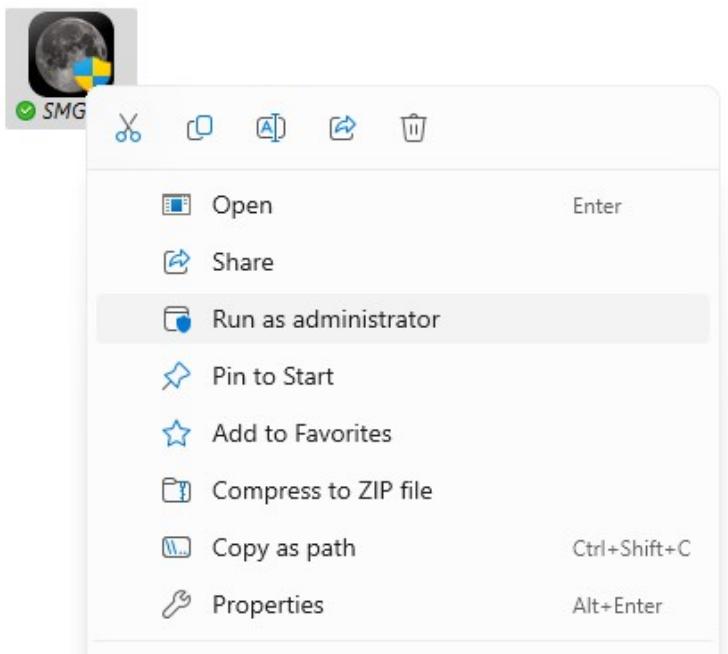


Figure 28: Option *Run as administrator* of the right click menu

- In the installer application (Fig. 29), make click on the *Next* button.

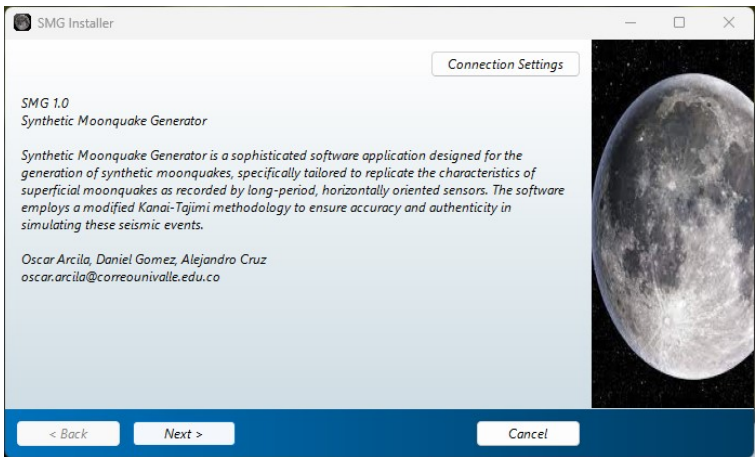


Figure 29: First screen of the installer application

- Check the box *Add a shortcut to the desktop* and click on the *Next* button.

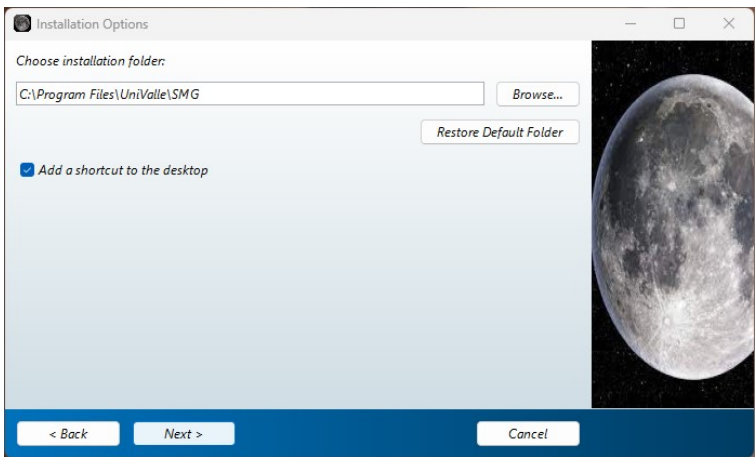


Figure 30: Install options

- Make click on the *Next* button.

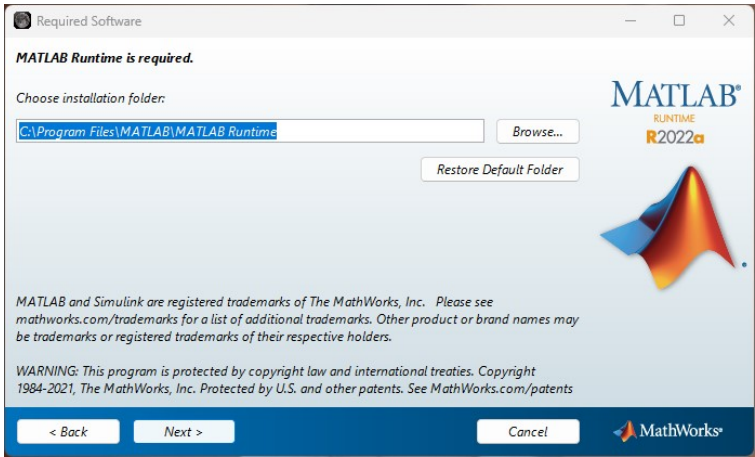


Figure 31: Required software

- Select the radio button *Yes*, and click the *Next* button.

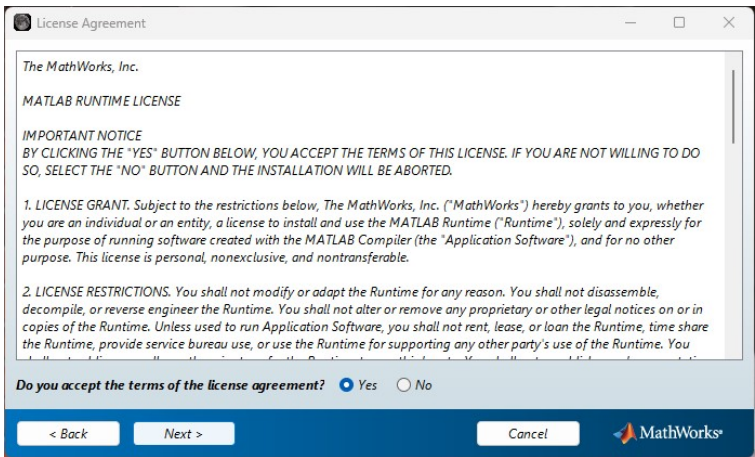


Figure 32: License Agreement for Matlab Runtime

- Click the *Install* button.

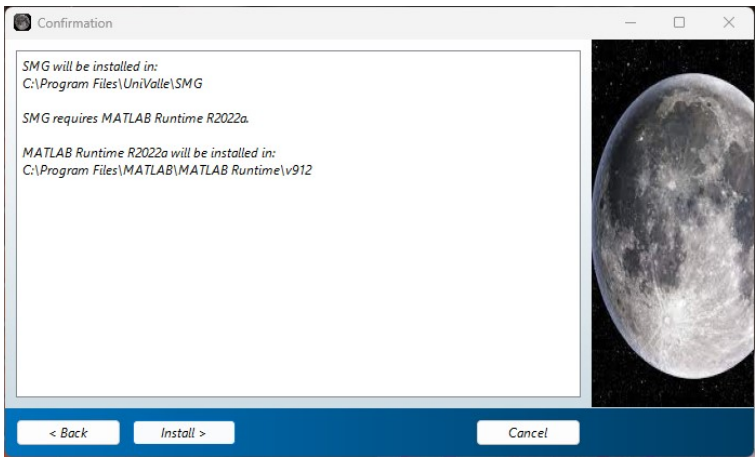


Figure 33: Confirmation

- Click the *Finish* button.

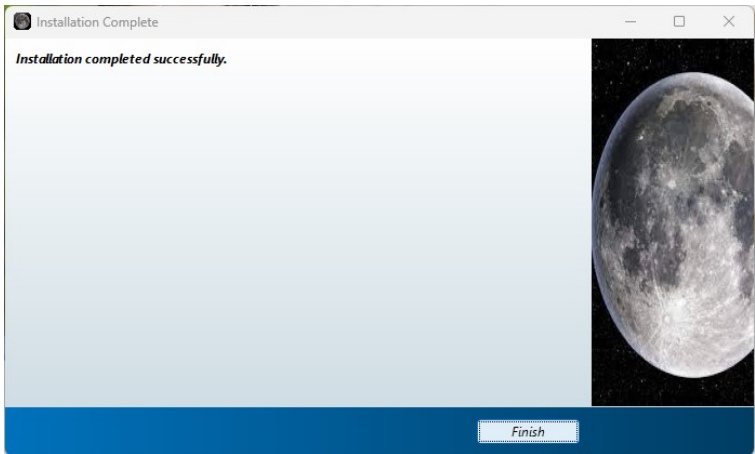


Figure 34: Install complete

7.4 File Association

The first step consists of creating a *.smg file using the *Save As* option of the application’s *File* menu. From this point, the steps are described below:

- Find the *.smg project, make right-click on it and select the *Open with* option of the deployed menu.

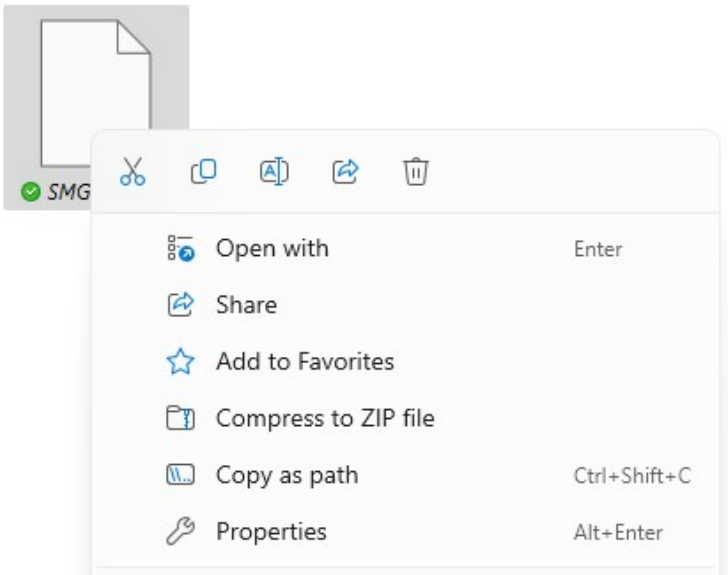


Figure 35: Right-click menu of the file *.smg

- Select the option *Choose an app on your PC*.

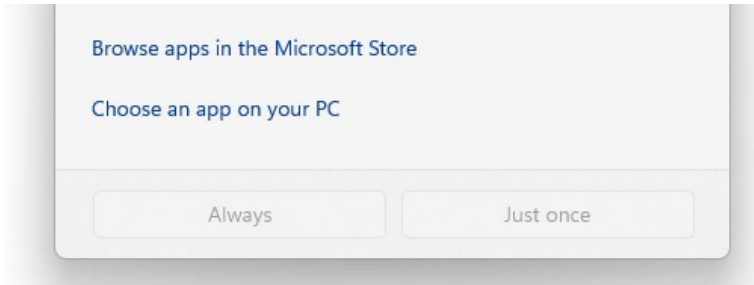


Figure 36: *Choose an app on your PC* option

- Find the *.exe* of the *SMG* application (By default is in: *C:\Program Files\UniValle\SMG\application*), and select the *SMG.exe* file.

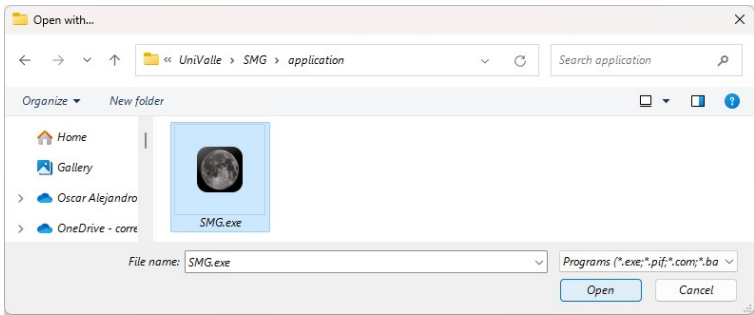


Figure 37: *SMG.exe* file on the install directory

- Select the application icon on the list and press the *Always* button.

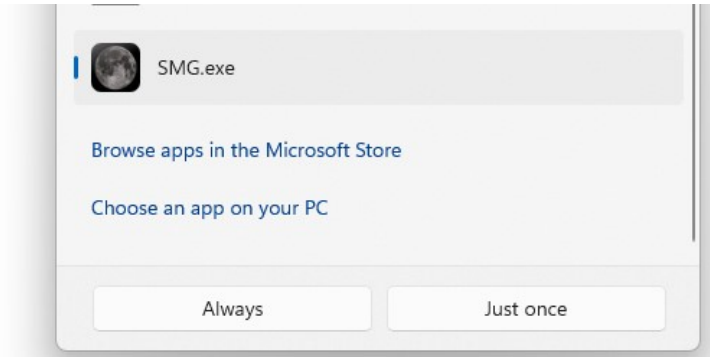
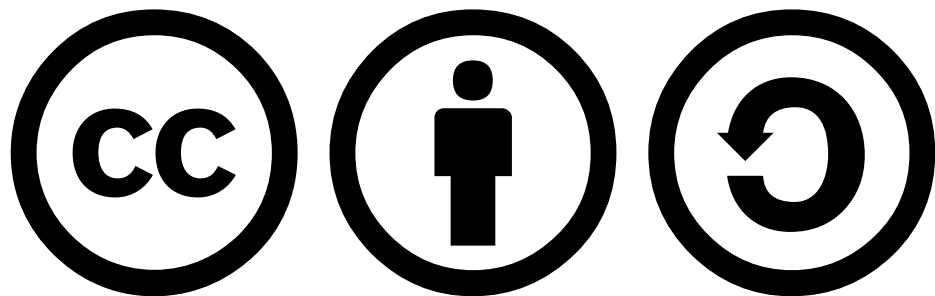


Figure 38: Final step of the file association

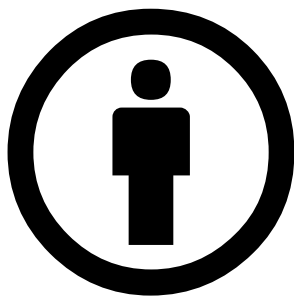
Once the association process is complete, you can double-click on a project file to open the *Synthetic Moonquake Generator* application and load the values associated with the project.

8 License Agreement

This work is licensed under a Creative Commons Attribution-ShareAlike 4.0 International License.



8.1 Attribution



This work contains unmodified code from the original work Image Mouse Zoom and Pan of Dany Cabrera published under a Creative Commons Attribution-ShareAlike 3.0 Unported License in the MATLAB FileSchange site: https://la.mathworks.com/matlabcentral/fileexchange/67583-image-mouse-zoom-and-pan?s_tid=blogs_rc_4

8.2 Attribution-ShareAlike 4.0 International

=====
Creative Commons Corporation ("Creative Commons") is not a law firm and does not provide legal services or legal advice. Distribution of Creative Commons public licenses does not create a lawyer-client or other relationship. Creative Commons makes its licenses and related information available on an "as-is" basis. Creative Commons gives no warranties regarding its licenses, any material licensed under their terms and conditions, or any related information. Creative Commons disclaims all liability for damages resulting from their use to the fullest extent possible.

Using Creative Commons Public Licenses

Creative Commons public licenses provide a standard set of terms and conditions that creators and other rights holders may use to share original works of authorship and other material subject to copyright and certain other rights specified in the public license below. The following considerations are for informational purposes only, are not exhaustive, and do not form part of our licenses.

Considerations for licensors: Our public licenses are intended for use by those authorized to give the public permission to use material in ways otherwise restricted by copyright and certain other rights. Our licenses are irrevocable. Licensors should read and understand the terms and conditions of the license they choose before applying it. Licensors should also secure all rights necessary before applying our licenses so that the public can reuse the material as expected. Licensors should clearly mark any material not subject to the license. This includes other CC- licensed material, or material used under an exception or limitation to copyright. More considerations for licensors: wiki.creativecommons.org/Considerations_for_licensors

Considerations for the public: By using one of our public licenses, a licensor grants the public permission to use the licensed material under specified terms and conditions. If the licensor's permission is not necessary for any reason—for example, because of any applicable exception or limitation to copyright—then that use is not regulated by the license. Our licenses grant only permissions under copyright and certain other rights that a licensor has authority to grant. Use of the licensed material may still be restricted for other reasons, including because others have copyright or other rights in the material. A licensor may make special requests, such as asking that all changes be marked or described. Although not required by our licenses, you are encouraged to respect those requests where reasonable. More considerations for the public: wiki.creativecommons.org/Considerations_for_licensees

=====
Creative Commons Attribution-ShareAlike 4.0 International Public License

By exercising the Licensed Rights (defined below), You accept and agree to be bound by the terms and conditions of this Creative Commons Attribution-ShareAlike 4.0 International Public License ("Public License"). To the extent this Public License may be interpreted as a contract, You are granted the Licensed Rights in consideration of Your acceptance of these terms and conditions, and the Licensor grants You such rights in consideration of benefits the Licensor receives from making the Licensed Material available under these terms and conditions.

Section 1 – Definitions.

a. Adapted Material means material subject to Copyright and Similar Rights that is derived from or based upon the Licensed Material and in which the Licensed Material is translated, altered, arranged, transformed, or otherwise modified in a manner requiring permission under the Copyright and Similar Rights held by the Licensor. For purposes of this Public License, where the Licensed Material is a musical work, performance, or sound recording, Adapted Material is always produced where the Licensed Material is synched in timed relation with a moving image.

b. Adapter’s License means the license You apply to Your Copyright and Similar Rights in Your contributions to Adapted Material in accordance with the terms and conditions of this Public License.

c. BY-SA Compatible License means a license listed at creativecommons.org/compatiblelicenses, approved by Creative Commons as essentially the equivalent of this Public License.

d. Copyright and Similar Rights means copyright and/or similar rights closely related to copyright including, without limitation, performance, broadcast, sound recording, and Sui Generis Database Rights, without regard to how the rights are labeled or categorized. For purposes of this Public License, the rights specified in Section 2(b)(1)-(2) are not Copyright and Similar Rights.

e. Effective Technological Measures means those measures that, in the absence of proper authority, may not be circumvented under laws fulfilling obligations under Article 11 of the WIPO Copyright Treaty adopted on December 20, 1996, and/or similar international agreements.

f. Exceptions and Limitations means fair use, fair dealing, and/or any other exception or limitation to Copyright and Similar Rights that applies to Your use of the Licensed Material.

g. License Elements means the license attributes listed in the name of a Creative Commons Public License. The License Elements of this Public License are Attribution and ShareAlike.

h. Licensed Material means the artistic or literary work, database, or other material to which the Licensors applied this Public License.

i. Licensed Rights means the rights granted to You subject to the terms and conditions of this Public License, which are limited to all Copyright and Similar Rights that apply to Your use of the Licensed Material and that the Licensors has authority to license.

j. Licensors means the individual(s) or entity(ies) granting rights under this Public License.

k. Share means to provide material to the public by any means or process that requires permission under the Licensed Rights, such as reproduction, public display, public performance, distribution, dissemination, communication, or importation, and to make material available to the public including in ways that members of the public may access the material from a place and at a time individually chosen by them.

l. Sui Generis Database Rights means rights other than copyright resulting from Directive 96/9/EC of the European Parliament and of the Council of 11 March 1996 on the legal protection of databases, as amended and/or succeeded, as well as other essentially equivalent rights anywhere in the world.

m. You means the individual or entity exercising the Licensed Rights under this Public License. Your has a corresponding meaning.

Section 2 – Scope.

a. License grant.

1. Subject to the terms and conditions of this Public License, the Licensors hereby grants You a worldwide, royalty-free, non-sublicensable, non-exclusive, irrevocable license to exercise the Licensed Rights in the Licensed Material to:

a. reproduce and Share the Licensed Material, in whole or in part; and

b. produce, reproduce, and Share Adapted Material.

2. Exceptions and Limitations. For the avoidance of doubt, where Exceptions and Limitations apply to Your use, this Public License does not apply, and You do not need to comply with its terms and conditions.

3. Term. The term of this Public License is specified in Section 6(a).

4. Media and formats; technical modifications allowed. The Licensors authorizes You to exercise the Licensed Rights in all media and formats whether now known or hereafter created, and to make technical modifications necessary to do so. The Licensors waives and/or agrees not to assert any right or authority to forbid You from making technical modifications necessary to exercise the Licensed Rights, including technical modifications necessary to circumvent Effective Technological Measures. For purposes of this Public License, simply making modifications authorized by this Section 2(a) (4) never produces Adapted Material.

5. Downstream recipients.

a. Offer from the Licensors – Licensed Material. Every recipient of the Licensed Material automatically receives an offer from the Licensors to exercise the Licensed Rights under the terms and conditions of this Public License.

b. Additional offer from the Licensors – Adapted Material. Every recipient of Adapted Material from You automatically receives an offer from the Licensors to exercise the Licensed Rights in the Adapted Material under the conditions of the Adapter’s License You apply.

c. No downstream restrictions. You may not offer or impose any additional or different terms or conditions on, or apply any Effective Technological Measures to, the Licensed Material if doing so restricts exercise of the Licensed Rights by any recipient of the Licensed Material.

6. No endorsement. Nothing in this Public License constitutes or may be construed as permission to assert or imply that You are, or that Your use of the Licensed Material is, connected with, or sponsored, endorsed, or granted official status by, the Licensor or others designated to receive attribution as provided in Section 3(a)(1)(A)(i).

b. Other rights.

1. Moral rights, such as the right of integrity, are not licensed under this Public License, nor are publicity, privacy, and/or other similar personality rights; however, to the extent possible, the Licensor waives and/or agrees not to assert any such rights held by the Licensor to the limited extent necessary to allow You to exercise the Licensed Rights, but not otherwise.

2. Patent and trademark rights are not licensed under this Public License.

3. To the extent possible, the Licensor waives any right to collect royalties from You for the exercise of the Licensed Rights, whether directly or through a collecting society under any voluntary or waivable statutory or compulsory licensing scheme. In all other cases the Licensor expressly reserves any right to collect such royalties.

Section 3 – License Conditions.

Your exercise of the Licensed Rights is expressly made subject to the following conditions.

a. Attribution.

1. If You Share the Licensed Material (including in modified form), You must:

a. retain the following if it is supplied by the Licensor with the Licensed Material:

i. identification of the creator(s) of the Licensed Material and any others designated to receive attribution, in any reasonable manner requested by the Licensor (including by pseudonym if designated);

ii. a copyright notice;

iii. a notice that refers to this Public License;

iv. a notice that refers to the disclaimer of warranties;

v. a URI or hyperlink to the Licensed Material to the extent reasonably practicable;

b. indicate if You modified the Licensed Material and retain an indication of any previous modifications; and

c. indicate the Licensed Material is licensed under this Public License, and include the text of, or the URI or hyperlink to, this Public License.

2. You may satisfy the conditions in Section 3(a)(1) in any reasonable manner based on the medium, means, and context in which You Share the Licensed Material. For example, it may be reasonable to satisfy the conditions by providing a URI or hyperlink to a resource that includes the required information.

3. If requested by the Licensor, You must remove any of the information required by Section 3(a)(1)(A) to the extent reasonably practicable.

b. ShareAlike.

In addition to the conditions in Section 3(a), if You Share Adapted Material You produce, the following conditions also apply.

1. The Adapter's License You apply must be a Creative Commons license with the same License Elements, this version or later, or a BY-SA Compatible License.

2. You must include the text of, or the URI or hyperlink to, the Adapter's License You apply. You may satisfy this condition in any reasonable manner based on the medium, means, and context in which You Share Adapted Material.

3. You may not offer or impose any additional or different terms or conditions on, or apply any Effective Technological Measures to, Adapted Material that restrict exercise of the rights granted under the Adapter's License You apply.

Section 4 – Sui Generis Database Rights.

Where the Licensed Rights include Sui Generis Database Rights that apply to Your use of the Licensed Material:

a. for the avoidance of doubt, Section 2(a)(1) grants You the right to extract, reuse, reproduce, and Share all or a substantial portion of the contents of the database;

b. if You include all or a substantial portion of the database contents in a database in which You have Sui Generis Database Rights, then the database in which You have Sui Generis Database Rights (but not its individual contents) is Adapted Material, including for purposes of Section 3(b); and

c. You must comply with the conditions in Section 3(a) if You Share all or a substantial portion of the contents of the database.

For the avoidance of doubt, this Section 4 supplements and does not replace Your obligations under this Public License where the Licensed Rights include other Copyright and Similar Rights.

Section 5 – Disclaimer of Warranties and Limitation of Liability.

a. UNLESS OTHERWISE SEPARATELY UNDERTAKEN BY THE LICENSOR, TO THE EXTENT POSSIBLE, THE LICENSOR OFFERS THE LICENSED MATERIAL AS-IS AND AS-AVAILABLE, AND MAKES NO REPRESENTATIONS OR WARRANTIES OF ANY KIND CONCERNING THE LICENSED MATERIAL, WHETHER EXPRESS, IMPLIED, STATUTORY, OR OTHER. THIS INCLUDES, WITHOUT LIMITATION, WARRANTIES OF TITLE, MERCHANTABILITY, FITNESS FOR A PARTICULAR PURPOSE, NON-INFRINGEMENT, ABSENCE OF LATENT OR OTHER DEFECTS, ACCURACY, OR THE PRESENCE OR ABSENCE OF ERRORS, WHETHER OR NOT KNOWN OR DISCOVERABLE. WHERE DISCLAIMERS OF WARRANTIES ARE NOT ALLOWED IN FULL OR IN PART, THIS DISCLAIMER MAY NOT APPLY TO YOU.

b. TO THE EXTENT POSSIBLE, IN NO EVENT WILL THE LICENSOR BE LIABLE TO YOU ON ANY LEGAL THEORY (INCLUDING, WITHOUT LIMITATION, NEGLIGENCE) OR OTHERWISE FOR ANY DIRECT, SPECIAL, INDIRECT, INCIDENTAL, CONSEQUENTIAL, PUNITIVE, EXEMPLARY, OR OTHER LOSSES, COSTS, EXPENSES, OR DAMAGES ARISING OUT OF THIS PUBLIC LICENSE OR USE OF THE LICENSED MATERIAL, EVEN IF THE LICENSOR HAS BEEN ADVISED OF THE POSSIBILITY OF SUCH LOSSES, COSTS, EXPENSES, OR DAMAGES. WHERE A LIMITATION OF LIABILITY IS NOT ALLOWED IN FULL OR IN PART, THIS LIMITATION MAY NOT APPLY TO YOU.

c. The disclaimer of warranties and limitation of liability provided above shall be interpreted in a manner that, to the extent possible, most closely approximates an absolute disclaimer and waiver of all liability.

Section 6 – Term and Termination.

a. This Public License applies for the term of the Copyright and Similar Rights licensed here. However, if You fail to comply with this Public License, then Your rights under this Public License terminate automatically.

b. Where Your right to use the Licensed Material has terminated under Section 6(a), it reinstates:

1. automatically as of the date the violation is cured, provided it is cured within 30 days of Your discovery of the violation; or
2. upon express reinstatement by the Licensor.

For the avoidance of doubt, this Section 6(b) does not affect any right the Licensor may have to seek remedies for Your violations of this Public License.

c. For the avoidance of doubt, the Licensor may also offer the Licensed Material under separate terms or conditions or stop distributing the Licensed Material at any time; however, doing so will not terminate this Public License.

d. Sections 1, 5, 6, 7, and 8 survive termination of this Public License.

Section 7 – Other Terms and Conditions.

a. The Licensor shall not be bound by any additional or different terms or conditions communicated by You unless expressly agreed.

b. Any arrangements, understandings, or agreements regarding the Licensed Material not stated herein are separate from and independent of the terms and conditions of this Public License.

Section 8 – Interpretation.

a. For the avoidance of doubt, this Public License does not, and shall not be interpreted to, reduce, limit, restrict, or impose conditions on any use of the Licensed Material that could lawfully be made without permission under this Public License.

b. To the extent possible, if any provision of this Public License is deemed unenforceable, it shall be automatically reformed to the minimum extent necessary to make it enforceable. If the provision cannot be reformed, it shall be severed from this Public License without affecting the enforceability of the remaining terms and conditions.

c. No term or condition of this Public License will be waived and no failure to comply consented to unless expressly agreed to by the Licensor.

d. Nothing in this Public License constitutes or may be interpreted as a limitation upon, or waiver of, any privileges and immunities that apply to the Licensor or You, including from the legal processes of any jurisdiction or authority.

=====
Creative Commons is not a party to its public licenses. Notwithstanding, Creative Commons may elect to apply one of its public licenses to material it publishes and in those instances will be considered the “Licensor.” The text of the Creative Commons public licenses is dedicated to the public domain under the CC0 Public Domain Dedication. Except for the limited purpose of indicating that material is shared under a Creative Commons public license or as otherwise permitted by the Creative Commons policies published at creativecommons.org/policies, Creative

Commons does not authorize the use of the trademark "Creative Commons" or any other trademark or logo of Creative Commons without its prior written consent including, without limitation, in connection with any unauthorized modifications to any of its public licenses or any other arrangements, understandings, or agreements concerning use of licensed material. For the avoidance of doubt, this paragraph does not form part of the public licenses.

Creative Commons may be contacted at creativecommons.org.

References

- Chopra, A. (2012). *Dynamics of structures: Theory and applications to earthquake engineering* (4th ed.). Prentice Hall.
- Clough, R., & Penzien, J. (2003). *Dynamics of structures* (Third). Computers; Structures inc.